

**RANCANG BANGUN PLATFORM LAYANAN FREELANCE DAN JOB
SHARING BERBASIS WEB DAN ANDROID**

TUGAS AKHIR

*Diajukan Sebagai Salah Satu Syarat Memperoleh Gelar Sarjana Pendidikan (S1) Pada
Departemen Teknik Elektronika Program Studi Pendidikan Teknik Informatika
Universitas Negeri Padang*



Fajri Nur Akbar
NIM. 22076069

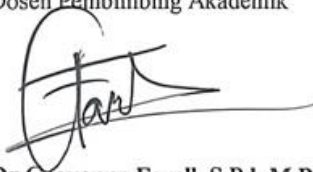
**PROGRAM STUDI PENDIDIKAN TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

HALAMAN PENGESAHAN DOSEN PEMBIMBING AKADEMIK

TUGAS AKHIR

*Diajukan Sebagai Salah Satu Syarat Memperoleh Gelar Sarjana Pendidikan (S1) Pada
Departemen Teknik Elektronika Program Studi Pendidikan Teknik Informatika Universitas
Negeri Padang*

Menyetujui,
Dosen Pembimbing Akademik



Dr. Geovanne Farell, S.Pd, M.Pd.T
NIDN. 0003029101

Penulis,
Mahasiswa,



Fairi Nur Akbar
NIM. 22076069

ABSTRAK

FAJRI NUR AKBAR.22076069 “Rancang bangun platform layanan freelance dan job sharing berbasis web dan android.” *Tugas Akhir*” Padang: Program Studi Informatika, Departemen Elektronika, Fakultas Teknik, Universitas Negeri Padang

Tingginya angka pengangguran dan keterbatasan akses terhadap informasi pekerjaan di Indonesia, khususnya di Sumatera Barat, menjadi permasalahan yang perlu diselesaikan melalui pendekatan teknologi digital. Belum tersedianya platform digital yang mengintegrasikan layanan *freelance* dan *job sharing* dalam satu sistem terpadu serta belum adanya mekanisme pembayaran jasa yang aman dan transparan antara pencari kerja dan pemberi kerja menjadi latar belakang penelitian ini. Penelitian ini bertujuan untuk merancang dan membangun platform digital ketenagakerjaan berbasis *web* dan *Android* yang mengintegrasikan layanan *freelance* dan *job sharing* secara terpadu, dilengkapi fitur pembayaran digital melalui *payment gateway* Midtrans. Metode pengembangan sistem yang digunakan adalah *Evolutionary Prototype*, yaitu model pengembangan perangkat lunak yang dilakukan secara bertahap dan iteratif dengan melibatkan pengguna dalam proses evaluasi sistem secara berkelanjutan. Platform dikembangkan menggunakan *Laravel* (PHP) untuk *backend* dan panel *web admin*, *Flutter* untuk aplikasi *Android*, serta *MySQL* sebagai basis data. Komunikasi antara aplikasi *Android* dan *backend* dilakukan melalui *REST API* dengan format *JSON* dan autentikasi berbasis *JSON Web Token* (JWT). Sistem pembayaran diintegrasikan menggunakan *Midtrans Snap API* yang mendukung transfer bank (*virtual account*), dompet digital (*e-wallet*), *QRIS*, dan kartu kredit. Platform memiliki empat jenis pengguna yaitu *Guest*, *Job Seeker*, *Employer*, dan *Administrator*. *Guest* dapat menelusuri daftar pekerjaan tanpa *login*, sementara pengguna yang telah *login* dapat memilih peran sebagai *Job Seeker* atau *Employer* secara bergantian dalam satu akun. Sistem pembayaran menerapkan biaya layanan otomatis sebesar 2,5% kepada *Employer* dan potongan 2% kepada *Job Seeker*, sehingga platform memperoleh pendapatan 4,5% dari setiap transaksi. Pengujian dilakukan menggunakan *Black Box Testing* dan *User Acceptance Testing* (UAT) dengan skala *Likert*.

Kata Kunci: *Android*, *Evolutionary Prototype*, *Freelance*, *Flutter*, *Job Sharing*, *Laravel*, *Midtrans*, *REST API*,

ABSTRACT

Fajri Nur Akbar. 220706969. Design and Development of a Web and Android-Based Freelance and Job Sharing Service Platform. *“Final Project”*. Informatics Engineering Education Study Program, Elektoronics Departement Faculty of Engineering, Universitas Negeri Padang.

The high unemployment rate and limited access to job information in Indonesia, particularly in West Sumatra, require a digital technology-based solution. The absence of an integrated platform combining freelance and job sharing services, along with the lack of a secure payment mechanism between job seekers and employers, served as the background for this research. This study aims to design and develop a web and Android-based digital employment platform that integrates freelance and job sharing services, equipped with a digital payment feature through the Midtrans payment gateway. The system was developed using the Evolutionary Prototype method, an incremental and iterative software development model that continuously involves users in the evaluation process. The platform was built using Laravel (PHP) for the backend and web admin panel, Flutter for the Android application, and MySQL as the database management system. Communication between the Android application and backend is conducted through a REST API with JSON format and JSON Web Token (JWT)-based authentication. The payment system is integrated using the Midtrans Snap API, supporting bank transfers (virtual account), digital wallets (e-wallet), QRIS, and credit cards. The platform serves four types of users: Guest, Job Seeker, Employer, and Administrator. Guests can browse job listings without logging in, while authenticated users can switch between Job Seeker and Employer roles using a single account. The payment system automatically applies a 2.5% service fee to Employers and a 2% deduction to Job Seekers, generating 4.5% platform revenue per transaction. System testing was conducted using Black Box Testing and User Acceptance Testing (UAT) with a Likert scale.

Keywords: Freelance, Job Sharing, Android, Laravel, Flutter, REST API, Midtrans, Evolutionary Prototype

KATA PENGANTAR

Puji syukur penulis ucapkan kehadiran Allah SWT yang telah melimpahkan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan proposal tugas akhir ini dengan judul **"Rancang Bangun Platform Layanan *Freelance* dan *Job Sharing* Berbasis Web dan Android"** sebagai salah satu syarat untuk menyelesaikan studi dan memperoleh gelar Sarjana Pendidikan pada Program Studi Pendidikan Teknik Informatika, Fakultas Teknik, Universitas Negeri Padang. Dalam penyusunan proposal tugas akhir ini, penulis menyadari bahwa keberhasilan dan kelancaran penulisan ini tidak lepas dari bantuan, bimbingan, dukungan, dan doa dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Muhammad, S.Pd., M.T. selaku Dekan Fakultas Teknik Universitas Negeri Padang.
2. Bapak Dr. Dedy Irfan, S.Pd., M.Kom. selaku Kepala Departemen Teknik Elektronika dan Koordinator Program Studi Pendidikan Teknik Informatika, Fakultas Teknik Universitas Negeri Padang.
3. Bapak Dr. Syafrijon, S.Pd., M.Kom. selaku dosen pembimbing Tugas Akhir
4. Bapak Dr. GEovanne Farrel, S.pd, M.Pd.T selaku dosen pembimbing akademik
5. Kedua orang tua tercinta serta keluarga yang selalu memberikan doa, dukungan, dan semangat kepada penulis.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang bersifat membangun sangat penulis harapkan demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat bagi pembaca serta menjadi tambahan pengetahuan khususnya dalam bidang teknologi dan informasi

Padang, 10 Mei 2026

Fajri nur akbar

22076069

DAFTAR ISI

ABSTRAK	i
ABSTRACT	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
DAFTAR TABEL	vii
DAFTAR GAMBAR.....	viii
BAB I. PENDAHULUAN	1
A. Latar Belakang Masalah	1
B. Identifikasi Masalah	6
C. Batasan Masalah	7
D. Rumusan Masalah	8
E. Tujuan Tugas Akhir	9
F. Manfaat Tugas Akhir	9
1. Manfaat teoritis	9
2. Manfaat praktis	9
3. Manfaat bagi institusi.....	10
BAB II. LANDASAN TEORI	11
A. Konsep Sistem Informasi	11
1. Pengertian sistem	11
2. pengertian informasi	13
B. Platform Digital Ketenagakerjaan	14
C. Freelance	16
D. Job Sharing	18
E. Website	19
F. Framework Laravel	21
G. Aplikasi Android	22
H. Application Programming Interface (API)	24
I. JSON (JavaScript Object Notation)	25
J. Arsitektur Client-Server	27
K. Basis Data (Database)	29
L. Sistem Autentikasi Berbasis Token	30
M. UML (Unified Modeling Language)	32

N. Rekayasa Perangkat Lunak.....	33
O. Pengujian Perangkat Lunak.....	35
BAB III. ANALISIS DAN PERANCANGAN SYSTEM	38
A. Analisis Sistem	38
1. Analisis Masalah.....	38
2. Analisis Kebutuhan Sistem	39
3. Analisis Pengguna.....	41
B. Metode Pengembangan Sistem.....	43
C. Perancangan Sistem	46
1. Arsitektur Sistem	46
2. Perancangan Use Case Diagram	47
3. Perancangan Activity Diagram	49
4. Perancangan Sequence Diagram.....	55
5. Perancangan Class Diagram	61
D. Perancangan Basis Data.....	63
1. Tabel users	63
2. Tabel jobs.....	64
3. Tabel applications	64
4. Tabel messages	65
5. Tabel personal_access_tokens	65
6. Tabel sessions	66
7. Tabel payments	66
8. Relasi Antar Tabel	67
E. Perancangan Antarmuka.....	69
1. Perancangan Antarmuka Aplikasi Android	69
2. Perancangan Antarmuka Website Admin.....	74
F. Perancangan API (Application Programming Interface).....	76
1. Autentikasi	77
2. Pekerjaan (Jobs).....	78
3. Lamaran (Applications)	79
4. Pesan (Messages).....	80
5. Profil (Profile).....	80
6. Admin (Panel Web Admin)	81
7. Enpoint pembayaran	82
8. Penanganan Respons API	83

G. Teknologi yang Digunakan	83
1. Flutter	84
2. Laravel	84
3. MySQL	85
4. JSON Web Token (JWT).....	86
5. Visual Studio Code	86
6. Postman.....	86
7. XAMPP.....	87
8. Android Studio.....	87
9. Git dan GitHub.....	88
10. Midtrans Payment Gateway	88
H. Rancangan Pengujian Sistem	89
1. Black Box Testing.....	90
2. User Acceptance Testing (UAT).....	94
DAFTAR PUSTAKA.....	98

DAFTAR TABEL

Tabel 1.users tabel	63
Tabel 2.jobs tabel.....	64
Tabel 3.Application tabel	64
Tabel 4.Messege tabel	65
Tabel 5.personal acces tokens tabel.....	65
Tabel 6.session tabel.....	66
Tabel 7.Payments table.....	66
Tabel 8.tabe Autentikasi Api	77
Tabel 9.Tabel pekerjaan Api	78
Tabel 10.tabel lamaran Api	79
Tabel 11.tabel pesan Api	80
Tabel 12.tabel Profil api	81
Tabel 13.tabel Admin Api	81
Tabel 14. tabel pembayaran Api.....	82
Tabel 15.tabel Pengujian Fitur Guest Mode	90
Tabel 16.tabel Pengujian Fitur Registrasi dan Login	91
Tabel 17.tabel fitur pemilihan peran.....	91
Tabel 18.tabel Pengujian Fitur Job Seeker	92
Tabel 19.tabel fitur employer	92
Tabel 20.tabel Pengujian Fitur Pesan	93
Tabel 21.tabel Pengujian Fitur Web Admin.....	93
Tabel 22.tabel aunthentikasi	94
Tabel 23.skala likert	95
Tabel 24. tabel Kisi-kisi UAT untuk Job Seeker.....	95
Tabel 25.tabel Kisi-kisi UAT untuk Employer	96
Tabel 26. tabel patokan kelayakan hasil UAT.....	96
Tabel 27. tabl ringkasan hasil pengujian	97

DAFTAR GAMBAR

gambar 1.use case digaram.....	48
gambar 2.activity diagram guest mode.....	49
gambar 3.Activity diagram login	50
gambar 4.Activity diagram pencarian pekerjaan.....	52
gambar 5.Activity diagram pengelolaan lowongan pekerjaan	53
gambar 6.Activity diagram pemebayaran jasa	54
gambar 7.sequence diagram guest-mode.....	56
gambar 8.sequence login dan pemeilihan peran	57
gambar 9.sequence diagram pencarian lamaran	58
gambar 10.sequence diagram pengelolaan lamaran	59
gambar 11.sequence diagram pembayaran jasa	60
gambar 12.use case diagram.....	62

BAB I.

PENDAHULUAN

A. Latar Belakang Masalah

Perkembangan teknologi digital telah membawa perubahan yang sangat signifikan dalam berbagai aspek kehidupan manusia, termasuk dalam bidang ketenagakerjaan. Transformasi digital yang ditandai dengan pesatnya perkembangan teknologi informasi dan komunikasi telah mengubah cara individu bekerja, berinteraksi, serta memperoleh penghasilan. Pola kerja yang sebelumnya didominasi oleh sistem kerja konvensional dengan jam kerja tetap dan lokasi kerja yang terpusat kini mulai bergeser menuju sistem kerja yang lebih fleksibel, berbasis teknologi, dan berorientasi pada hasil kerja. Pergeseran ini merupakan konsekuensi dari meningkatnya kebutuhan akan efisiensi, fleksibilitas waktu, serta tuntutan adaptasi terhadap dinamika ekonomi dan politik global.

pada era ekonomi modern , sistem kerja freelance serta fleksibel menjadi model ketenagakerjaan yang semakin banyak diminati dan di terapkan.sistem freelance memungkinkan penerima kerja dapat memberikan keahlian ,keterampilan dan jasa profesional kepada pemberi kerja tanpa harus terikat oleh kontrak kerja Panjang yang mengikat satu sama lain. Model kerja ini dinilai mampu menjawab kebutuhan industri yang memerlukan tenaga kerja dengan keahlian spesifik dalam waktu tertentu. Sementara itu, sistem job sharing memberikan kesempatan bagi dua atau lebih pekerja untuk berbagi tanggung jawab dalam satu posisi pekerjaan penuh waktu. Sistem ini tidak hanya meningkatkan fleksibilitas kerja, tetapi juga membuka peluang kerja bagi kelompok masyarakat yang memiliki keterbatasan waktu, seperti mahasiswa, ibu rumah tangga, dan pekerja paruh waktu. Dengan demikian, kedua sistem tersebut berpotensi menciptakan

ekosistem ketenagakerjaan yang lebih adaptif, inklusif, berkelanjutan dan diharapkan dapat menurunkan angka pengangguran dengan lebih signifikan .

Meskipun konsep freelance dan job sharing terus berkembang secara global, implementasinya di Indonesia masih menghadapi berbagai kendala. Salah satu permasalahan utama adalah keterbatasan platform digital yang mampu mengintegrasikan kedua sistem kerja tersebut secara terpadu dalam satu layanan serta kurangnya informasi, diharapkan platform ini dapat berkembang terlebih dahulu di kalangan mahasiswa yang sering kekurangan biaya. Sebagian besar platform ketenagakerjaan digital yang tersedia di Indonesia masih berfokus pada layanan freelance atau lowongan kerja konvensional secara terpisah. Selain itu, beberapa platform belum menyediakan dukungan lintas platform yang optimal antara web dan aplikasi mobile, sehingga pengalaman pengguna menjadi kurang maksimal, ditambah banyak penyedia lapangan kerja lebih memilih pekerja full time ketimbang pekerja freelance karena dianggap mempekerjakan pekerja freelance akan membuat administrasi pekerjaan dan system pekerjaan menjadi lebih rumit dan ruwet, Kondisi ini menyebabkan pencari kerja mengalami kesulitan dalam menemukan peluang kerja yang sesuai dengan kompetensi dan ketersediaan waktu, sementara pemberi kerja juga menghadapi kendala dalam menjangkau tenaga kerja yang tepat, mengelola komunikasi, serta memastikan keamanan transaksi dan data, dan mengatur administrasi pekerjaan.

Di sisi lain, perkembangan penggunaan perangkat mobile menunjukkan potensi besar dalam mendukung penyediaan layanan ketenagakerjaan berbasis digital. Smartphone, khususnya yang berbasis sistem operasi Android, telah menjadi perangkat utama yang digunakan masyarakat Indonesia dalam mengakses informasi dan layanan digital (Asosiasi Penyelenggara Jasa Internet Indonesia, 2025), tingkat penetrasi internet

di Indonesia mencapai 80,66%, meningkat dari 79,50% pada tahun 2024 dan 78,19% pada tahun 2023. Data tersebut menunjukkan adanya tren peningkatan pengguna internet yang konsisten setiap tahunnya. Selain itu, APJII juga mencatat bahwa lebih dari 70% pengguna internet di Indonesia mengakses layanan digital melalui perangkat mobile, yang menunjukkan tingginya ketergantungan masyarakat terhadap aplikasi berbasis smartphone.

Pada tingkat regional, sebagai contoh di Provinsi Sumatera Barat juga menunjukkan tingkat penetrasi internet yang relatif tinggi. Berdasarkan data APJII tahun 2025, tingkat penetrasi internet di Provinsi Sumatera Barat mencapai 79,45%, dengan mayoritas akses dilakukan melalui perangkat mobile. Tingginya tingkat penetrasi internet ini menunjukkan bahwa masyarakat Sumatera Barat memiliki kesiapan infrastruktur digital serta literasi teknologi yang memadai untuk memanfaatkan layanan berbasis web dan aplikasi Android. Kondisi ini menjadi peluang strategis dalam pengembangan platform digital yang berorientasi pada penyediaan layanan ketenagakerjaan.

Selain perkembangan teknologi, permasalahan ketenagakerjaan masih menjadi isu strategis di Indonesia. Berdasarkan data Survei Angkatan Kerja Nasional (SAKERNAS), jumlah pengangguran di Indonesia pada Agustus 2025 mencapai 7,46 juta orang atau sebesar 4,85% dari total angkatan kerja. Angka ini menunjukkan bahwa permasalahan pengangguran masih menjadi tantangan serius dalam pembangunan nasional. Lebih lanjut, data Badan Pusat Statistik (BPS) per Februari 2025 menunjukkan bahwa dari sepuluh provinsi di Pulau Sumatera, Provinsi Sumatera Barat menempati posisi kedua tertinggi tingkat pengangguran, yaitu sebesar 5,69%, berada di bawah Provinsi Kepulauan Riau dengan tingkat pengangguran sebesar 6,89%. Tingginya angka

pengangguran di Sumatera Barat mengindikasikan adanya ketidakseimbangan antara jumlah pencari kerja dengan ketersediaan lapangan pekerjaan yang ada.

Data yang telah diuraikan sebelumnya juga dirasakan secara langsung oleh penulis berdasarkan kondisi di lingkungan sekitar. Banyak lulusan baru maupun mahasiswa yang sedang menempuh semester akhir mengalami permasalahan keuangan akibat keterbatasan akses terhadap pekerjaan yang sesuai. Pada umumnya, pencarian pekerjaan formal menuntut persyaratan administrasi yang kompleks serta proses birokrasi yang relatif panjang, sehingga menyulitkan mahasiswa dan lulusan baru untuk segera memperoleh penghasilan. Kondisi tersebut menyebabkan sebagian dari mereka memilih untuk menganggur sementara atau menunggu panggilan wawancara dari perusahaan. Padahal, mahasiswa dan lulusan baru memiliki potensi untuk menawarkan jasa sesuai dengan bidang keahlian yang dimiliki melalui sistem kerja freelance. Apabila mahasiswa telah terlibat secara langsung dalam dunia kerja sejak masih menempuh pendidikan, maka mereka dapat memperoleh pengalaman praktis, meningkatkan keterampilan profesional, serta memahami kebutuhan industri yang tidak sepenuhnya diperoleh melalui proses pembelajaran di bangku perkuliahan. Selain itu, keterlibatan dalam pekerjaan freelance juga dapat membantu mahasiswa memperoleh penghasilan tambahan secara mandiri, sehingga dapat mengurangi ketergantungan terhadap dukungan finansial orang tua. Dengan demikian, sistem kerja freelance memberikan manfaat ganda, baik dalam peningkatan kompetensi maupun dalam pemenuhan kebutuhan ekonomi mahasiswa dan lulusan baru.

Keadaan tersebut menunjukkan perlunya inovasi dan solusi alternatif dalam penyediaan lapangan kerja, khususnya melalui pemanfaatan teknologi digital yang semakin berkembang pesat. Perkembangan teknologi informasi dan komunikasi telah

mengubah pola kerja masyarakat, dari sistem kerja konvensional menuju sistem kerja yang lebih fleksibel dan berbasis digital. Dalam konteks ini, platform ketenagakerjaan berbasis web dan Android dinilai mampu menjadi sarana yang efektif untuk memperluas akses masyarakat terhadap berbagai peluang kerja, terutama pekerjaan yang bersifat fleksibel dan berbasis keterampilan. Selain itu, pemanfaatan platform digital juga dapat mempercepat proses pencarian kerja, mempertemukan pencari kerja dan pemberi kerja secara lebih efisien, serta mengurangi keterbatasan ruang dan waktu dalam proses rekrutmen. Integrasi layanan freelance dan job sharing dalam satu platform diharapkan dapat menjawab kebutuhan pasar kerja modern yang menuntut fleksibilitas, efisiensi, dan kemudahan akses, sekaligus mendukung upaya pemerintah dan masyarakat dalam mengurangi tingkat pengangguran serta meningkatkan kesejahteraan ekonomi.

Berdasarkan uraian tersebut, penulis membangun sebuah platform layanan freelance dan job sharing berbasis web dan Android yang terintegrasi sebagai solusi atas permasalahan ketenagakerjaan yang ada. Platform ini dirancang dengan memanfaatkan teknologi *Application Programming Interface* (API) sebagai penghubung utama antara sistem web, aplikasi Android, dan basis data, sehingga pertukaran data dapat dilakukan secara terpusat dan terstruktur. Arsitektur sistem yang diterapkan adalah arsitektur *client-server*, di mana aplikasi web dan Android berperan sebagai klien yang mengakses layanan dari server melalui API menggunakan format pertukaran data JSON serta mekanisme autentikasi berbasis token. Penerapan arsitektur ini diharapkan mampu menjamin keamanan data pengguna, menjaga konsistensi informasi, meningkatkan keandalan sistem, serta memudahkan proses pengelolaan dan pengembangan aplikasi di masa mendatang. Dengan demikian, platform yang dibangun tidak hanya berfungsi sebagai media penyedia layanan kerja, tetapi juga sebagai sistem yang adaptif, skalabel, dan berkelanjutan untuk mendukung kebutuhan pasar kerja digital.

B. Identifikasi Masalah

Berdasarkan latar belakang yang telah diuraikan, maka dapat diidentifikasi beberapa permasalahan utama sebagai berikut:

1. Terbatasnya akses terhadap peluang kerja yang sesuai dan bersifat fleksibel, khususnya bagi mahasiswa semester akhir dan lulusan baru yang memiliki keterampilan tertentu, namun belum memperoleh pekerjaan tetap.
2. Masih tingginya tingkat pengangguran, terutama pada kelompok usia produktif, yang menunjukkan belum optimalnya penyerapan tenaga kerja oleh pasar kerja yang tersedia.
3. Belum tersedianya platform digital ketenagakerjaan yang terintegrasi di Indonesia yang secara khusus menggabungkan layanan freelance, job sharing, dan fulltime, dan partime dalam satu sistem, sehingga pengguna harus memanfaatkan platform yang berbeda untuk memenuhi kebutuhan kerja fleksibel yang beragam.
4. Belum optimalnya aksesibilitas dan perancangan antarmuka mobile, khususnya pada perangkat berbasis Android yang paling banyak digunakan masyarakat, sehingga menghambat proses pencarian, penawaran, dan pengelolaan pekerjaan secara cepat dan efisien.
5. Masih minimnya fitur pendukung kolaborasi dan manajemen kerja, seperti sistem komunikasi internal, pengelolaan tugas, serta pemantauan status pekerjaan, yang diperlukan untuk mendukung kelancaran kerja sama antara pencari kerja dan pemberi kerja, terutama dalam skema job sharing.
6. Adanya risiko keamanan dan rendahnya tingkat kepercayaan dalam transaksi kerja digital, yang disebabkan oleh belum optimalnya mekanisme verifikasi

identitas pengguna, sistem penilaian atau reputasi , serta pengelolaan transaksi dan data pengguna.

7. Belum optimalnya integrasi antara platform berbasis web dan aplikasi Android, sehingga menyebabkan ketidakkonsistenan pengalaman pengguna serta keterlambatan atau ketidaksinkronan data secara real-time.

Permasalahan-permasalahan tersebut menunjukkan perlunya suatu solusi teknologi berupa platform layanan yang terintegrasi, responsif, aman, dan dirancang khusus untuk mendukung dua model kerja fleksibel freelance dan job sharing melalui dua saluran utama: web dan Android.

C. Batasan Masalah

1. Penelitian ini berfokus pada perancangan dan pembangunan platform layanan freelance dan job sharing berbasis web dan Android sebagai solusi penyediaan lapangan kerja fleksibel.
2. Platform yang dibangun ditujukan untuk pengguna umum, dengan penekanan pada mahasiswa semester akhir dan lulusan baru sebagai target utama pengguna, tanpa membahas secara spesifik sektor industri tertentu.
3. Jenis pekerjaan yang difasilitasi dalam platform ini dibatasi pada pekerjaan berbasis keterampilan (skill-based work) yang dapat dilakukan secara fleksibel dan tidak terikat pada lokasi kerja tertentu.
4. Sistem yang dikembangkan mencakup fitur utama berupa manajemen akun pengguna, pencarian dan penawaran pekerjaan, komunikasi dasar antara pencari kerja dan pemberi kerja, serta pengelolaan pekerjaan freelance dan job sharing, tanpa membahas proses hukum ketenagakerjaan secara mendalam.

5. Aspek keamanan sistem dibatasi pada penerapan autentikasi pengguna, pengelolaan hak akses , dan perlindungan data dasar tanpa membahas implementasi sistem keamanan tingkat lanjut seperti enkripsi end to end.
6. Integrasi sistem dibatasi pada sinkronisasi data antara aplikasi web dan aplikasi Android melalui Application Programming Interface (API) dengan format pertukaran data JSON dan mekanisme autentikasi berbasis token.
7. Penelitian ini tidak membahas secara mendalam aspek finansial, kebijakan ketenagakerjaan nasional, maupun dampak ekonomi makro, melainkan berfokus pada aspek teknis dan fungsional pengembangan sistem.
8. Pengujian sistem dibatasi pada pengujian fungsionalitas dan uji coba penggunaan (user testing) secara terbatas, tanpa melakukan pengujian performa sistem dalam skala besar atau jangka panjang.

D. Rumusan Masalah

1. Bagaimana merancang dan membangun platform layanan freelance dan job sharing yang terintegrasi dalam satu sistem berbasis web dan Android?
2. Bagaimana mengimplementasikan Application Programming Interface (API) sebagai penghubung antara aplikasi web, aplikasi Android, dan basis data agar data dapat tersinkronisasi secara konsisten?
3. Bagaimana merancang sistem autentikasi pengguna berbasis token yang aman untuk proses registrasi dan login pada platform?
4. Bagaimana mengelola data pengguna dan data pekerjaan secara efektif melalui web base/admin?
5. Bagaimana memastikan platform yang dibangun dapat memberikan kemudahan akses, keamanan dasar dan pengalaman pengguna yang baik pada aplikasi

E. Tujuan Tugas Akhir

1. Membangun sebuah platform layanan freelance dan job sharing yang terintegrasi dalam satu sistem berbasis web dan Android.
2. Merancang dan mengimplementasikan Application Programming Interface (API) sebagai penghubung antara aplikasi web, aplikasi Android, dan basis data.
3. Mengimplementasikan sistem autentikasi pengguna berbasis token untuk mendukung proses registrasi dan login yang aman.
4. Mengembangkan web base/admin sebagai media pengelolaan data pengguna dan data pekerjaan secara terpusat.
5. Menghasilkan platform digital yang mudah diakses, terintegrasi, dan fungsional untuk mendukung kebutuhan kerja fleksibel berbasis freelance dan job sharing.

F. Manfaat Tugas Akhir**1. Manfaat teoritis**

- 1) Menambah wawasan dan pemahaman penulis dalam bidang pengembangan sistem informasi, khususnya dalam perancangan dan implementasi platform berbasis web dan Android.
- 2) Menjadi referensi akademik bagi mahasiswa atau peneliti selanjutnya yang ingin mengembangkan sistem layanan freelance dan job sharing berbasis teknologi digital.

2. Manfaat praktis

- 1) Memberikan solusi teknologi berupa platform layanan freelance dan job sharing yang dapat membantu pencari kerja dan pemberi kerja dalam menemukan dan mengelola pekerjaan secara lebih mudah dan efisien.

- 2) Memudahkan pengguna dalam mengakses layanan kerja fleksibel melalui aplikasi Android yang praktis dan mudah digunakan.
- 3) Membantu administrator dalam melakukan pengelolaan data pengguna dan data pekerjaan secara terpusat melalui web base/admin.

3. Manfaat bagi institusi

- 1) Menjadi salah satu bentuk kontribusi dalam pengembangan karya ilmiah dan inovasi teknologi di lingkungan Universitas Negeri Padang, khususnya pada Program Studi Pendidikan Teknik Informatika.
- 2) Dapat dijadikan sebagai bahan rujukan dalam pembelajaran maupun penelitian terkait pengembangan aplikasi berbasis web dan mobile.

BAB II.

LANDASAN TEORI

A. Konsep Sistem Informasi

Untuk mengetahui konsep system informasi maka kita perlu memahami apa itu system dan infromasi.

1. Pengertian sistem

Istilah sistem hingga saat ini digunakan hampir di seluruh cabang ilmu pengetahuan. Sebagai contoh ilmu biologi menjelaskan bahwa sistem organisme atau makhluk yang hidup terdiri dari sel-sel dan organ yang saling berkoneksi satu dengan yang lain. Bidang ilmu rekayasa merancang sistem mekanik, elektronik, dan perangkat lunak yang kompleks. Bahkan para ahli ilmu sosial dan ekonomi memperkenalkan konsep “sistem sosial” yang meliputi organisasi, komunitas, dan ekonomi. Ahli ekologi secara konsisten menggunakan istilah ekosistem yang merupakan saling keterhubungan antara organisme dengan lingkungan yang kompleks. Lalu sejak kapan istilah sistem pertama kali digunakan?.Sudah berabad-abad istilah sistem digunakan manusia, namun para ahli mengalami kesulitan melacak awal mula penggunaannya. Bukti sejarah menunjukkan Aristoteles, seorang filsuf Yunani, pernah menggunakan istilah “systema” yang mendeskripsikan pengaturan atau pengorganisasian benda-benda yang terstruktur. Aritstoteles menggunakan istilah sistem dalam disiplin ilmu biologi, fisika, dan metafisika. Saat ini hampir seluruh bidang ilmu menggunakan terminologi sistem. Bidang ilmu yang kemudian menggunakan istilah sistem adalah fisika untuk menjelaskan peralatan mekanis sehingga mulai dikenal istilah “mechanical system”. Penggunaan istilah system di dorong oleh perkembangan teknologi mekanika yang menggunakan suku cadang (spare-parts) dalam jumlah besar dan terkoneksi satu dengan yang lain.

Pemahaman manusia terhadap sistem semakin terbentuk ketika Galileo Galilei dan Isaac Newton pada abad ke-16 dan 17 mengembangkan pendekatan sistematis (systematic approach) untuk mempelajari bumi secara alamiah. Temuan kedua ilmuwan ini menyebabkan perumusan hukum dan teori yang menjelaskan perilaku sistem fisik semakin berkembang. Selanjutnya pada abad 19 istilah sistem mulai digunakan secara luas dalam dunia industri untuk mengembangkan large-scale system di dunia pabrik, seperti jaringan pabrik dan transportasi. Sistem industri dengan ukuran besar ini membutuhkan perencanaan, koordinasi, dan pengelolaan yang hati-hati agar dapat berfungsi secara efisien. Aktivitas dan jumlah manusia semakin besar, menyebabkan perkembangan sistem semakin kompleks. Agar tidak mengalami “kekacauan” maka perlu ada cara untuk mengendalikan dan mengkomunikasikan (control and communication). Bidang ilmu yang bertujuan menciptakan alat yang dapat melakukan pengendalian dan pengkomunikasian terhadap sistem ini disebut dengan cybernetic yang dapat menganalisis dan memahami seluruh jenis sistem. Akibatnya pada abad-20 bidang ilmu sibernetik dan teori sistem mulai digabungkan. Kebutuhan masyarakat akan data dan informasi yang tinggi di era modern menyebabkan sistem berkembang pesat digunakan pada ilmu komputer. Pakar computer telah berhasil menciptakan sistem yang kompleks untuk memproses informasi dan menjalankan tugas-tugas spesifik seperti sistem operasi, database, dan perangkat lunak.

Berdasarkan uraian perkembangan sistem tersebut, dapat disimpulkan sistem adalah sekumpulan komponen yang saling terkait dan “bekerja” atau “bergerak” bersama sama untuk mencapai tujuan. Apakah komponen yang saling terkait dan bergerak bersama sama tersebut? Beberapa pakar menyatakan komponen sistem adalah input, proses, output, dan umpan balik.(Heryana, 2024)

2. pengertian informasi

informasi adalah sekumpulan data fakta yang terorganisir atau diolah dengan cara tertentu sehingga mempunyai arti bagi penerima. data yang telah diolah menjadi suatu yang berguna bagi si penerima maksudnya yaitu dapat memberikan keterangan atau pengetahuan. informasi sangat penting pada suatu organisasi/instansi. informasi merupakan data yang telah dalam. di bentuk, ataupun di manipulasi sesuai dengan keperluan tertentu bagi penggunaannya. informasi merupakan suatu kumpulan data yang sudah di proses untuk memperoleh pengetahuan yang lebih berguna untuk mencapai suatu sasaran. suatu informasi dapat dikatakan bernilai apabila informasi tersebut memberikan suatu manfaat yang lebih di banding dengan kita hanya melihat data yang ada. (Soufitri, 2023)

sistem informasi adalah suatu sistem didalam suatu organisasi yang mempertemukan kebutuhan pengolahan transaksi harian, mendukung operasi, bersifat manajerial dan kegiatan strategi dari suatu organisasi yang menyediakan pihak luar tertentu dengan laporan – laporan yang diperlukan. (Cahyaningtyas & Iriyani, 2014)

sistem informasi merupakan sebuah kombinasi antara prosedur kerja, informasi, orang, dan teknologi informasi yang diorganisasikan untuk mencapai tujuan yang sama dalam sebuah organisasi. Sistem informasi selalu menggambarkan, merancang, mengimplementasikan dengan menggunakan proses perkembangan sistematis dan merancang sebuah sistem informasi berdasarkan analisa kebutuhan. Jadi bagian utama dari proses ini adalah mengetahui rancangan analisis sistem. Seluruh aktivitas utama dilibatkan dalam siklus perkembangan yang lengkap. Siklus perkembangan sistem informasi memiliki tahapan antara lain yaitu :

Pemeriksaan, Analisis, Rancangan, Mengimplementasikan, dan Pemeliharaan.(Hermawan & Hidayat, 2016)

Berdasarkan penjelasan dari para ahli dapat disimpulkan Sistem informasi merupakan suatu sistem yang terdiri dari komponen-komponen yang saling berinteraksi untuk mengumpulkan, mengolah, menyimpan, dan mendistribusikan informasi guna mendukung proses pengambilan keputusan serta pengendalian dalam suatu organisasi. Dalam konteks penelitian ini, sistem informasi digunakan sebagai landasan dalam pembangunan sebuah platform yang mampu mengelola data pengguna, data pekerjaan, serta memfasilitasi interaksi antara pencari kerja dan pemberi kerja secara terintegrasi dan efisien.

B. Platform Digital Ketenagakerjaan

Kata platform saat ini akan selalu disandingkan kata digital, lalu apa itu platform digital, mari kita lihat arti kata “Platform” dan “Digital”. menurut KBBI platform adalah rencana kerja atau program, sedangkan Digital berasal dari bahasa Yunani “digitus” artinya jari jemari yang melambangkan sistem perhitungan, menurut KBBI Digital adalah segala sesuatu yang berkaitan dengan angka, penomoran, atau sistem perhitungan biner (0 dan 1). secara umum, istilah ini merujuk pada teknologi modern berbasis internet dan komputer yang mengubah data analog menjadi data digital (elektronik) untuk mempermudah pemrosesan informasi.

Adapun pengertian Digital platform merupakan suatu media berbasis digital yang bertujuan untuk memudahkan kehidupan masyarakat di saat ini. Banyak platform digital yang digunakan oleh hampir semua masyarakat bahkan menjadi sebuah kebiasaan atau keseharian, misalnya platform social media seperti Facebook, Instagram, dan WhatsApp.(Zuraidah et al., 2021)

Platform digital adalah arsitektur teknologi yang memungkinkan pengembangan fungsi komputasinya dan memungkinkan integrasi platform teknologi informasi, komputasi, dan konektivitas yang tersedia untuk organisasi.(Xie et al., 2022)

Adapun jika di simpulkan platform digital adalah infrastruktur berbasis internet (situs web atau aplikasi) yang menghubungkan pengguna, memfasilitasi interaksi, transaksi, atau pertukaran informasi. Ini bertindak sebagai perantara yang mempertemukan berbagai pihak seperti penjual-pembeli atau penyedia layanan-pelanggan untuk berkolaborasi secara online dengan efisien.

Adapun ketenagakerjaan Menurut KBBI V, Ketenagakerjaan merupakan hal tenaga kerja. Dalam Undang-Undang No. 13 Tahun 2013 tentang ketenagakerjaan, disebutkan bahwa : “Ketenagakerjaan adalah segala sesuatu yang berkaitan dengan tenaga kerja baik pada waktu sebelum, selama dan sesudah masa kerja.” Ketenagakerjaan tidak selalu berhubungan dengan subjek, melainkan dengan berbagai faktor seperti sebelum masa kerja ada masalah kesempatan kerja yang sempit, lalu selama masa kerja ada masalah penggajian atau kualitas tenaga kerja yang rendah, dan sesudah masa kerja ada masalah pemenuhan hak pensiunan atau yang lainnya. Semua itu adalah bukti bahwa ketenagakerjaan menyangkut hal yang kompleks .(Manajemen, 2014)

Infrastruktur berbasis internet (situs web atau aplikasi) yang menghubungkan berbagai pihak dalam ekosistem ketenagakerjaan — mulai dari pencari kerja, pemberi kerja, hingga penyedia layanan pelatihan — guna memfasilitasi interaksi, transaksi, dan pertukaran informasi yang berkaitan dengan segala aspek tenaga kerja, baik sebelum, selama, maupun sesudah masa kerja, secara efisien dan digital.

Platform digital berperan sebagai perantara berbasis teknologi internet yang mempertemukan berbagai pihak secara online, sedangkan ketenagakerjaan mencakup

segala hal yang berhubungan dengan tenaga kerja, sehingga platform digital ketenagakerjaan adalah teknologi berbasis internet yang mempertemukan berbagai pihak berhubungan dengan ketenagakerjaan secara online.

Platform ketenagakerjaan modern umumnya menyediakan fitur seperti:

- 1) Pembuatan profil pengguna
- 2) Pencarian dan penawaran pekerjaan
- 3) Sistem komunikasi
- 4) Sistem penilaian atau reputasi

Dalam penelitian ini, platform yang dibangun mengintegrasikan dua model kerja, yaitu freelance dan job sharing, dalam satu sistem terpadu.

C. Freelance

Istilah freelance berasal dari bahasa Inggris yang pertama kali diperkenalkan oleh Sir Walter Scott (1771-1832) dari Britania Raya. Freelance terdiri dari kata free (bebas) dan lance (tombak) yang artinya tombak yang bebas. Menunjukkan bahwa tombak tidak disumpah untuk melayani majikan apapun, bukan bahwa tombak tersedia gratis. Pengertian lain dari freelance adalah seseorang yang bekerja sendiri dan tidak berkomitmen kepada majikan jangka panjang tertentu. Menurut Wikipedia, freelance (pekerja lepas) adalah seseorang yang bekerja sendiri dan tidak berkomitmen kepada majikan dalam jangka panjang tertentu.

menurut (Indriana, 2021), Freelance merupakan pekerjaan paruh waktu atau pekerjaan lepas atau istilah yang biasa digunakan untuk orang yang bekerja sendiri dan tidak harus berkomitmen untuk jangka panjang kepada perusahaan, owner bisnis maupun pemilik usaha tertentu sehingga kerjaan sebagai freelance

tidak bersifat terikat dengan aturan perusahaan. Dan orang yang melakukan pekerjaan freelance disebut sebagai freelancer

Para pekerja lepas (freelancer) atau tenaga lepas dapat disebut juga dengan istilah *ondemand worker* atau pekerja yang mau bekerja dan dapat dibutuhkan kapan saja (Silitonga, 2018). Secara umum, *freelance* merupakan sistem kerja di mana seseorang bekerja secara mandiri dengan menawarkan jasa atau keahlian kepada klien tanpa terikat kontrak jangka panjang. Pekerja *freelance* biasanya dibayar berdasarkan proyek atau pekerjaan yang diselesaikan. Adapun beberapa bidang pekerjaan yang umum digeluti oleh para *freelancer* antara lain desain grafis, pengembangan perangkat lunak (*web/app developer*), penulisan konten (*content writer*), fotografi, penerjemahan, hingga konsultasi bisnis. Perkembangan teknologi digital turut memperluas ekosistem *freelance* secara signifikan. Kehadiran platform digital seperti Upwork, Fiverr, dan Fastwork memudahkan *freelancer* untuk terhubung dengan klien dari berbagai penjuru dunia tanpa batasan geografis. Di Indonesia sendiri, tren *freelance* terus mengalami pertumbuhan seiring meningkatnya penetrasi internet dan pergeseran budaya kerja yang lebih fleksibel, terutama setelah pandemi COVID-19 yang mendorong banyak tenaga kerja beralih ke model kerja mandiri. Berdasarkan uraian tersebut, dapat disimpulkan bahwa *freelance* adalah sistem kerja mandiri yang memberikan kebebasan bagi individu dalam menentukan klien, waktu kerja, dan jenis proyek yang dikerjakan, tanpa adanya ikatan kontrak jangka panjang dengan satu perusahaan atau pemberi kerja tertentu.

Adapun Karakteristik freelance Tidak terikat waktu kerja tetap, Tidak terikat pada satu perusahaan, Berbasis proyek atau tugas, Fleksibel dalam memilih pekerjaan. Sistem freelance sangat relevan dengan perkembangan ekonomi digital karena memberikan fleksibilitas bagi pekerja serta efisiensi bagi pemberi kerja.

D. Job Sharing

Job sharing atau dalam bahasa Indonesia disebut pembagian kerja merupakan salah satu bentuk sistem kerja fleksibel yang mulai banyak diterapkan oleh perusahaan modern. Menurut (Dessler, 2015), *job sharing* adalah pengaturan kerja di mana dua orang atau lebih berbagi satu pekerjaan penuh waktu, dengan membagi jam kerja, tanggung jawab, dan kompensasi secara proporsional. Sistem ini hadir sebagai respons atas kebutuhan pekerja akan fleksibilitas waktu sekaligus kebutuhan perusahaan dalam mempertahankan produktivitas kerja.

Secara operasional, *job sharing* mengatur dua karyawan untuk mengerjakan satu posisi atau jabatan secara bersama-sama. Kedua karyawan tersebut memiliki tanggung jawab yang setara atas pekerjaan yang diemban, namun masing-masing mengerjakan bagian yang menjadi keahlian atau jadwal yang telah disepakati. Dengan demikian, satu posisi penuh waktu (*full-time*) dapat dikerjakan oleh dua karyawan *part-time* yang saling melengkapi.

Penerapan *job sharing* memberikan sejumlah manfaat bagi kedua belah pihak. Bagi karyawan, sistem ini memberikan fleksibilitas waktu sehingga memungkinkan keseimbangan antara kehidupan kerja dan kehidupan pribadi (*work-life balance*). Bagi perusahaan, *job sharing* dapat menjadi solusi efisiensi biaya, terutama pada kondisi ekonomi yang tidak stabil, karena perusahaan tetap mendapatkan output kerja yang optimal tanpa harus menanggung biaya penuh untuk dua posisi yang terpisah.

Meskipun demikian, penerapan *job sharing* tidak dapat dilakukan secara sembarangan. Perusahaan perlu melakukan analisis mendalam mengenai kondisi dan kebutuhan tenaga kerja, termasuk aspek penggajian, penilaian kinerja, serta mekanisme koordinasi antara kedua karyawan yang berbagi pekerjaan. Keberhasilan sistem ini

sangat bergantung pada komunikasi yang baik antara kedua pekerja serta kejelasan pembagian tugas yang telah ditetapkan sejak awal.

Berdasarkan uraian di atas, dapat disimpulkan bahwa *job sharing* merupakan sistem pembagian kerja fleksibel di mana dua orang karyawan berbagi satu posisi penuh waktu dengan tanggung jawab, jam kerja, dan kompensasi yang dibagi secara proporsional. Sistem ini relevan dalam konteks manajemen sumber daya manusia modern yang semakin mengutamakan fleksibilitas dan efisiensi dalam pengelolaan tenaga kerja. Adapun Keunggulan *job sharing* dapat Meningkatkan fleksibilitas kerja Membuka peluang kerja bagi lebih banyak orang Mengurangi beban kerja individu Mendukung keseimbangan kehidupan dan pekerjaan (*work-life balance*)

Dalam penelitian ini, *job sharing* diintegrasikan dalam platform untuk memberikan alternatif kerja fleksibel selain freelance.

E. Website

Website atau yang sering disebut dengan situs web merupakan salah satu produk dari perkembangan teknologi internet yang saat ini telah menjadi bagian penting dalam kehidupan sehari-hari. Menurut (Dessler, 2015), *website* adalah kumpulan halaman yang digunakan untuk menampilkan informasi berupa teks, gambar, animasi, suara, atau gabungan dari semuanya, baik yang bersifat statis maupun dinamis, yang membentuk satu rangkaian saling terkait dan masing-masing dihubungkan dengan jaringan halaman. Dengan demikian, *website* bukan sekadar satu halaman tunggal, melainkan sebuah sistem halaman yang saling terhubung dan membentuk satu kesatuan informasi yang utuh.

Senada dengan hal tersebut, (Yuhefizar, 2013) mendefinisikan *website* sebagai keseluruhan halaman-halaman web yang terdapat dalam sebuah domain yang

mengandung informasi, di mana *website* biasanya dibangun atas banyak halaman web yang saling terkait satu sama lain. Sementara itu, (Abdulloh, 2016) menambahkan bahwa *website* merupakan kumpulan halaman yang berisi informasi dalam bentuk data digital seperti teks, gambar, animasi, suara, dan video, yang disediakan melalui koneksi internet sehingga dapat diakses oleh semua orang di seluruh dunia.

Berdasarkan sifatnya, *website* dibedakan menjadi dua jenis, yaitu *website* statis dan *website* dinamis. *Website* statis adalah jenis *website* yang isinya bersifat tetap dan hanya dapat diubah melalui perubahan kode programnya secara langsung, sehingga informasi yang ditampilkan bersifat satu arah dari pemilik *website*. Sebaliknya, *website* dinamis adalah jenis *website* yang menyediakan kemudahan bagi pengguna yang memiliki hak akses untuk memperbarui, menambah, atau menghapus konten tanpa harus mengubah kode program secara langsung (Miftahuljannah & Suharso, 2023). Contoh *website* statis antara lain halaman profil perusahaan sederhana, sedangkan contoh *website* dinamis meliputi portal berita, toko *online*, dan media sosial.

Dalam perkembangannya, *website* tidak hanya berfungsi sebagai media penyampaian informasi semata, tetapi telah berkembang menjadi platform multifungsi yang mendukung berbagai kebutuhan seperti komunikasi, transaksi bisnis, pendidikan, hiburan, hingga pemasaran digital. Hal ini sejalan dengan pernyataan dalam jurnal INFOTECH (2023) bahwa *website* merupakan kumpulan dokumen yang berada dalam sebuah domain atau subdomain di dalam *World Wide Web* (WWW), yang perkembangannya telah menjadikan teknologi internet sebagai alat komunikasi utama yang sangat berguna bagi masyarakat luas.

Berdasarkan berbagai definisi di atas, dapat disimpulkan bahwa *website* adalah kumpulan halaman web yang saling terhubung dalam sebuah domain, berisi informasi

dalam berbagai bentuk media digital, dan dapat diakses oleh siapapun melalui jaringan internet. Dalam konteks penelitian ini, *website* berperan sebagai media utama yang digunakan untuk menyajikan informasi dan layanan secara digital kepada pengguna secara luas.

Dalam sistem ini, website berfungsi sebagai media admin (web base/admin), Pengelolaan data pengguna dan pekerjaan, Monitoring dan aktivitas sistem

F. Framework Laravel

Laravel merupakan salah satu framework PHP berbasis Model View Controller (MVC) yang banyak digunakan dalam pengembangan aplikasi web modern. Laravel dikembangkan oleh Taylor Otwell dan dirancang untuk mempermudah proses pengembangan aplikasi dengan sintaks yang sederhana, elegan, dan terstruktur. Menurut (Herdiansah, 2024), Laravel adalah framework PHP yang menyediakan berbagai fitur pendukung pengembangan aplikasi seperti routing, middleware, autentikasi, migrasi database, dan Object Relational Mapping (ORM) sehingga proses pembangunan sistem dapat dilakukan secara lebih cepat dan efisien. Dengan konsep MVC, Laravel mampu memisahkan logika program, tampilan, dan pengolahan data sehingga kode program menjadi lebih rapi dan mudah dipelihara.

Dalam pengembangan sistem informasi, Laravel banyak digunakan karena mendukung pembuatan aplikasi berbasis web dan REST API secara terintegrasi. Menurut (Apriliando, 2021), Laravel memiliki keunggulan dalam pengembangan backend karena menyediakan sistem keamanan yang baik, manajemen database yang terstruktur, serta dukungan API yang memudahkan komunikasi data antara server dan aplikasi client seperti Android maupun website. Selain itu, Laravel juga menyediakan fitur Artisan Command Line Interface (CLI) yang membantu pengembang dalam mengotomatisasi

berbagai proses pengembangan aplikasi seperti pembuatan controller, model, migration, dan route. Dukungan komunitas yang besar dan dokumentasi yang lengkap menjadikan Laravel sebagai salah satu framework PHP yang paling populer dalam pengembangan aplikasi berbasis web.

Berdasarkan uraian di atas, dapat disimpulkan bahwa Laravel merupakan framework PHP modern yang digunakan untuk membangun aplikasi web secara terstruktur, aman, dan efisien. Dalam penelitian ini, Laravel digunakan sebagai framework backend untuk mengelola proses bisnis sistem, pengolahan data, autentikasi pengguna, serta penyediaan REST API yang menghubungkan aplikasi Android dengan database server secara real-time. Penggunaan Laravel diharapkan mampu meningkatkan efisiensi pengembangan sistem serta mempermudah proses pemeliharaan dan pengembangan aplikasi di masa mendatang.

G. Aplikasi Android

Android pertama kali dikembangkan oleh perusahaan Android Inc. pada tahun 2003, kemudian diakuisisi oleh Google pada tahun 2005 dan resmi diperkenalkan kepada publik pada tahun 2008. Sejak saat itu, Android berkembang pesat dan menjadi sistem operasi *mobile* paling banyak digunakan di seluruh dunia. Menurut (Nazruddin Safaat, 2012), Android adalah sebuah sistem operasi untuk perangkat *mobile* berbasis Linux yang mencakup sistem operasi, *middleware*, dan aplikasi. Dengan arsitektur berbasis Linux, Android dirancang untuk memberikan stabilitas, keamanan, dan fleksibilitas tinggi dalam pengoperasian perangkat *mobile*.

Senada dengan hal tersebut, (Kasman, 2013) mendefinisikan Android sebagai sebuah sistem operasi telepon seluler dan komputer tablet layar sentuh (*touchscreen*) yang berbasis Linux. Sementara itu, (Arifianto, 2011) menyatakan bahwa Android

merupakan perangkat bergerak pada sistem operasi untuk telepon seluler yang berbasis Linux. Dari beberapa definisi tersebut, dapat dipahami bahwa Android merupakan platform sistem operasi berbasis Linux yang dirancang khusus untuk perangkat bergerak (*mobile*) seperti smartphone dan tablet.

Salah satu keunggulan utama Android adalah sifatnya yang *open source*, yaitu kode sumbernya dapat diakses, dimodifikasi, dan dikembangkan secara bebas oleh siapapun. Android merupakan *operating system* dengan kebijakan terbuka yang dikembangkan oleh Google Inc., sehingga produsen *gadget* bebas menggunakan sistem operasi Android pada produknya (Universitas Islam Indonesia, 2024). Kebijakan *open source* inilah yang mendorong pertumbuhan ekosistem Android secara masif, baik dari sisi produsen perangkat maupun pengembang aplikasi dari seluruh dunia.

Aplikasi Android adalah perangkat lunak yang dirancang dan dijalankan pada sistem operasi Android. Pengembangan aplikasi Android umumnya menggunakan bahasa pemrograman Java atau Kotlin dengan memanfaatkan *Android Software Development Kit* (SDK) yang disediakan oleh Google. Berdasarkan kajian literatur yang dilakukan oleh (Wahyudi, 2022) dalam jurnal IJCS BSI , pengembangan aplikasi berbasis Android telah memudahkan banyak orang dalam mengakses informasi, dan sektor pendidikan menjadi sektor yang paling banyak memanfaatkan pengembangan aplikasi Android untuk membantu kinerja pengguna, disusul oleh sektor pelayanan pemerintah dan swasta.

Berdasarkan uraian di atas, dapat disimpulkan bahwa Android merupakan sistem operasi berbasis Linux yang dikembangkan untuk perangkat *mobile* seperti *smartphone* dan tablet. Android memungkinkan para pengembang untuk membuat aplikasi yang dapat diakses secara mudah oleh pengguna. Menurut Android Developers (2023),

Android menyediakan berbagai *tools* dan API (*Application Programming Interface*) yang mendukung pengembangan aplikasi *mobile* yang interaktif dan responsif.

Dalam penelitian ini, aplikasi Android digunakan sebagai media utama bagi pengguna (*user*) dalam mengakses fitur pencarian dan pengelolaan pekerjaan, serta sebagai sarana interaksi antara pencari kerja dan pemberi kerja secara digital.

H. Application Programming Interface (API)

Application Programming Interface (API) merupakan salah satu komponen penting dalam pengembangan sistem perangkat lunak modern, khususnya dalam membangun konektivitas antara berbagai aplikasi yang berbeda. Menurut (Triawan & Siboro, 2021), API atau *Application Programming Interface* merupakan teknologi antarmuka yang dibangun oleh pengembang sistem agar sebagian atau keseluruhan fungsi sistem dapat diakses secara bersamaan dengan baik. Dengan demikian, API berperan sebagai jembatan yang memungkinkan dua sistem atau lebih untuk saling berkomunikasi dan berbagi data tanpa harus membangun fungsi tersebut dari awal. Senada dengan hal tersebut, (Nurhayati & Agussalim, 2023) menyatakan bahwa API merupakan sebuah antarmuka yang menjadi penghubung antara sistem aplikasi yang berbeda sehingga sebagian atau keseluruhan fungsi sistem dapat diakses secara bersamaan. Hal ini menjadikan API sebagai elemen krusial dalam arsitektur perangkat lunak modern, karena memungkinkan pengembang untuk memanfaatkan fitur atau layanan dari sistem lain secara efisien tanpa perlu mengembangkannya dari nol. Selain itu, Manuaba & Rudiastini (2018) yang dikutip dalam jurnal JUTIKOMP oleh (Sulistyo et al., 2024) mendefinisikan API sebagai dokumen yang terdiri dari antarmuka, fungsi, kelas, struktur dan lain-lain untuk membuat perangkat lunak, di mana manfaat utama API adalah memungkinkan aplikasi untuk terhubung dan berinteraksi dengan aplikasi lain.

Dalam praktiknya, API bekerja menggunakan mekanisme *request* dan *response*, di mana sebuah aplikasi mengirimkan permintaan (*request*) kepada sistem lain melalui API, kemudian sistem tersebut memberikan respons (*response*) berupa data atau hasil yang diminta. Salah satu arsitektur API yang paling banyak digunakan saat ini adalah *Representational State Transfer* (REST) atau yang dikenal dengan sebutan REST API. REST API menggunakan protokol *Hypertext Transfer Protocol* (HTTP) dalam melakukan komunikasi data, serta bekerja dengan memisahkan peran antara *client* dan *server* sehingga perubahan pada salah satu sisi tidak berdampak pada sisi lainnya (Nurhayati & Agussalim, 2023).

Berdasarkan uraian di atas, dapat disimpulkan bahwa API (*Application Programming Interface*) adalah teknologi antarmuka yang berfungsi sebagai penghubung antara dua sistem atau aplikasi yang berbeda, sehingga data dan fungsi dari suatu sistem dapat diakses oleh sistem lain secara terstruktur dan efisien. Dalam penelitian ini, API digunakan sebagai penghubung antara aplikasi Android dan *website* agar data dapat dikomunikasikan secara real-time antara pengguna pencari kerja dan pemberi kerja, API digunakan dengan format JSON dan metode komunikasi HTTP/HTTPS. Fungsi API dalam system Menghubungkan web dan aplikasi Android, Mengelola pertukaran data dan Menjamin sinkronisasi data

I. JSON (JavaScript Object Notation)

JavaScript Object Notation (JSON) merupakan salah satu format pertukaran data yang paling banyak digunakan dalam pengembangan aplikasi berbasis web dan *mobile* saat ini. Menurut (Buwono, 2019), JSON merupakan seperangkat aturan untuk memformat data berbasis teks yang ringan, digunakan pada pertukaran data antara sistem. Dengan karakteristiknya yang ringan dan mudah dipahami, JSON menjadi

standar utama dalam komunikasi data antara aplikasi *client* dan *server* pada sistem modern.

Senada dengan hal tersebut, (Sis.binus.ac.id, 2022) mendefinisikan JSON sebagai format pertukaran data yang ringan, mudah dibaca, dan mudah ditulis oleh manusia, sehingga memberikan alternatif yang lebih baik dibandingkan format XML yang lebih kompleks. JSON menggunakan struktur berbasis teks yang menyerupai objek *JavaScript*, sehingga lebih intuitif dan mudah diintegrasikan ke dalam berbagai bahasa pemrograman seperti Java, Python, PHP, dan lain-lain. Dalam implementasinya, JSON mengenal dua struktur utama yaitu *JSONObject* yang direpresentasikan dengan tanda kurung kurawal { } dan *JSONArray* yang direpresentasikan dengan tanda kurung siku [].

Keunggulan JSON dibandingkan format pertukaran data lainnya menjadi alasan utama penggunaannya yang sangat luas. Berdasarkan kajian yang dipublikasikan oleh (Saifudin et al., 2026), penggunaan JSON sebagai format pertukaran data dalam RESTful API memberikan keunggulan dalam hal ukuran file yang lebih kecil dibandingkan XML, *parsing* yang lebih cepat, serta struktur data yang sederhana namun *powerful*. Hal ini menjadikan JSON sangat cocok digunakan dalam pengembangan aplikasi yang membutuhkan pertukaran data secara cepat dan efisien, termasuk aplikasi *mobile* berbasis Android.

Dalam konteks pengembangan sistem berbasis API, JSON berperan sangat penting sebagai format standar komunikasi data. Menurut (Najibulloh Muzaki et al., n.d.), Alasan penggunaan web service JSON dalam penelitian ini adalah karena JSON merupakan format tipe data yang sederhana, tidak memerlukan ruang penyimpanan besar, dan tidak membutuhkan sumber daya yang besar dalam proses transfer data meskipun hanya

menggunakan koneksi jaringan yang lambat, serta tidak memerlukan sumber daya prosesor yang besar pada smartphone berbasis Android untuk mendekode format JSON. Hal ini sangat relevan dalam pengembangan aplikasi yang melibatkan pertukaran data secara *real-time* antara aplikasi Android dan *server*.

Berdasarkan uraian di atas, dapat disimpulkan bahwa JSON (*JavaScript Object Notation*) adalah format pertukaran data berbasis teks yang ringan, fleksibel, dan mudah dipahami, yang berfungsi sebagai media komunikasi data antara *client* dan *server*. Dalam penelitian ini, JSON digunakan sebagai format standar pertukaran data antara aplikasi Android dan *server* melalui REST API, sehingga data antara pencari kerja dan pemberi kerja dapat dikomunikasikan secara cepat, terstruktur, dan efisien.

Contoh struktur JSON:

```
{
  "nama": "Fajri",
  "pekerjaan": "Programmer"
}
```

JSON digunakan dalam penelitian ini sebagai format utama dalam pertukaran data antara *client* (web dan Android) dengan *server*.

J. Arsitektur Client-Server

Arsitektur *client-server* merupakan salah satu model arsitektur sistem yang paling banyak diterapkan dalam pengembangan aplikasi berbasis jaringan, baik aplikasi *web* maupun *mobile*. Menurut (Solichin, 2016), arsitektur *client-server* adalah model komunikasi yang memisahkan antara pihak yang meminta layanan (*client*) dan pihak

yang menyediakan layanan (*server*), di mana keduanya terhubung melalui jaringan komputer. Dengan pemisahan peran ini, setiap komponen dapat dikembangkan dan dikelola secara independen tanpa mengganggu komponen lainnya.

Dalam arsitektur *client-server*, *client* bertugas mengirimkan permintaan (*request*) kepada *server*, kemudian *server* akan memproses permintaan tersebut dan mengembalikan hasilnya (*response*) kepada *client*. Pada penelitian ini, komponen *client* terdiri dari aplikasi Android yang digunakan oleh pengguna (*user*) dan antarmuka *web* yang digunakan oleh pengelola sistem, sedangkan komponen *server* terdiri dari *backend* yang menangani logika bisnis serta basis data (*database*) yang menyimpan seluruh data sistem. Komunikasi antara *client* dan *server* dilakukan melalui protokol HTTP dengan memanfaatkan REST API dan format JSON sebagai media pertukaran data. Arsitektur *client-server* memiliki beberapa keunggulan dibandingkan arsitektur lainnya. Pertama, sistem bersifat terpusat (*centralized*) sehingga pengelolaan data dan keamanan sistem dapat dilakukan secara lebih terstruktur dari satu titik kendali. Kedua, arsitektur ini bersifat mudah dikembangkan (*maintainable*) karena perubahan pada sisi *server* tidak memerlukan pembaruan pada sisi *client* secara langsung. Ketiga, arsitektur *client-server* bersifat *scalable*, artinya sistem dapat dikembangkan kapasitasnya sesuai dengan pertumbuhan jumlah pengguna tanpa harus mengubah struktur sistem secara menyeluruh (Pressman, 2015a). Berdasarkan uraian tersebut, dapat disimpulkan bahwa arsitektur *client-server* adalah model arsitektur sistem yang memisahkan peran antara *client* sebagai peminta layanan dan *server* sebagai penyedia layanan, yang terhubung melalui jaringan komputer. Dalam penelitian ini, arsitektur *client-server* diterapkan sebagai fondasi utama sistem, di mana aplikasi Android dan *website* berperan sebagai *client* yang berkomunikasi dengan *server* melalui REST API untuk mengakses dan mengelola data secara terpusat.

K. Basis Data (Database)

Basis data (*database*) merupakan komponen fundamental dalam pengembangan sistem informasi modern. Menurut (Elmasri & Navathe, 2016), basis data adalah kumpulan data yang saling berhubungan secara logis dan dirancang untuk memenuhi kebutuhan informasi suatu organisasi, di mana data tersebut disimpan secara efisien serta mendukung proses manipulasi data seperti penyimpanan, pencarian, pembaruan, dan penghapusan. Dengan adanya basis data, data dapat dikelola secara terstruktur sehingga memudahkan proses pengolahan informasi yang dibutuhkan oleh sistem. Pendapat senada dikemukakan oleh (Connolly & Begg, 2015) yang mendefinisikan basis data sebagai kumpulan data yang saling terkait secara logis beserta deskripsinya, yang dirancang untuk memenuhi kebutuhan informasi suatu organisasi. Sementara itu, Fathansyah (2018) menyatakan bahwa basis data merupakan kumpulan data yang saling berhubungan yang disimpan secara bersama tanpa pengulangan (*redundancy*) yang tidak perlu, untuk memenuhi berbagai kebutuhan. Dari beberapa definisi tersebut, dapat dipahami bahwa basis data bukan sekadar kumpulan data biasa, melainkan data yang terorganisir dengan baik sehingga dapat diakses, dikelola, dan diperbarui secara efisien. Dalam pengelolaan basis data, digunakan perangkat lunak yang disebut *Database Management System* (DBMS). DBMS berfungsi sebagai perantara antara pengguna dan basis data, sehingga pengguna dapat melakukan berbagai operasi data seperti penyimpanan, pencarian, pembaruan, dan penghapusan data dengan mudah dan aman. Salah satu DBMS yang paling banyak digunakan dalam pengembangan sistem berbasis web dan *mobile* adalah MySQL, yang menggunakan bahasa *Structured Query Language* (SQL) sebagai bahasa standar dalam pengelolaan data (Renaldi et al., 2020).

Dalam penelitian ini, basis data digunakan sebagai media penyimpanan seluruh data yang dibutuhkan oleh sistem. Data yang disimpan mencakup data pengguna (*user*)

yang terdiri dari data pencari kerja dan pemberi kerja, data pekerjaan yang berisi informasi lowongan yang tersedia, serta data transaksi atau interaksi yang merekam aktivitas antara pencari kerja dan pemberi kerja di dalam sistem. Seluruh data tersebut dikelola menggunakan DBMS MySQL yang terhubung dengan *server* melalui REST API sehingga dapat diakses secara dinamis oleh aplikasi Android maupun *website*.

Berdasarkan uraian di atas, dapat disimpulkan bahwa basis data (*database*) adalah kumpulan data yang terorganisir secara sistematis dan saling berhubungan, yang dirancang untuk memudahkan penyimpanan, pengelolaan, dan pengaksesan data secara efisien. Dalam penelitian ini, basis data berperan sebagai pondasi penyimpanan data sistem yang menjamin integritas, konsistensi, dan keamanan data yang digunakan oleh seluruh komponen sistem.

L. Sistem Autentikasi Berbasis Token

Autentikasi merupakan salah satu aspek keamanan yang paling penting dalam pengembangan sistem informasi berbasis jaringan. Menurut (Stallings, 2017), autentikasi adalah proses verifikasi identitas pengguna yang bertujuan untuk memastikan bahwa pihak yang mengakses sistem adalah pengguna yang sah dan memiliki hak akses yang sesuai. Tanpa mekanisme autentikasi yang baik, sistem rentan terhadap akses tidak sah yang dapat membahayakan kerahasiaan dan integritas data pengguna.

Salah satu pendekatan autentikasi yang banyak digunakan dalam pengembangan aplikasi berbasis API saat ini adalah sistem autentikasi berbasis token (*token-based authentication*). Menurut (Setiawan & Purnamasari, 2020), autentikasi berbasis token adalah mekanisme keamanan di mana *server* menghasilkan sebuah token unik setelah pengguna berhasil melakukan proses *login*, dan token tersebut selanjutnya digunakan oleh *client* untuk mengakses sumber daya yang dilindungi tanpa perlu mengirimkan

kredensial seperti *username* dan *password* pada setiap permintaan berikutnya. Salah satu standar token yang paling banyak digunakan adalah *JSON Web Token* (JWT), yang menyimpan informasi identitas pengguna dalam format terenkripsi dan dapat diverifikasi oleh *server* secara mandiri.

Secara operasional, sistem autentikasi berbasis token bekerja melalui serangkaian tahapan yang terstruktur. Pertama, pengguna melakukan proses *login* dengan mengirimkan kredensial berupa *username* dan *password* ke *server*. Kedua, *server* memverifikasi kredensial tersebut dan apabila valid, *server* akan menghasilkan token unik yang dikirimkan kembali kepada *client*. Ketiga, *client* menyimpan token tersebut dan menyertakannya pada setiap permintaan (*request*) berikutnya sebagai bukti bahwa pengguna telah terautentikasi dengan sah.

Menurut (Setiawan & Purnamasari, 2020), implementasi *JSON Web Token* (JWT) dengan algoritma SHA-512 pada proses autentikasi terbukti dapat mempercepat proses autentikasi sekaligus meningkatkan keamanan sistem, karena penggunaan token terenkripsi pada saat autentikasi mencegah akses tidak sah tanpa perlu menyimpan informasi sesi (*stateless*) di sisi *server*.

Sistem autentikasi berbasis token memiliki sejumlah keunggulan dibandingkan metode autentikasi konvensional berbasis sesi (*session-based*). Pertama, sistem ini bersifat *stateless*, artinya *server* tidak perlu menyimpan data sesi pengguna sehingga beban *server* berkurang secara signifikan. Kedua, token yang dihasilkan bersifat terenkripsi sehingga lebih aman dari risiko pencurian data dan pemalsuan identitas. Ketiga, sistem ini sangat cocok digunakan dalam pengembangan REST API karena mendukung komunikasi lintas platform antara berbagai jenis *client* seperti aplikasi Android, *website*, maupun aplikasi pihak ketiga (Stallings, 2017).

Berdasarkan uraian di atas, dapat disimpulkan bahwa sistem autentikasi berbasis token adalah mekanisme keamanan yang memverifikasi identitas pengguna melalui token unik yang dihasilkan oleh *server* setelah proses *login* berhasil dilakukan. Dalam penelitian ini, sistem autentikasi berbasis token diterapkan untuk menjaga keamanan akses pada setiap permintaan yang dilakukan oleh aplikasi Android maupun *website*, sehingga hanya pengguna yang terautentikasi yang dapat mengakses fitur dan data di dalam sistem.

M. UML (Unified Modeling Language)

Unified Modeling Language (UML) merupakan bahasa pemodelan visual standar yang digunakan secara luas dalam pengembangan sistem perangkat lunak. Menurut (Hendini, 2016), UML adalah bahasa pemodelan yang digunakan untuk memvisualisasikan, merancang, dan mendokumentasikan sistem perangkat lunak secara terstruktur. Dengan menggunakan UML, pengembang sistem dapat menggambarkan arsitektur dan alur kerja sistem secara visual sehingga lebih mudah dipahami oleh seluruh pihak yang terlibat dalam pengembangan, baik pengembang teknis maupun pemangku kepentingan non-teknis.

Senada dengan hal tersebut, (As & Shalahudin, 2021) mendefinisikan UML sebagai standar bahasa yang banyak digunakan di dunia industri untuk mendefinisikan kebutuhan (*requirement*), membuat analisis dan desain, serta menggambarkan arsitektur dalam pemrograman berorientasi objek. Penggunaan UML dalam pengembangan sistem memberikan manfaat yang signifikan, antara lain mempermudah komunikasi antar anggota tim pengembang, meminimalkan kesalahan dalam perancangan sistem, serta menjadi dokumentasi sistem yang dapat digunakan sebagai acuan pada tahap pengembangan berikutnya.

UML terdiri dari berbagai jenis diagram yang masing-masing memiliki fungsi dan tujuan yang berbeda. Dalam penelitian ini, diagram UML yang digunakan meliputi empat jenis diagram utama. Pertama, *Use Case Diagram* yang digunakan untuk menggambarkan interaksi antara pengguna (*actor*) dengan sistem, sehingga kebutuhan fungsional sistem dapat terdefinisi dengan jelas. Kedua, *Activity Diagram* yang digunakan untuk menggambarkan alur aktivitas atau proses bisnis yang terjadi di dalam sistem secara berurutan. Ketiga, *Sequence Diagram* yang digunakan untuk menggambarkan urutan interaksi antar objek dalam sistem berdasarkan waktu, sehingga alur komunikasi antar komponen sistem dapat terpetakan dengan baik. Keempat, *Class Diagram* yang digunakan untuk menggambarkan struktur kelas, atribut, metode, serta hubungan antar kelas yang terdapat di dalam system (Shalahuddin & Rosa, 2013).

Berdasarkan uraian di atas, dapat disimpulkan bahwa UML (*Unified Modeling Language*) adalah bahasa pemodelan visual standar yang digunakan untuk merancang, memvisualisasikan, dan mendokumentasikan sistem perangkat lunak secara terstruktur. Dalam penelitian ini, UML digunakan sebagai alat bantu perancangan sistem yang mencakup *Use Case Diagram*, *Activity Diagram*, *Sequence Diagram*, dan *Class Diagram*, guna memastikan bahwa sistem yang dibangun sesuai dengan kebutuhan dan dapat dipahami secara menyeluruh oleh seluruh pihak yang terlibat.

N. Rekayasa Perangkat Lunak

Rekayasa perangkat lunak (*Software Engineering*) merupakan salah satu disiplin ilmu dalam bidang teknologi informasi yang membahas tentang pengembangan perangkat lunak secara sistematis, terstruktur, dan terukur. Pengembangan perangkat lunak terdapat empat tahapan dengan prosedur SDLC yaitu *planning*, *analysis*, *design*, dan *implementation*, dan untuk mengembangkan perangkat lunak terdapat beberapa

metode seperti Siklus Klasik (*Waterfall Model*), *Prototyping*, Model Spiral, dan gabungan beberapa metode (Amrozi et al., 2020). Dengan adanya rekayasa perangkat lunak, proses pembangunan sistem tidak dilakukan secara sembarangan, melainkan mengikuti tahapan dan metodologi yang telah terstandarisasi sehingga menghasilkan perangkat lunak yang berkualitas, andal, dan sesuai dengan kebutuhan pengguna.

Menurut (Bakri & Nasution, 2024), penerapan metodologi rekayasa perangkat lunak yang tepat terbukti dapat meningkatkan efisiensi dalam proses pengembangan sistem, karena setiap tahapan pengembangan dilakukan secara terstruktur dan terukur sesuai dengan kebutuhan yang telah didefinisikan sebelumnya.

Rekayasa perangkat lunak hadir sebagai solusi atas permasalahan yang sering terjadi dalam pengembangan sistem, seperti keterlambatan penyelesaian proyek, pembengkakan biaya, serta rendahnya kualitas perangkat lunak yang dihasilkan akibat proses pengembangan yang tidak terencana dengan baik.

Secara umum, rekayasa perangkat lunak mencakup serangkaian tahapan utama yang harus dilalui dalam proses pengembangan sistem. Tahap pertama adalah analisis kebutuhan (*requirements analysis*), yaitu proses mengidentifikasi dan mendefinisikan kebutuhan fungsional maupun non-fungsional sistem. Tahap kedua adalah perancangan (*design*), yaitu proses menerjemahkan kebutuhan menjadi rancangan arsitektur sistem, basis data, dan antarmuka pengguna. Tahap ketiga adalah implementasi (*implementation*), yaitu proses penulisan kode program berdasarkan rancangan yang telah dibuat. Tahap keempat adalah pengujian (*testing*), yaitu proses memverifikasi dan memvalidasi perangkat lunak untuk memastikan sistem berjalan sesuai kebutuhan dan bebas dari kesalahan (*bug*) (Amrozi et al., 2020)

Dalam praktiknya, proses rekayasa perangkat lunak dilaksanakan menggunakan berbagai model pengembangan atau yang dikenal dengan istilah *Software Development Life Cycle* (SDLC). Beberapa model SDLC yang umum digunakan antara lain *Waterfall*, *Prototyping*, *Spiral*, dan *Agile*, di mana pemilihan model pengembangan yang tepat bergantung pada karakteristik proyek dan kebutuhan sistem yang dikembangkan (Bakri & Nasution, 2024).

Berdasarkan uraian di atas, dapat disimpulkan bahwa rekayasa perangkat lunak adalah disiplin ilmu yang mengatur proses pengembangan perangkat lunak secara sistematis melalui tahapan analisis kebutuhan, perancangan, implementasi, dan pengujian. Dalam penelitian ini, prinsip-prinsip rekayasa perangkat lunak diterapkan sebagai landasan dalam proses perancangan dan pembangunan sistem, guna menghasilkan aplikasi yang berkualitas, fungsional, dan sesuai dengan kebutuhan pengguna.

O. Pengujian Perangkat Lunak

Pengujian perangkat lunak merupakan tahapan yang sangat penting dalam proses pengembangan sistem informasi untuk memastikan bahwa sistem yang dibangun berjalan sesuai dengan kebutuhan yang telah ditetapkan. Pengujian perangkat lunak adalah proses atau rangkaian proses yang dirancang untuk memastikan bahwa kode komputer berfungsi sesuai dengan tujuan yang telah ditentukan dan tidak melakukan hal-hal yang tidak diinginkan, sehingga perangkat lunak harus dapat berfungsi secara konsisten dan dapat diprediksi tanpa memberikan kejutan kepada pengguna (Alessandro et al., 2024). Tanpa adanya pengujian yang memadai, sistem berisiko mengalami kegagalan fungsi yang dapat merugikan pengguna dan menurunkan kepercayaan terhadap sistem yang dikembangkan.

Salah satu metode pengujian yang paling banyak digunakan dalam pengembangan sistem informasi adalah *Black Box Testing*. *Black box testing* merupakan pengujian kualitas perangkat lunak yang berfokus pada fungsionalitas perangkat lunak, di mana pengujian ini bertujuan untuk menemukan fungsi yang tidak benar, kesalahan antarmuka, kesalahan pada struktur data, kesalahan performansi, serta kesalahan inisialisasi dan terminasi(Wijaya & Astuti, 2021). Keunggulan utama metode ini adalah pengujian dapat dilakukan tanpa penguji harus memahami struktur kode program secara mendalam, melainkan cukup memahami alur dan ekspektasi sistem yang diuji.

Selain *Black Box Testing*, pengujian juga dilakukan melalui *User Acceptance Testing* (UAT) atau pengujian penerimaan pengguna. UAT merupakan tahap pengujian yang melibatkan pengguna akhir untuk menilai apakah sistem yang dikembangkan sudah memenuhi harapan dan kebutuhan mereka sebelum resmi dioperasikan (Cimperman, 2006 dalam Jurnal Inventor, 2025). UAT efektif karena melibatkan pengguna aktual yang mengoperasikan sistem sehari-hari, sehingga mampu mengidentifikasi permasalahan *usability* dan kesesuaian alur kerja operasional yang mungkin terlewatkan pada tahap pengujian teknis sebelumnya(Aliyah et al., 2025).

Dalam penelitian ini, pengujian perangkat lunak dilakukan menggunakan dua metode yang telah disebutkan di atas. *Black Box Testing* digunakan untuk memverifikasi seluruh fungsi sistem berjalan dengan benar sesuai spesifikasi yang telah dirancang, mencakup fungsi *login*, pencarian pekerjaan, pengelolaan data, dan fitur interaksi antara pencari kerja dengan pemberi kerja. Sementara itu, *User Acceptance Testing* (UAT) dilakukan untuk mengevaluasi kemudahan penggunaan (*usability*) sistem dari sudut pandang pengguna secara langsung, sehingga sistem yang dihasilkan benar-benar sesuai dengan kebutuhan dan harapan pengguna akhir.

Berdasarkan uraian di atas, dapat disimpulkan bahwa pengujian perangkat lunak adalah proses penting yang harus dilakukan sebelum sistem dioperasikan secara resmi. Dalam penelitian ini, kombinasi metode *Black Box Testing* dan *User Acceptance Testing* digunakan untuk memastikan bahwa sistem yang dibangun tidak hanya berfungsi secara teknis dengan benar, tetapi juga mudah digunakan dan dapat diterima oleh pengguna akhir.

BAB III.

ANALISIS DAN PERANCANGAN SYSTEM

A. Analisis Sistem

Analisis sistem merupakan tahapan awal dalam proses pengembangan perangkat lunak yang bertujuan untuk mengidentifikasi kebutuhan sistem secara menyeluruh sebelum proses perancangan dan implementasi dilakukan. Pada tahap ini, dilakukan analisis terhadap permasalahan yang ada, kebutuhan pengguna, serta spesifikasi sistem yang akan dibangun, sehingga platform yang dikembangkan dapat menjawab permasalahan ketenagakerjaan yang telah diidentifikasi pada Bab I.

1. Analisis Masalah

Berdasarkan identifikasi masalah yang telah diuraikan pada Bab I, terdapat beberapa permasalahan utama yang menjadi dasar pengembangan sistem ini. Pertama, belum tersedianya platform digital yang mengintegrasikan layanan *freelance* dan *job sharing* dalam satu sistem terpadu di Indonesia. Kedua, keterbatasan aksesibilitas platform ketenagakerjaan yang belum optimal pada perangkat *mobile* berbasis Android. Ketiga, minimnya fitur pendukung kolaborasi dan manajemen kerja antara pencari kerja dan pemberi kerja. Keempat, rendahnya tingkat keamanan dan kepercayaan dalam transaksi kerja digital akibat belum optimalnya mekanisme autentikasi pengguna. Kelima, belum adanya platform ketenagakerjaan yang memungkinkan calon pengguna menjelajahi dan mencari daftar pekerjaan secara langsung tanpa harus melakukan registrasi terlebih dahulu, sehingga menghambat keterjangkauan platform kepada pengguna baru.

Berdasarkan permasalahan tersebut, dibutuhkan sebuah platform yang mampu mengintegrasikan layanan *freelance* dan *job sharing* sekaligus, dapat diakses melalui

website maupun aplikasi Android, memiliki sistem autentikasi yang aman berbasis token, menyediakan fitur penelusuran pekerjaan tanpa *login* bagi pengguna baru, serta menyediakan fitur pengelolaan pekerjaan yang lengkap bagi pencari kerja, pemberi kerja, maupun administrator sistem.

2. Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dibagi menjadi dua bagian, yaitu kebutuhan fungsional dan kebutuhan non-fungsional.

a. Kebutuhan Fungsional

Kebutuhan fungsional merupakan kebutuhan yang berkaitan langsung dengan fitur dan fungsi yang harus disediakan oleh sistem. Kebutuhan fungsional sistem ini adalah sebagai berikut.

- 1) Pertama, sistem harus menyediakan fitur *guest mode* yang memungkinkan pengguna mengakses halaman daftar pekerjaan dan melakukan pencarian tanpa harus melakukan registrasi atau *login* terlebih dahulu. Data penelusuran pada mode ini disimpan sementara dalam *session* aplikasi dan apabila pengguna mencoba mengakses pratinjau detail pekerjaan, sistem akan mengarahkan pengguna untuk melakukan *login* atau registrasi terlebih dahulu.
- 2) Kedua, sistem harus menyediakan fitur registrasi dan *login* bagi pengguna dengan mekanisme autentikasi berbasis token (*JSON Web Token/JWT*). Setelah berhasil *login*, pengguna akan diarahkan ke halaman pemilihan peran (*role selection*) untuk memilih apakah akan masuk sebagai *Job Seeker* (Pencari Kerja) atau *Employer* (Pemberi Kerja), di mana menu dan antarmuka yang ditampilkan akan berbeda sesuai peran yang dipilih.

- 3) Ketiga, sistem harus menyediakan fitur pengelolaan profil pengguna yang memungkinkan pengguna memperbarui data diri, keahlian,, portofolio serta hal lain yang menyangkut data diri yang di perlukan.
- 4) Keempat, sistem harus menyediakan fitur pencarian dan penawaran pekerjaan *freelance* maupun *job sharing* yang dapat difilter berdasarkan kategori, lokasi, dan bidang keahlian.
- 5) Kelima, sistem harus menyediakan fitur komunikasi dasar antara pencari kerja dan pemberi kerja.
- 6) Keenam, sistem harus menyediakan fitur pengelolaan data pengguna, data pekerjaan, dan pemantauan aktivitas sistem melalui panel *web admin*.
- 7) Ketujuh, sistem harus mampu menyinkronisasi data secara *real-time* antara aplikasi Android dan *website* melalui REST API dengan format pertukaran data JSON.
- 8) Kedelapan, sistem harus menyediakan fitur pembayaran digital yang memungkinkan *Employer* membayar jasa kepada *Job Seeker* melalui *payment gateway* Midtrans dengan berbagai metode pembayaran meliputi transfer bank (*virtual account*), dompet digital (*e-wallet*) seperti GoPay, OVO, dan Dana, QRIS, serta kartu kredit. Sistem secara otomatis menambahkan biaya layanan sebesar 2,5% dari nilai *project* kepada *Employer* sebagai biaya penggunaan platform, sehingga total yang dibayarkan *Employer* adalah nilai *project* ditambah 2,5%. Dari sisi *Job Seeker*, sistem secara otomatis memotong 2% dari nilai *project* sebagai biaya layanan platform, sehingga *Job Seeker* menerima 98% dari nilai *project* yang disepakati. Seluruh transaksi pembayaran dikelola secara otomatis oleh sistem melalui integrasi REST API Midtrans dan dicatat secara transparan di dalam basis data sistem.

b. Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan yang berkaitan dengan kualitas dan performa sistem secara keseluruhan. Kebutuhan non-fungsional sistem ini meliputi aspek keamanan, di mana sistem harus menerapkan autentikasi berbasis JWT dan enkripsi *password* menggunakan algoritma *bcrypt*. Dari aspek keandalan (*reliability*), sistem harus mampu menangani permintaan dari banyak pengguna secara bersamaan tanpa mengalami gangguan performa yang signifikan. Dari aspek kemudahan penggunaan (*usability*), antarmuka sistem harus dirancang secara intuitif dan mudah dioperasikan oleh pengguna awam, termasuk pengguna yang belum melakukan *login*. Dari aspek *scalability*, arsitektur sistem harus dirancang sedemikian rupa sehingga memungkinkan pengembangan fitur dan peningkatan kapasitas di masa mendatang.

Dari aspek keamanan transaksi, sistem harus memastikan bahwa seluruh proses pembayaran dilakukan melalui protokol HTTPS dan diverifikasi menggunakan *signature key* Midtrans untuk mencegah pemalsuan notifikasi pembayaran. Sistem juga harus mampu menangani *webhook* notifikasi dari Midtrans secara real-time untuk memperbarui status transaksi secara otomatis tanpa intervensi manual.

3. Analisis Pengguna

Sistem yang dikembangkan memiliki tiga jenis pengguna (*actor*) dengan peran dan hak akses yang berbeda, yaitu *Guest*, *User*, dan *Administrator*.

Guest merupakan pengguna yang mengakses aplikasi Android tanpa melakukan registrasi maupun *login* terlebih dahulu. Saat pertama kali membuka aplikasi, *Guest* langsung diarahkan ke halaman daftar pekerjaan dan dapat melakukan pencarian pekerjaan berdasarkan kata kunci atau kategori tertentu. Data penelusuran yang dilakukan oleh *Guest* disimpan sementara dalam *session* aplikasi dan tidak tersimpan

secara permanen di basis data. Apabila *Guest* mencoba mengakses fitur pratinjau detail pekerjaan, sistem akan menampilkan halaman peringatan yang mengarahkan pengguna untuk melakukan *login* atau registrasi terlebih dahulu. Dengan adanya fitur ini, pengguna baru dapat mengenal dan menjelajahi konten platform tanpa hambatan di awal, sehingga meningkatkan kemungkinan pengguna untuk mendaftar setelah menemukan pekerjaan yang relevan.

User merupakan pengguna yang telah melakukan registrasi dan *login* ke dalam sistem. Setelah berhasil *login*, pengguna akan diarahkan ke halaman pemilihan peran (*role selection*) yang menyediakan dua pilihan, yaitu sebagai *Job Seeker* (Pencari Kerja) atau sebagai *Employer* (Pemberi Kerja). Pemilihan peran ini bersifat dinamis, artinya satu akun yang sama dapat difungsikan untuk kedua peran tersebut pada waktu yang berbeda sesuai kebutuhan pengguna, sehingga tidak diperlukan dua akun yang terpisah untuk menjalankan kedua fungsi tersebut.

Apabila pengguna memilih peran sebagai *Job Seeker*, sistem akan menampilkan antarmuka dan menu yang dirancang khusus untuk pencari kerja, meliputi fitur pratinjau detail pekerjaan, pengajuan lamaran pekerjaan *freelance* dan *job sharing*, pemantauan status lamaran, pengelolaan profil, dan komunikasi dengan pemberi kerja. Sebaliknya, apabila pengguna memilih peran sebagai *Employer*, sistem akan menampilkan antarmuka dan menu yang dirancang khusus untuk pemberi kerja, meliputi fitur pembuatan dan pengelolaan lowongan pekerjaan, manajemen lamaran yang masuk, pengelolaan profil, dan komunikasi dengan pencari kerja.

Administrator merupakan pengelola sistem yang mengakses platform melalui panel *web admin* berbasis Laravel. Administrator memiliki hak akses penuh terhadap seluruh data dan fitur sistem, termasuk manajemen akun pengguna, pengelolaan data

pekerjaan, pemantauan aktivitas sistem, serta penanganan laporan dan pengaduan dari pengguna.

B. Metode Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam penelitian ini adalah metode *Prototype* dengan pendekatan *Evolutionary Prototype*. Metode *Prototype* merupakan model pengembangan perangkat lunak yang dilakukan secara iteratif melalui pembuatan rancangan awal sistem (*prototype*) yang kemudian dievaluasi dan disempurnakan secara berulang hingga menghasilkan sistem akhir yang sesuai dengan kebutuhan pengguna. Menurut (Pressman, 2015b), metode *Prototype* dimulai dengan pengumpulan kebutuhan awal pengguna, dilanjutkan dengan pembangunan *prototype*, evaluasi *prototype* bersama pengguna, serta proses perbaikan sistem secara terus-menerus hingga diperoleh sistem yang sesuai dengan kebutuhan. Pendekatan ini sangat cocok digunakan pada pengembangan sistem yang kebutuhan penggunaannya belum terdefinisi secara rinci pada tahap awal pengembangan.

Jenis *prototype* yang diterapkan dalam penelitian ini adalah *Evolutionary Prototype*. *Evolutionary Prototype* merupakan jenis *prototype* yang dikembangkan secara bertahap dan terus disempurnakan hingga menjadi sistem final. Berbeda dengan *Throwaway Prototype* yang hanya digunakan sementara lalu dibuang, *Evolutionary Prototype* menjadikan *prototype* awal sebagai dasar utama pengembangan sistem akhir. Menurut (Sommerville, 2011), *Evolutionary Prototype* memungkinkan pengembang dan pengguna melakukan evaluasi sistem secara berkelanjutan sehingga perubahan kebutuhan dapat langsung diakomodasi ke dalam sistem yang sedang dikembangkan. Pendekatan ini dipilih karena platform digital ketenagakerjaan yang dibangun memiliki

kebutuhan sistem yang dinamis serta membutuhkan keterlibatan pengguna dalam proses evaluasi fitur dan antarmuka sistem.

Pemilihan metode *Evolutionary Prototype* didasarkan pada beberapa pertimbangan. Pertama, sistem yang dikembangkan memiliki fitur yang cukup kompleks, mencakup fitur *guest mode* untuk penelusuran pekerjaan tanpa *login*, mekanisme pemilihan peran (*role selection*) antara *Job Seeker* dan *Employer* setelah *login*, pengelolaan pekerjaan *freelance* dan *job sharing*, autentikasi pengguna berbasis JWT, REST API, serta integrasi antara *website* admin dan aplikasi Android. Kedua, metode ini memungkinkan proses evaluasi dilakukan lebih awal sehingga kesalahan desain dan kebutuhan sistem dapat segera diperbaiki, termasuk evaluasi terhadap alur *guest mode* dan halaman pemilihan peran yang menjadi fitur khas platform ini. Ketiga, keterlibatan pengguna dalam setiap siklus pengembangan membantu memastikan bahwa sistem yang dibangun benar-benar sesuai dengan kebutuhan pengguna akhir, baik dari sisi *Guest*, *Job Seeker*, *Employer*, maupun *Administrator*. Dengan pendekatan iteratif, sistem dapat dikembangkan secara bertahap mulai dari fitur inti hingga fitur pendukung lainnya secara lebih fleksibel dan terstruktur.

Adapun tahapan *Evolutionary Prototype* yang diterapkan dalam penelitian ini meliputi beberapa tahap sebagai berikut.

- 1) Tahap pertama adalah pengumpulan kebutuhan (*requirement gathering*), yaitu proses identifikasi kebutuhan fungsional dan non-fungsional sistem melalui studi literatur, observasi, dan analisis kebutuhan pengguna. Pada tahap ini turut diidentifikasi kebutuhan khusus sistem seperti fitur *guest mode*, mekanisme *session* untuk pengguna tanpa *login*, serta alur pemilihan peran setelah *login*.

- 2) Tahap kedua adalah perancangan *prototype* awal, yaitu proses pembuatan rancangan antarmuka sistem, basis data, dan alur kerja sistem berdasarkan kebutuhan yang telah diperoleh. Perancangan mencakup alur lengkap penggunaan aplikasi mulai dari halaman daftar pekerjaan untuk *Guest*, halaman peringatan *login* saat mengakses detail pekerjaan, halaman pemilihan peran setelah *login*, hingga perbedaan antarmuka dan menu antara *Job Seeker* dan *Employer*.
- 3) Tahap ketiga adalah pembangunan *prototype*, yaitu proses implementasi *prototype* menggunakan *framework* Laravel untuk *backend website* dan REST API, serta Flutter sebagai *framework* pengembangan aplikasi Android yang berperan sebagai *client system*.
- 4) Tahap keempat adalah evaluasi *prototype*, yaitu proses pengujian dan pengumpulan umpan balik dari pengguna terhadap fitur dan tampilan sistem yang telah dibangun. Evaluasi dilakukan terhadap seluruh jenis pengguna sistem, yaitu *Guest*, *Job Seeker*, *Employer*, dan *Administrator*, untuk memastikan setiap alur penggunaan berjalan sesuai dengan kebutuhan yang telah ditetapkan.
- 5) Tahap kelima adalah penyempurnaan *prototype*, yaitu proses perbaikan dan pengembangan sistem berdasarkan hasil evaluasi pengguna. Tahapan ini dilakukan secara berulang hingga sistem mencapai bentuk akhir yang sesuai dengan kebutuhan penelitian.
- 6) Tahap terakhir adalah implementasi sistem akhir, yaitu proses finalisasi dan penerapan sistem yang telah selesai dikembangkan dan siap digunakan oleh seluruh jenis pengguna sistem.

Berdasarkan uraian di atas, dapat disimpulkan bahwa metode *Evolutionary Prototype* merupakan metode pengembangan perangkat lunak yang dilakukan secara

bertahap dan iteratif dengan melibatkan pengguna dalam proses evaluasi sistem secara terus-menerus. Dalam penelitian ini, metode tersebut digunakan untuk membantu pengembangan platform digital ketenagakerjaan yang mencakup fitur *guest mode*, mekanisme pemilihan peran, serta integrasi antara aplikasi Android dan *website* admin, agar sistem yang dibangun dapat menyesuaikan kebutuhan pengguna secara fleksibel, terstruktur, dan efisien.

C. Perancangan Sistem

1. Arsitektur Sistem

Platform yang dibangun dalam penelitian ini menerapkan arsitektur *client-server* dengan tiga komponen utama yang saling terhubung. Komponen pertama adalah aplikasi Android berbasis Flutter yang berperan sebagai *client* utama bagi seluruh jenis pengguna, yaitu *Guest*, *Job Seeker*, dan *Employer*, untuk mengakses fitur sistem melalui perangkat *smartphone*. Komponen kedua adalah *website* berbasis Laravel (PHP) yang berperan ganda, yaitu sebagai panel *web admin* untuk pengelolaan data oleh *Administrator*, sekaligus sebagai *backend* yang menyediakan REST API untuk komunikasi dengan aplikasi Android. Komponen ketiga adalah basis data MySQL yang berperan sebagai penyimpanan terpusat seluruh data sistem, diakses secara eksklusif oleh *backend* Laravel melalui *Eloquent ORM*.

Komunikasi antara aplikasi Android dan *backend* dilakukan melalui REST API dengan menggunakan format pertukaran data JSON serta mekanisme autentikasi berbasis *JSON Web Token* (JWT). *Endpoint* API yang bersifat publik, yaitu *endpoint* daftar pekerjaan dan pencarian pekerjaan, dapat diakses tanpa JWT sehingga mendukung fitur *guest mode*. Sementara itu, seluruh *endpoint* lainnya yang memerlukan identitas pengguna, seperti pratinjau detail pekerjaan, pengajuan lamaran, dan pengelolaan profil, wajib menyertakan JWT sebagai *header* autentikasi. Data penelusuran yang dilakukan

oleh *Guest* disimpan sementara dalam *session* aplikasi dan tidak tersimpan secara permanen di basis data.

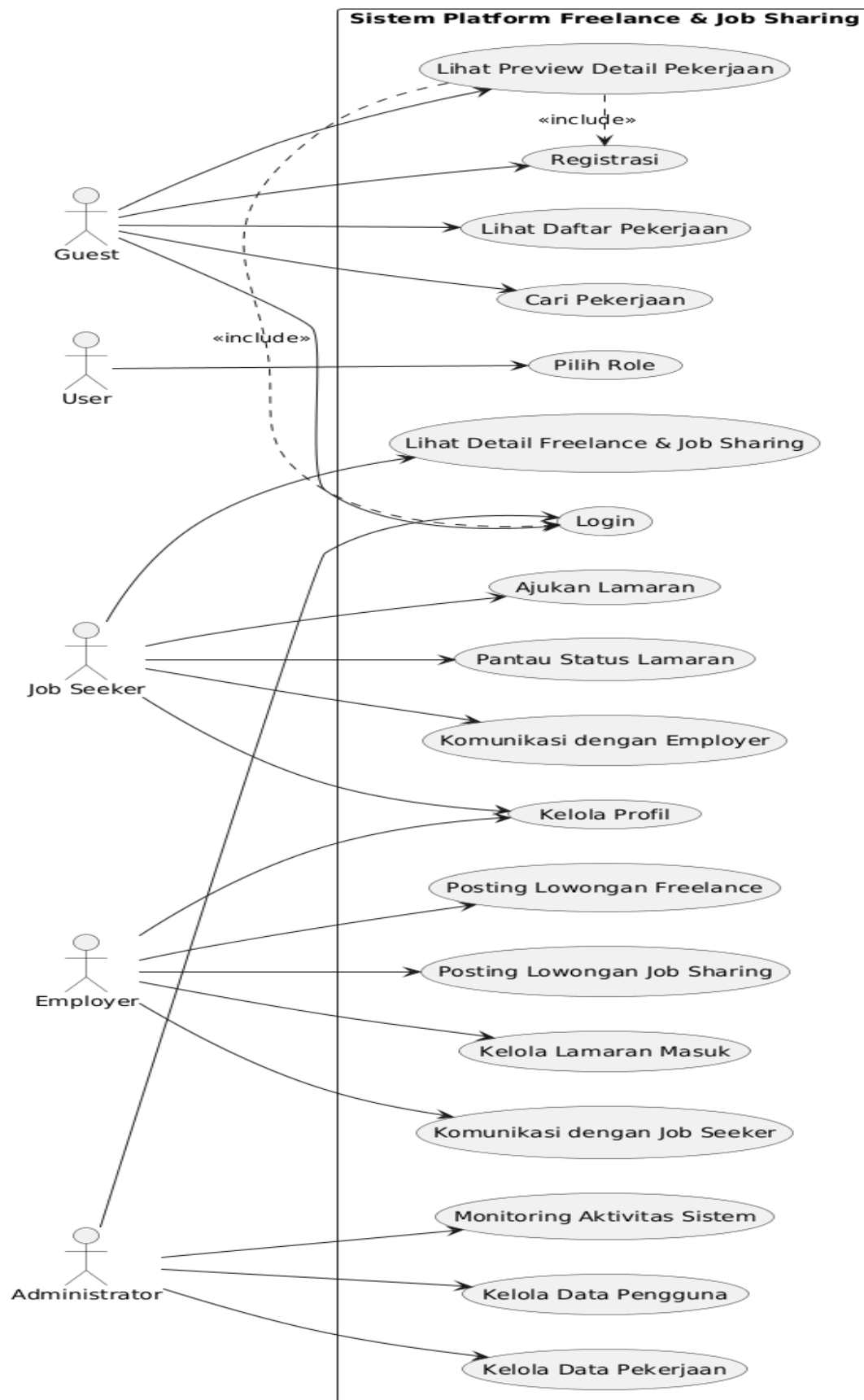
2. Perancangan Use Case Diagram

Use Case Diagram digunakan untuk menggambarkan interaksi antara pengguna (*actor*) dengan sistem secara keseluruhan. Sistem ini memiliki empat *actor* utama, yaitu *Guest*, *Job Seeker*, *Employer*, dan *Administrator*, dengan masing-masing *use case* sebagai berikut.

Guest merupakan pengguna yang belum melakukan *login* dan hanya dapat melakukan penelusuran daftar pekerjaan serta pencarian pekerjaan berdasarkan kata kunci atau kategori. Apabila *Guest* mencoba mengakses pratinjau detail pekerjaan, sistem akan menampilkan halaman peringatan yang mengarahkan pengguna untuk melakukan *login* atau registrasi terlebih dahulu.

User yang telah berhasil *login* akan diarahkan ke halaman pemilihan peran (*role selection*). Apabila pengguna memilih peran sebagai *Job Seeker*, maka *use case* yang tersedia meliputi mengelola profil, melihat pratinjau detail pekerjaan *freelance* dan *job sharing*, mengajukan lamaran, memantau status lamaran, dan berkomunikasi dengan pemberi kerja. Apabila pengguna memilih peran sebagai *Employer*, maka *use case* yang tersedia meliputi mengelola profil, memposting lowongan pekerjaan *freelance* dan *job sharing*, mengelola lamaran yang masuk, dan berkomunikasi dengan pencari kerja. Satu akun *User* dapat menjalankan kedua peran tersebut secara bergantian sesuai kebutuhan. *Administrator* dapat melakukan *login* melalui panel *web admin*, mengelola data pengguna, mengelola data pekerjaan, serta memantau aktivitas sistem secara keseluruhan.

Berikut gambaran use case digaram :



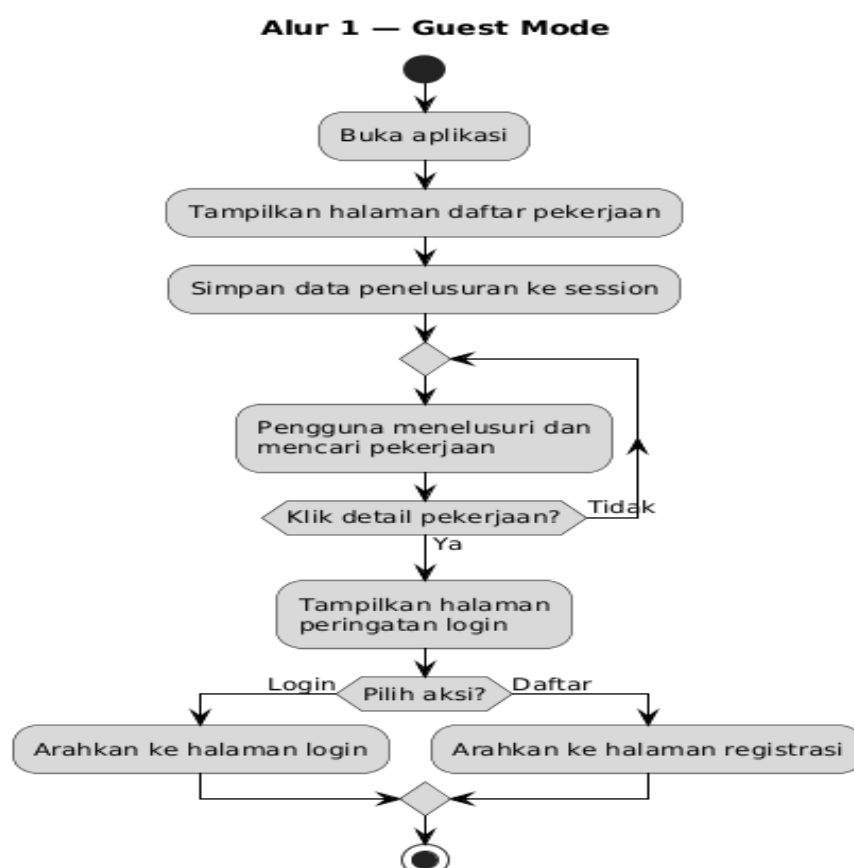
gambar 1.use case digaram

3. Perancangan Activity Diagram

Activity Diagram menggambarkan alur aktivitas utama yang terjadi di dalam sistem. Alur proses utama dalam sistem ini adalah sebagai berikut.

a. Alur 1--Guest mode

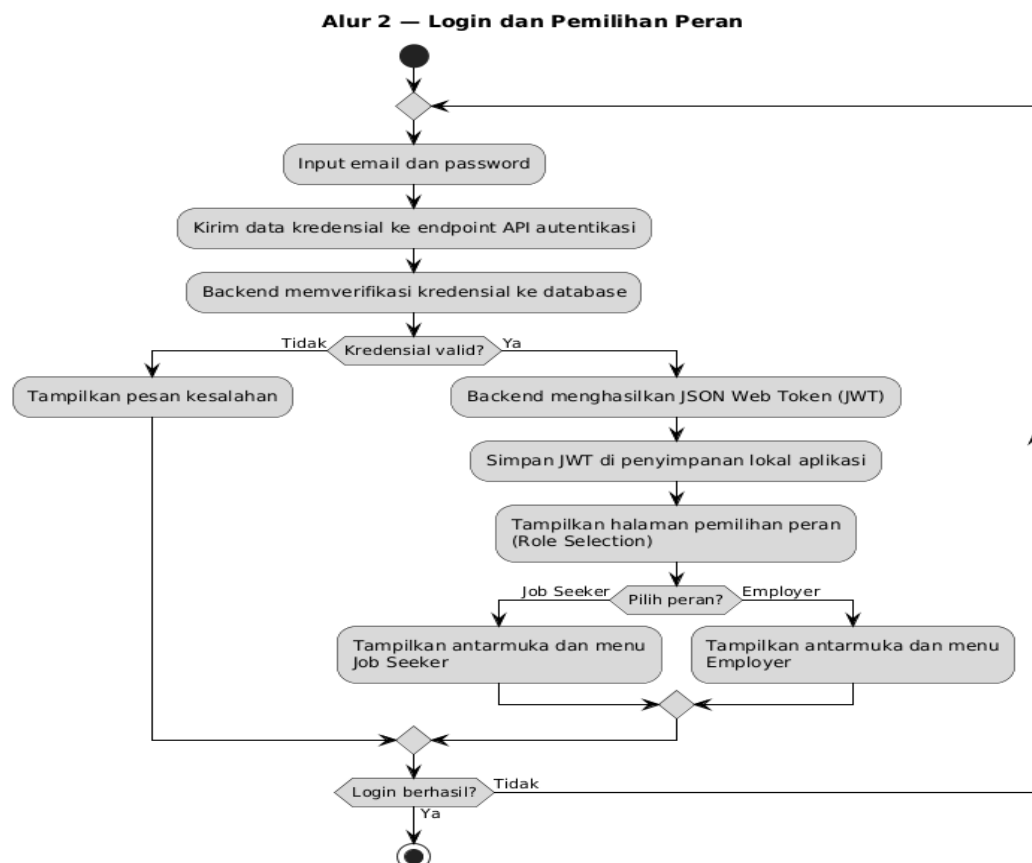
Proses pertama adalah alur *guest mode*. Saat pertama kali membuka aplikasi tanpa *login*, pengguna langsung diarahkan ke halaman daftar pekerjaan. Pengguna dapat melakukan pencarian pekerjaan berdasarkan kata kunci atau kategori, dan data penelusuran disimpan sementara dalam *session* aplikasi. Apabila pengguna mencoba mengakses pratinjau detail pekerjaan, sistem menampilkan halaman peringatan dan mengarahkan pengguna ke halaman *login* atau registrasi.



gambar 2.activity diagram guest mode

b. Alur 2 — Login dan Pemilihan Peran

Proses kedua adalah alur *login* dan pemilihan peran. Pengguna memasukkan *email* dan *password* pada halaman *login*. Sistem mengirimkan data kredensial ke *endpoint* API autentikasi. *Backend* memverifikasi kredensial dengan mencocokkan data di basis data. Apabila tidak valid, sistem menampilkan pesan kesalahan. Apabila valid, *backend* menghasilkan JWT dan mengirimkannya kepada aplikasi, kemudian JWT disimpan di penyimpanan lokal aplikasi. Selanjutnya, pengguna diarahkan ke halaman pemilihan peran (*role selection*) untuk memilih apakah akan masuk sebagai *Job Seeker* atau *Employer*. Berdasarkan peran yang dipilih, sistem menampilkan antarmuka dan menu yang berbeda sesuai dengan hak akses masing-masing peran.

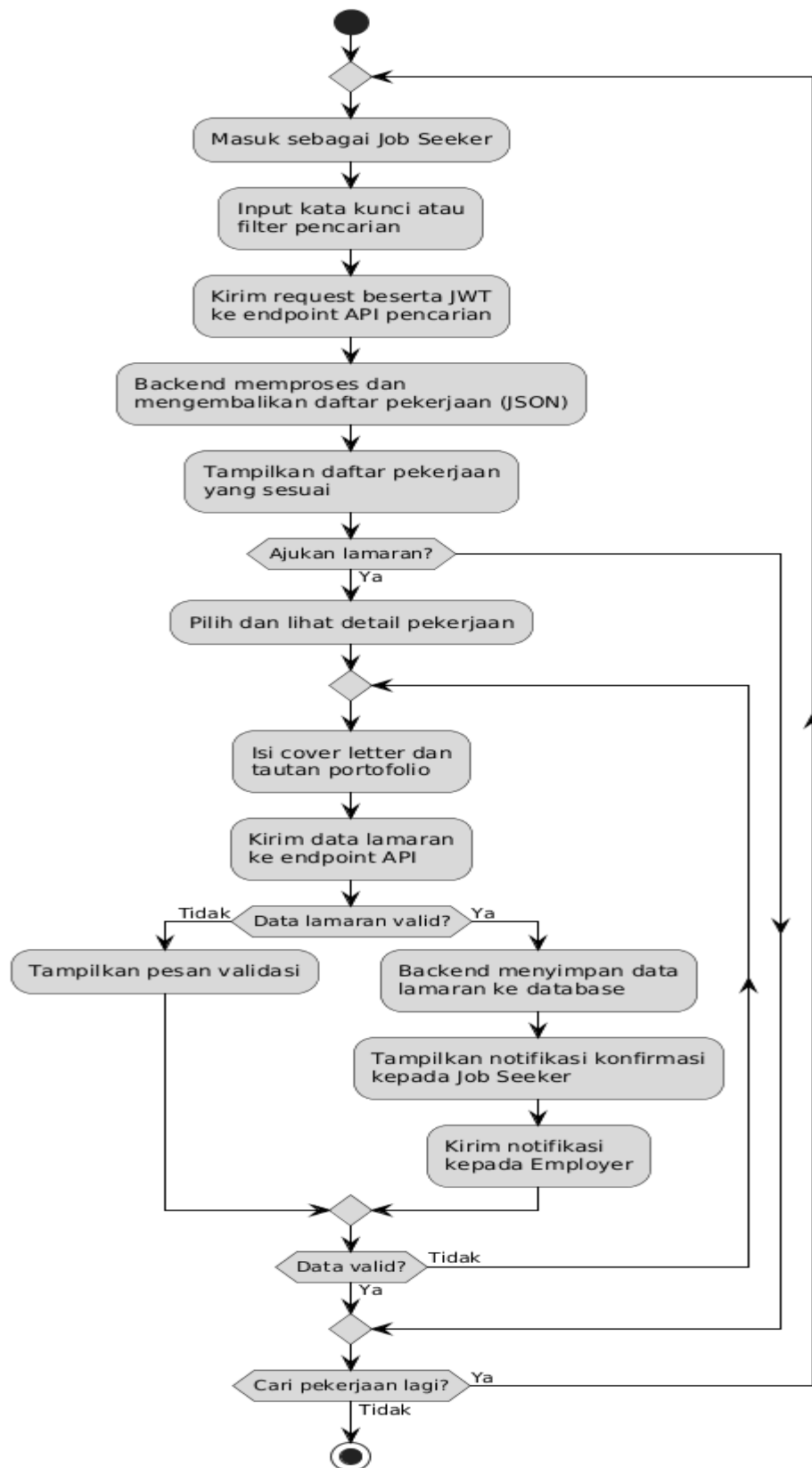


gambar 3. Activity diagram login

c. Alur 3 — Pencarian dan Lamaran Pekerjaan (Job Seeker)

Proses ketiga adalah alur pencarian dan lamaran pekerjaan oleh *Job Seeker*. Setelah memilih peran sebagai *Job Seeker*, pengguna dapat mengakses fitur pencarian pekerjaan dengan memasukkan kata kunci atau filter yang diinginkan. Aplikasi mengirimkan permintaan ke *endpoint* API pencarian pekerjaan dengan menyertakan JWT sebagai *header* autentikasi. *Backend* memproses permintaan dan mengembalikan daftar pekerjaan yang sesuai dalam format JSON. Pengguna memilih pekerjaan yang diminati, mengakses pratinjau detail pekerjaan, dan mengajukan lamaran. *Backend* menyimpan data lamaran ke basis data dan mengirimkan notifikasi kepada pemberi kerja.

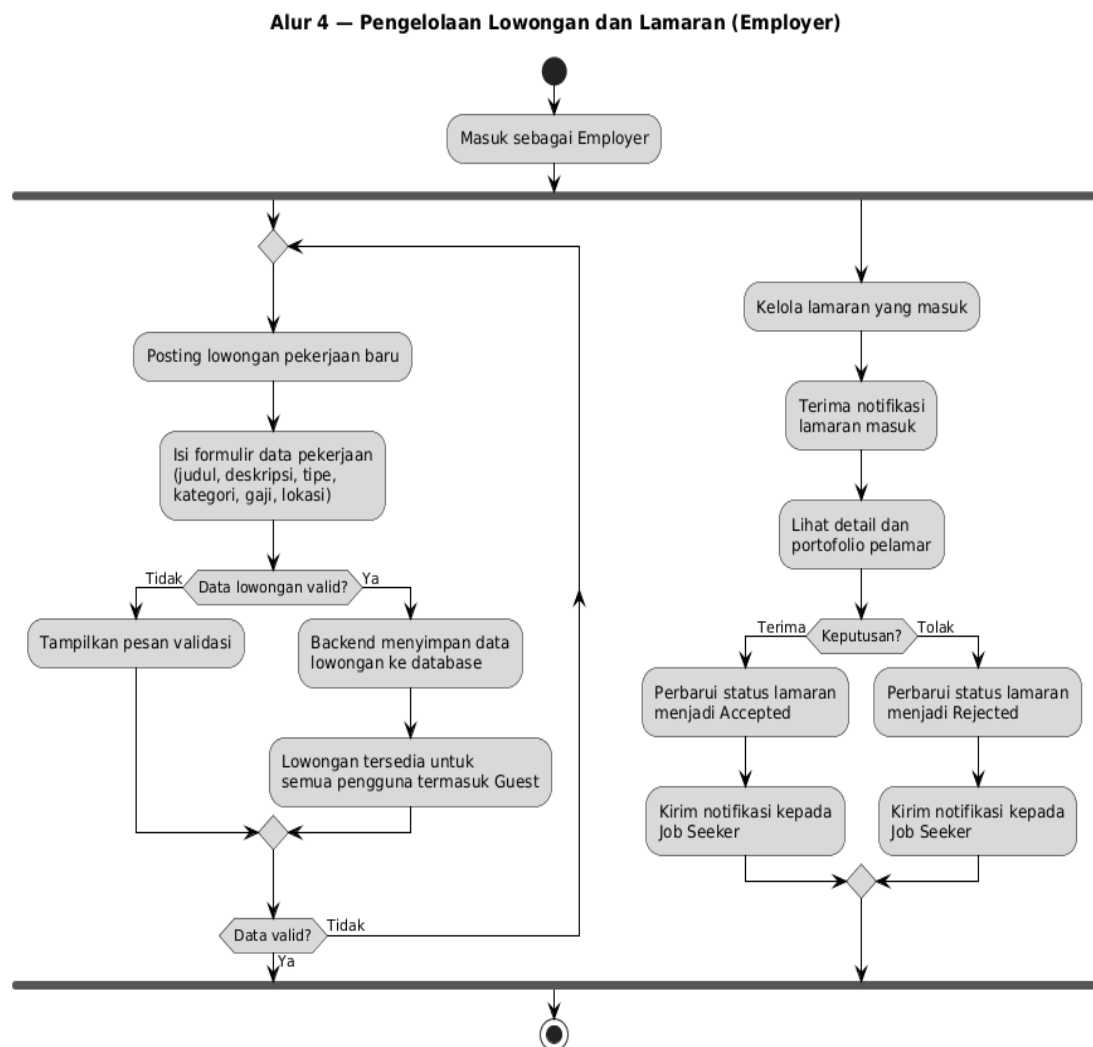
Alur 3 — Pencarian dan Lamaran Pekerjaan (Job Seeker)



gambar 4. Activity diagram pencarian pekerjaan

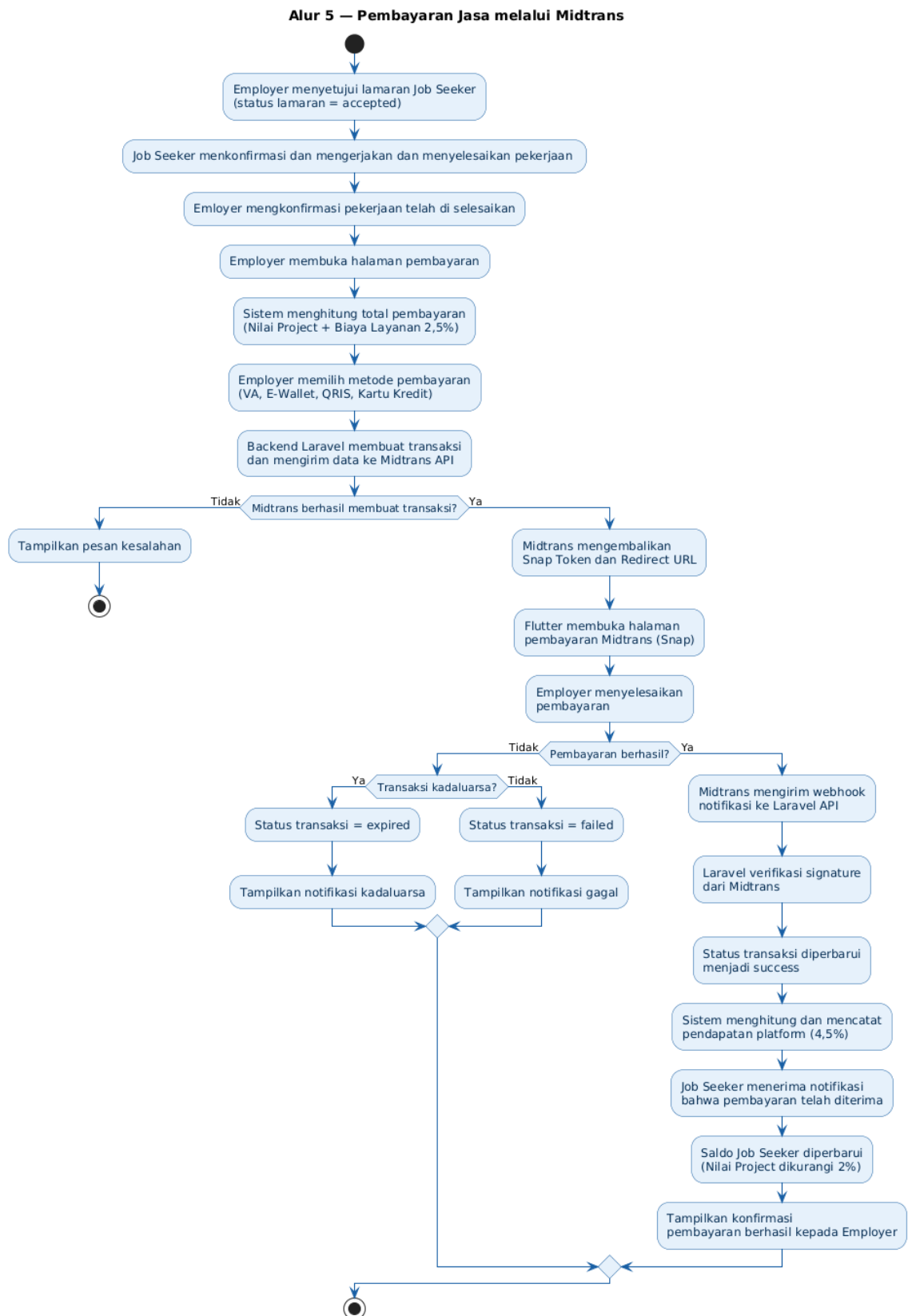
d. Alur 4 — Pengelolaan Lowongan dan Lamaran (Employer)

Proses keempat adalah alur pengelolaan lowongan oleh *Employer*. Setelah memilih peran sebagai *Employer*, pengguna dapat memposting lowongan pekerjaan baru dengan mengisi data pekerjaan yang dibutuhkan. *Backend* menyimpan data lowongan ke basis data dan menjadikannya tersedia untuk dilihat oleh seluruh pengguna termasuk *Guest*. *Employer* juga dapat mengelola lamaran yang masuk dengan memperbarui status lamaran menjadi *accepted* atau *rejected*.



gambar 5. Activity diagram pengelolaan lowongan pekerjaan

e. Alur 5 -pembayaran jasa melalui midtrans



gambar 6.Activity diagram pemebayaran jasa

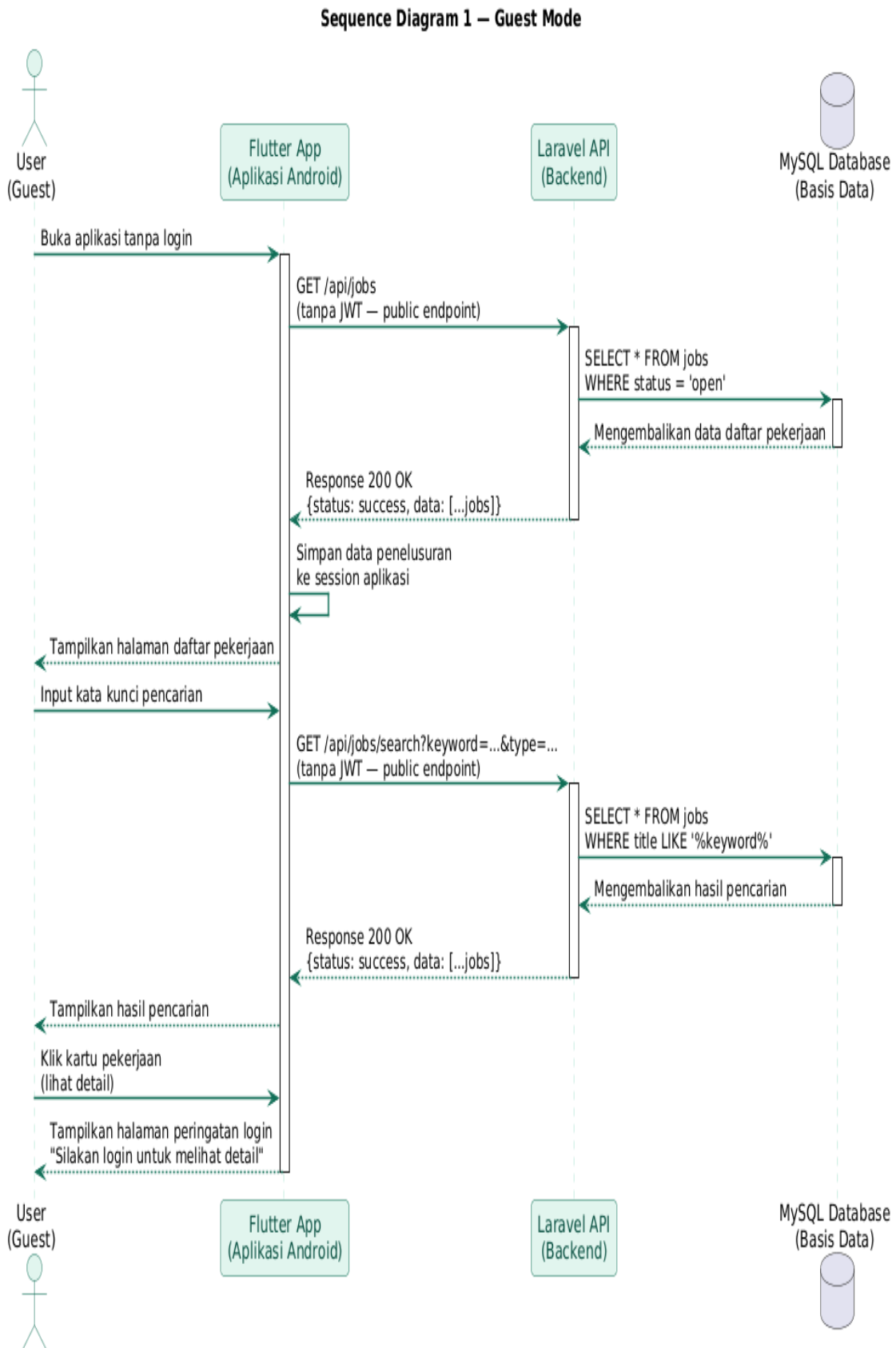
4. Perancangan Sequence Diagram

Sequence Diagram menggambarkan urutan interaksi antar objek dalam sistem berdasarkan waktu. Interaksi utama dalam sistem ini melibatkan empat komponen, yaitu *User* (pengguna melalui aplikasi Android), *Flutter App* (aplikasi Android), *Laravel API* (*backend*), dan *MySQL Database* (basis data).

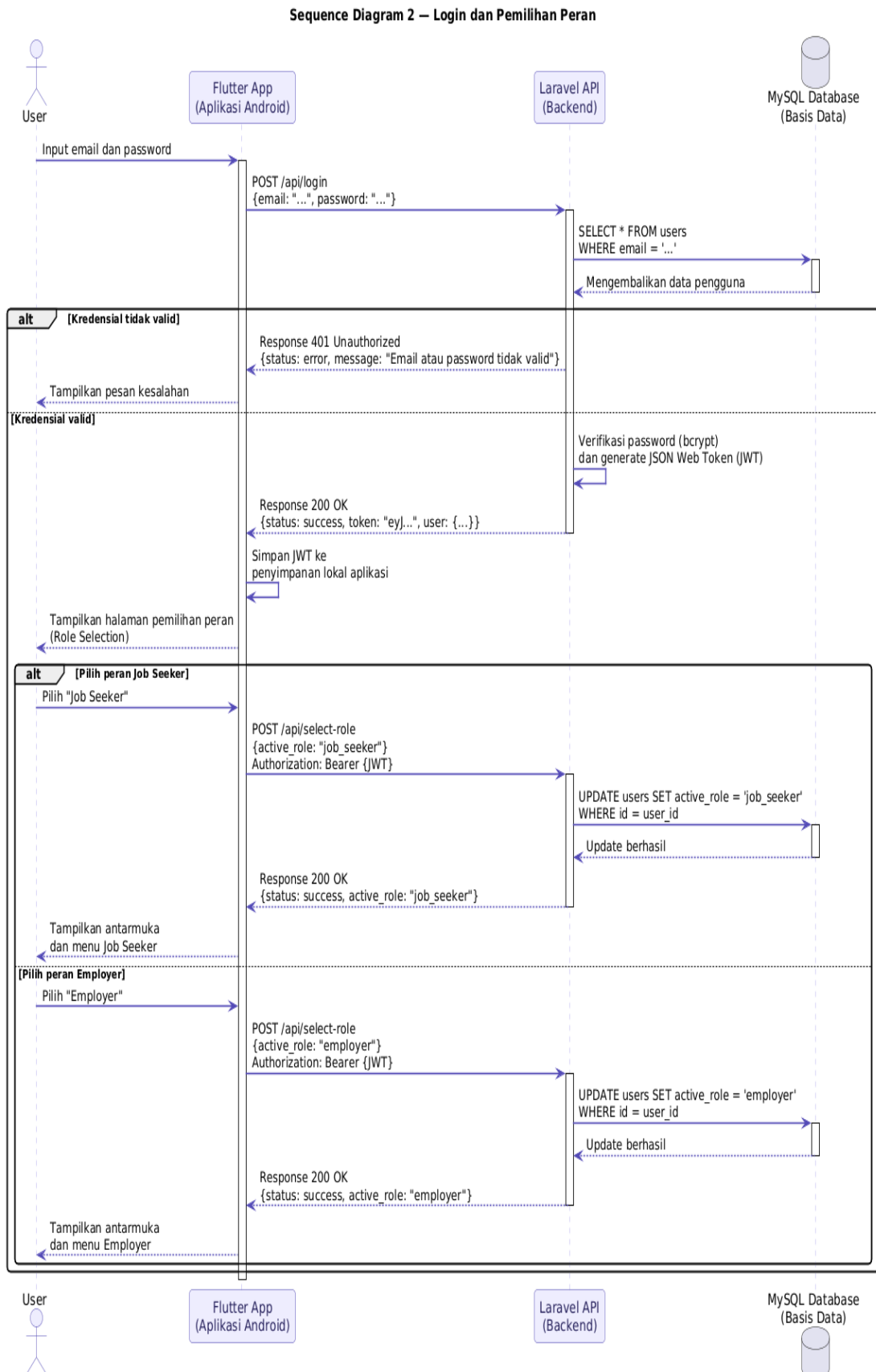
Pada proses *guest mode*, pengguna membuka aplikasi tanpa *login* dan *Flutter App* secara otomatis mengirimkan permintaan *GET /api/jobs* ke *Laravel API* tanpa menyertakan JWT. *Laravel API* memproses permintaan tersebut sebagai *public endpoint* dan mengambil data daftar pekerjaan dari *MySQL Database*, kemudian mengembalikan data dalam format JSON kepada *Flutter App* untuk ditampilkan kepada pengguna. Data penelusuran disimpan sementara dalam *session* aplikasi oleh *Flutter App*.

Pada proses *login* dan pemilihan peran, pengguna memasukkan kredensial pada antarmuka aplikasi, kemudian *Flutter App* mengirimkan permintaan *POST /api/login* ke *Laravel API* dengan menyertakan *email* dan *password* dalam format JSON. *Laravel API* meneruskan permintaan verifikasi ke *MySQL Database*. Apabila data valid, *MySQL Database* mengembalikan data pengguna ke *Laravel API*, kemudian *Laravel API* menghasilkan JWT dan mengirimkannya kembali ke *Flutter App* dalam format JSON. *Flutter App* menyimpan JWT dan menampilkan halaman pemilihan peran (*role selection*) kepada pengguna. Berdasarkan peran yang dipilih, *Flutter App* menampilkan antarmuka yang sesuai dengan peran tersebut.

Berikut adalah beberapa rancangan sequence diagram pada 5 proses yaitu *guest mode*, *login* dan pemilihan peran pencarian dan lamaran pekerjaan, pengelolaan lamaran dan lowongan, dan transaksi pembayaran jasa.

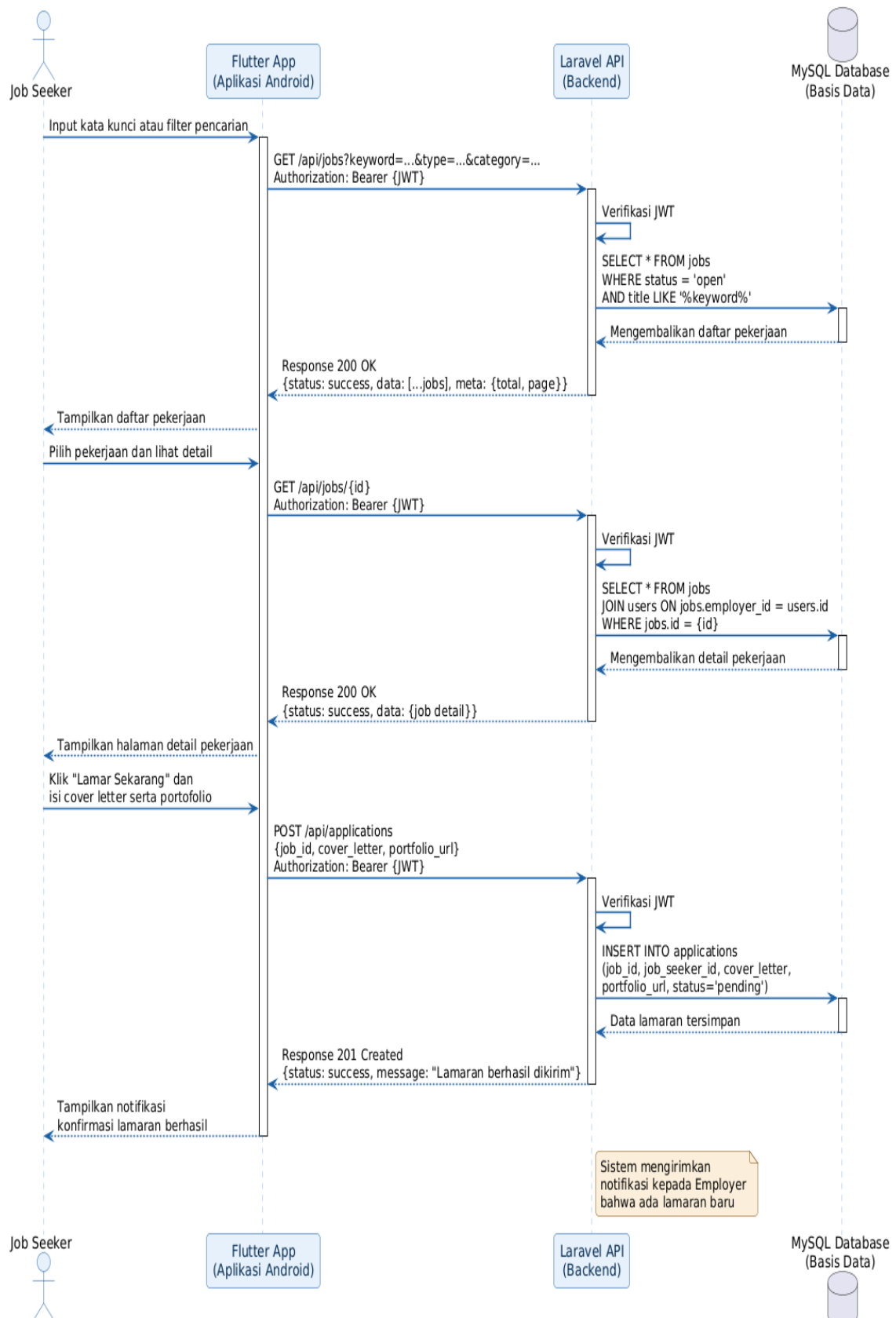


gambar 7.sequence diagram guest-mode

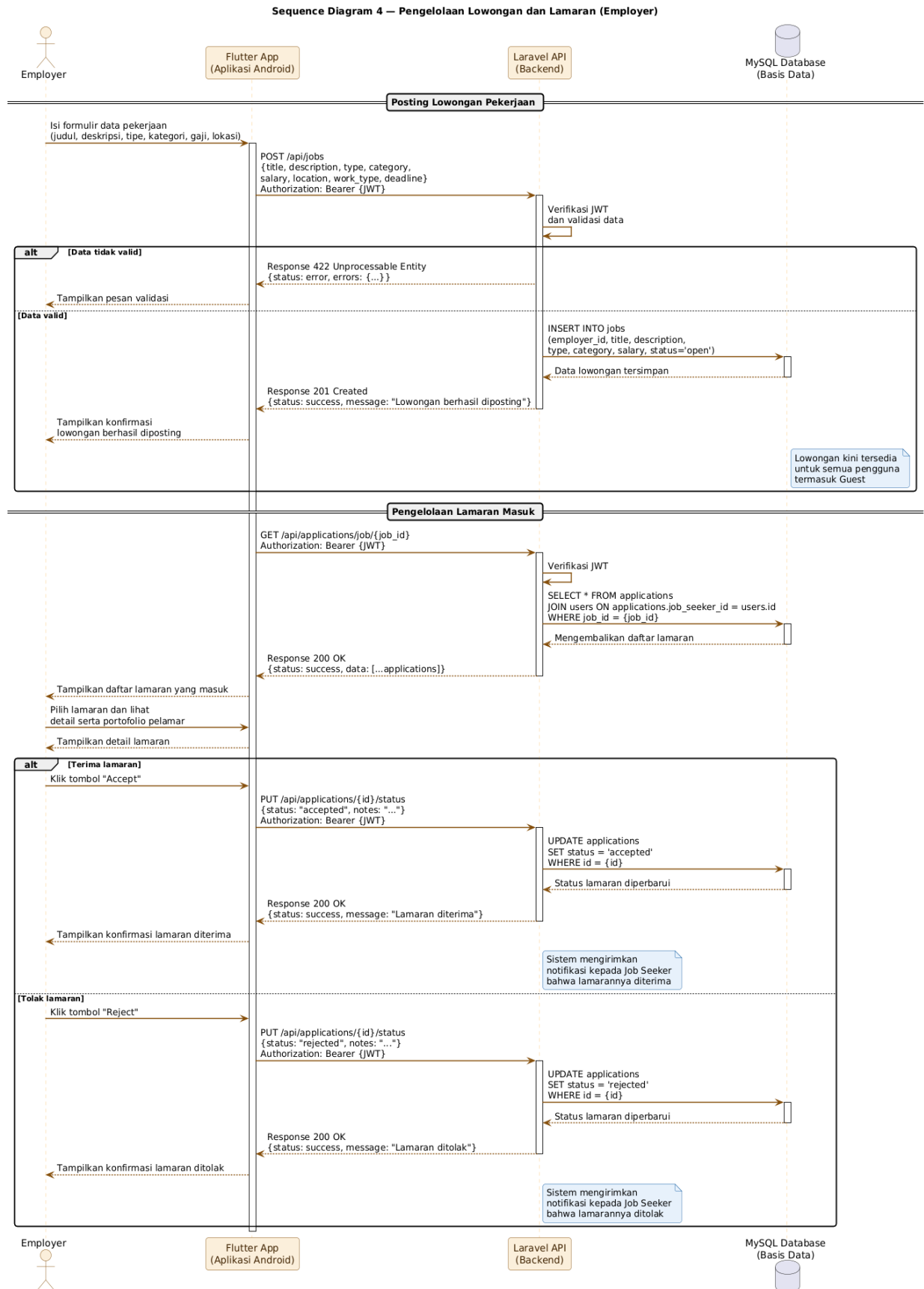


gambar 8.sequence login dan pemeilihan peran

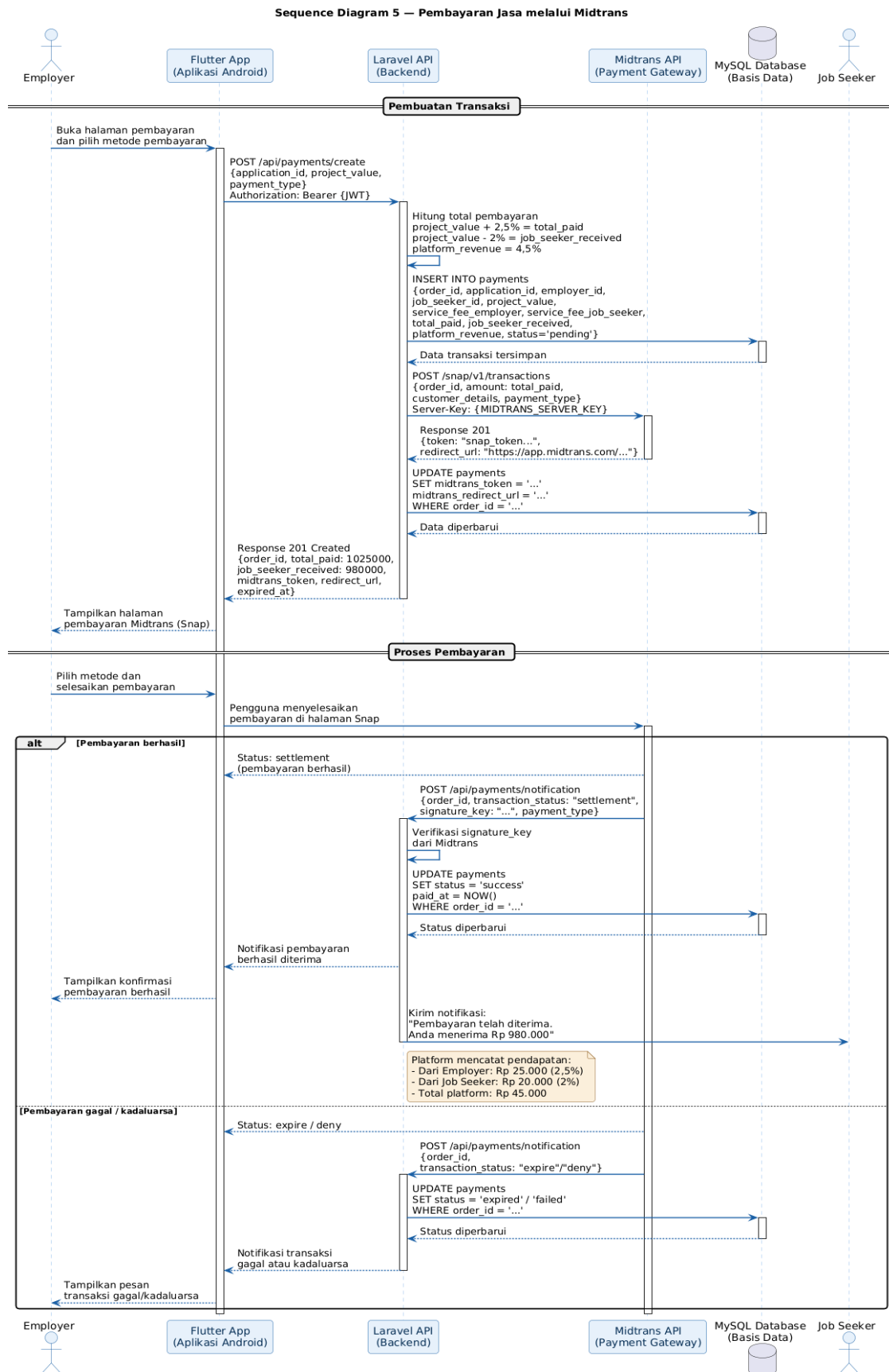
Sequence Diagram 3 – Pencarian dan Lamaran Pekerjaan (Job Seeker)



gambar 9.sequence diagram pencarian lamaran



gambar 10.sequence diagram pengelolaan lamaran

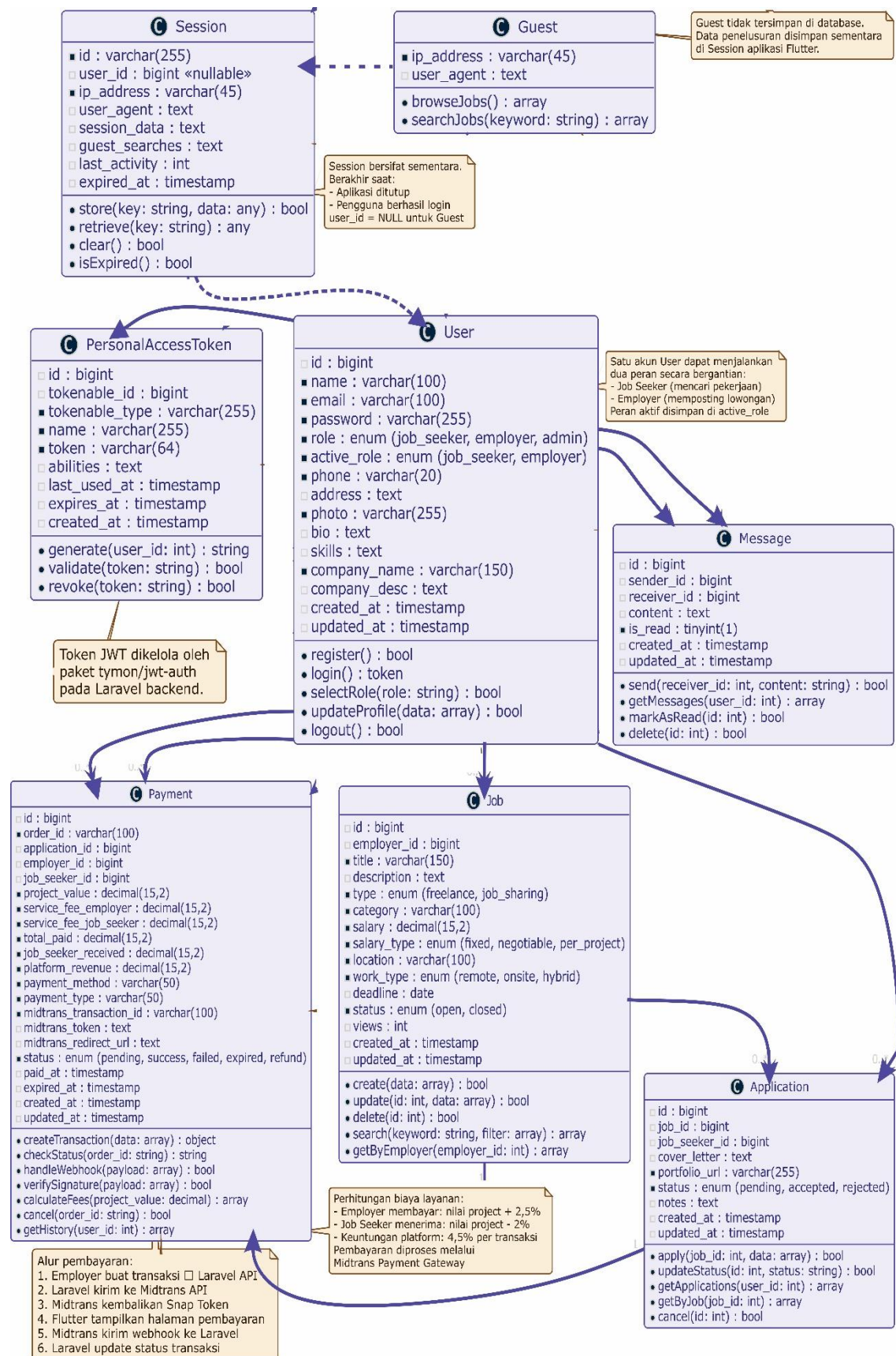


gambar 11.sequence diagram pembayaran jasa

5. Perancangan Class Diagram

Kelas *User* memiliki atribut *id*, *name*, *email*, *password*, *role* (*job_seeker/ employer/admin*), *active_role* (peran yang sedang aktif saat *login*), *phone*, *address*, dan *created_at*, dengan metode *register()*, *login()*, *selectRole()*, *updateProfile()*, dan *logout()*. Penambahan atribut *active_role* dan metode *selectRole()* bertujuan untuk mendukung mekanisme pemilihan peran dinamis antara *Job Seeker* dan *Employer* dalam satu akun yang sama. Kelas *Job* memiliki atribut *id*, *employer_id*, *title*, *description*, *type* (*freelance/job_sharing*), *category*, *salary*, *location*, *status*, dan *created_at*, dengan metode *create()*, *update()*, *delete()*, dan *search()*. Kelas *Application* memiliki atribut *id*, *job_id*, *job_seeker_id*, *cover_letter*, *status* (*pending/ accepted/ rejected*), dan *created_at*, dengan metode *apply()*, *updateStatus()*, dan *getApplications()*. Kelas *Message* memiliki atribut *id*, *sender_id*, *receiver_id*, *content*, *is_read*, dan *created_at*, dengan metode *send()*, *getMessages()*, dan *markAsRead()*. Kelas *Session* memiliki atribut *id*, *session_data*, *guest_searches*, dan *expired_at*, dengan metode *store()*, *retrieve()*, dan *clear()*, yang berfungsi untuk menyimpan data penelusuran pengguna *Guest* secara sementara. Hubungan antar kelas tersebut adalah sebagai berikut. *User* memiliki hubungan *one-to-many* dengan *Job* (satu *Employer* dapat memiliki banyak pekerjaan). *Job* memiliki hubungan *one-to-many* dengan *Application* (satu pekerjaan dapat memiliki banyak lamaran). *User* memiliki hubungan *one-to-many* dengan *Application* (satu *Job Seeker* dapat mengajukan banyak lamaran). *User* memiliki hubungan *one-to-many* dengan *Message* (satu pengguna dapat mengirim dan menerima banyak pesan). *Guest* memiliki hubungan sementara dengan *Session* yang berakhir saat aplikasi ditutup atau pengguna berhasil melakukan *login*.

Berikut adalah gambaran dari class diagram:



gambar 12.use case diagram

D. Perancangan Basis Data

Perancangan basis data sistem ini menggunakan MySQL dengan pendekatan relasional. Basis data dirancang untuk mendukung seluruh fitur sistem yang telah diidentifikasi pada sub bab sebelumnya, mencakup fitur *guest mode*, mekanisme pemilihan peran (*role selection*), pengelolaan pekerjaan *freelance* dan *job sharing*, pengajuan lamaran, serta komunikasi antar pengguna. Berikut adalah rancangan tabel-tabel utama yang terdapat dalam basis data sistem.

1. Tabel users

Tabel *users* merupakan tabel utama yang menyimpan data seluruh pengguna sistem yang telah melakukan registrasi, mencakup pengguna dengan peran *Job Seeker*, *Employer*, maupun *Administrator*. Tabel ini dirancang untuk mendukung mekanisme pemilihan peran dinamis, di mana satu akun pengguna dapat menjalankan peran sebagai *Job Seeker* maupun *Employer* secara bergantian.

Tabel 1.users tabel

No	Nama Kolom	Tipe Data	Keterangan
1	id	<i>bigint(20)</i>	<i>Primary Key, Auto Increment</i>
2	name	<i>varchar(100)</i>	Nama lengkap pengguna
3	email	<i>varchar(100)</i>	Email pengguna, bersifat <i>unique</i>
4	password	<i>varchar(255)</i>	Password terenkripsi (<i>bcrypt</i>)
5	role	<i>enum</i>	Peran tetap: <i>job_seeker, employer, admin</i>
6	active_role	<i>enum</i>	Peran aktif saat <i>login</i> : <i>job_seeker, employer</i>
7	phone	<i>varchar(20)</i>	Nomor telepon pengguna
8	address	<i>text</i>	Alamat pengguna
9	photo	<i>varchar(255)</i>	Path foto profil pengguna
10	bio	<i>text</i>	Deskripsi singkat pengguna
11	skills	<i>text</i>	Keahlian pengguna (<i>Job Seeker</i>)
12	company_name	<i>varchar(150)</i>	Nama perusahaan (<i>Employer</i>)
13	company_desc	<i>text</i>	Deskripsi perusahaan (<i>Employer</i>)
14	email_verified_at	<i>timestamp</i>	Waktu verifikasi email
15	remember_token	<i>varchar(100)</i>	Token <i>remember me</i>
16	created_at	<i>timestamp</i>	Waktu data dibuat
17	updated_at	<i>timestamp</i>	Waktu data diperbarui

2. Tabel jobs

Tabel *jobs* menyimpan data seluruh lowongan pekerjaan yang diposting oleh pengguna dengan peran *Employer*. Tabel ini mendukung dua jenis pekerjaan, yaitu *freelance* dan *job sharing*, serta dapat diakses secara public seperti *guest*.

Tabel 2. *jobs* tabel

No	Nama Kolom	Type Data	Keterangan
1	id	bigint(20)	Primary Key, Auto Increment
2	employer_id	bigint(20)	Foreign Key → users.id
3	title	varchar(150)	Judul pekerjaan
4	description	text	Deskripsi lengkap pekerjaan
5	type	enum	Jenis: <i>freelance</i> , <i>job_sharing</i>
6	category	varchar(100)	Kategori bidang pekerjaan
7	salary	decimal(15,2)	Nominal gaji atau bayaran
8	salary_type	enum	Jenis bayaran: <i>fixed</i> , <i>negotiable</i> , <i>per_project</i>
9	location	varchar(100)	Lokasi pekerjaan
10	work_type	enum	Jenis kerja: <i>remote</i> , <i>onsite</i> , <i>hybrid</i>
11	deadline	date	Batas waktu pendaftaran
12	status	enum	Status: <i>open</i> , <i>closed</i>
13	views	int(11)	Jumlah tampilan lowongan
14	created_at	timestamp	Waktu data dibuat
15	updated_at	timestamp	Waktu data diperbarui

3. Tabel applications

Tabel *applications* menyimpan data seluruh lamaran pekerjaan yang diajukan oleh pengguna dengan peran *Job Seeker*. Setiap lamaran terhubung dengan satu pekerjaan dan satu pengguna pencari kerja melalui *foreign key*.

Tabel 3. *Application* tabel

No	Nama Kolom	Type Data	Keterangan
1	id	bigint(20)	Primary Key, Auto Increment
2	job_id	bigint(20)	Foreign Key → jobs.id
3	job_seeker_id	bigint(20)	Foreign Key → users.id
4	cover_letter	text	Surat lamaran pengguna
5	portfolio_url	varchar(255)	Tautan portofolio pelamar
6	status	enum	Status: <i>pending</i> , <i>accepted</i> , <i>rejected</i>
7	notes	text	Catatan dari <i>Employer</i>
8	created_at	timestamp	Waktu lamaran diajukan
9	updated_at	timestamp	Waktu data diperbarui

4. Tabel messages

Tabel *messages* menyimpan data pesan komunikasi antara *Job Seeker* dan *Employer*. Fitur komunikasi ini hanya dapat diakses oleh pengguna yang telah *login* dan tidak tersedia bagi *Guest*.

Tabel 4. Messege tabel

No	Nama Kolom	Tipe Data	Keterangan
1	id	<i>bigint(20)</i>	<i>Primary Key, Auto Increment</i>
2	sender_id	<i>bigint(20)</i>	<i>Foreign Key → users.id (pengirim)</i>
3	receiver_id	<i>bigint(20)</i>	<i>Foreign Key → users.id (penerima)</i>
4	content	<i>text</i>	Isi pesan
5	is_read	<i>tinyint(1)</i>	Status baca: 0 = belum, 1 = sudah
6	created_at	<i>timestamp</i>	Waktu pesan dikirim
7	updated_at	<i>timestamp</i>	Waktu data diperbarui

5. Tabel personal_access_tokens

Tabel *personal_access_tokens* menyimpan data token JWT yang dihasilkan oleh sistem setelah pengguna berhasil melakukan *login*. Tabel ini dikelola secara otomatis oleh paket *tymon/jwt-auth* pada Laravel dan berperan penting dalam mendukung mekanisme autentikasi *stateless* berbasis token.

Tabel 5. personal acces tokens tabel

No	Nama Kolom	Tipe Data	Keterangan
1	id	<i>bigint(20)</i>	<i>Primary Key, Auto Increment</i>
2	tokenable_type	<i>varchar(255)</i>	Tipe model pemilik token
3	tokenable_id	<i>bigint(20)</i>	ID pemilik token (<i>users.id</i>)
4	name	<i>varchar(255)</i>	Nama token
5	token	<i>varchar(64)</i>	Token yang di-hash
6	abilities	<i>text</i>	Hak akses token
7	last_used_at	<i>timestamp</i>	Waktu token terakhir digunakan
8	expires_at	<i>timestamp</i>	Waktu kadaluarsa token
9	created_at	<i>timestamp</i>	Waktu token dibuat
10	updated_at	<i>timestamp</i>	Waktu data diperbarui

6. Tabel sessions

Tabel *sessions* menyimpan data *session* pengguna secara sementara, termasuk data penelusuran yang dilakukan oleh *Guest* sebelum melakukan *login*. Data pada tabel ini bersifat sementara dan akan otomatis dihapus setelah *session* berakhir atau pengguna berhasil melakukan *login*.

Tabel 6.session tabel

No	Nama Kolom	Tipe Data	Keterangan
1	id	varchar(255)	Primary Key, ID session unik
2	user_id	bigint(20)	Foreign Key → users.id (nullable untuk Guest)
3	ip_address	varchar(45)	Alamat IP pengguna
4	user_agent	text	Informasi perangkat pengguna
5	payload	longtext	Data session dalam format terenkripsi
6	guest_searches	text	Data pencarian Guest (JSON)
7	last_activity	int(11)	Waktu aktivitas terakhir (Unix timestamp)

7. Tabel payments

Tabel *payment* menyimpan data seluruh transaksi pembayaran yang terjadi di dalam sistem, metode pembayaran otomatis di jalankan oleh system payment gateway berdasarkan request dari jSON, dengan menggunakan pihak ketiga yaitu midtrans payments gateway maka dapat memperingkas alur kerja aplikasi dan mempermudah pengembangan.

Tabel 7.Payments table

No	Nama Kolom	Tipe Data	Keterangan
1	id	bigint(20)	Primary Key, Auto Increment
2	order_id	varchar(100)	ID unik transaksi (unique)
3	application_id	bigint(20)	Foreign Key → applications.id
4	employer_id	bigint(20)	Foreign Key → users.id (pembayar)
5	job_seeker_id	bigint(20)	Foreign Key → users.id (penerima)
6	project_value	decimal(15,2)	Nilai project yang disepakati
7	service_fee_employer	decimal(15,2)	Biaya layanan Employer (2,5%)
8	service_fee_job_seeker	decimal(15,2)	Potongan Job Seeker (2%)

9	total_paid	decimal(15,2)	Total yang dibayar Employer
10	job_seeker_received	decimal(15,2)	Total yang diterima Job Seeker
11	platform_revenue	decimal(15,2)	Total keuntungan platform (4,5%)
12	payment_method	varchar(50)	Metode pembayaran yang dipilih
13	payment_type	varchar(50)	Tipe: bank_transfer, e-wallet, qris, credit_card
14	midtrans_transaction_id	varchar(100)	ID transaksi dari Midtrans
15	midtrans_token	text	Token pembayaran dari Midtrans
16	midtrans_redirect_url	text	URL halaman pembayaran Midtrans
17	status	enum	Status: pending, success, failed, expired, refund
18	paid_at	timestamp	Waktu pembayaran berhasil
19	expired_at	timestamp	Waktu kadaluarsa transaksi
20	created_at	timestamp	Waktu transaksi dibuat
21	updated_at	timestamp	Waktu data diperbarui

8. Relasi Antar Tabel

Relasi antar tabel dalam basis data sistem ini adalah sebagai berikut.

Pertama, tabel *users* berelasi *one-to-many* dengan tabel *jobs*, di mana satu pengguna dengan peran *Employer* dapat memiliki banyak data pekerjaan yang diposting melalui kolom *employer_id* pada tabel *jobs*.

Kedua, tabel *jobs* berelasi *one-to-many* dengan tabel *applications*, di mana satu pekerjaan dapat memiliki banyak data lamaran yang masuk melalui kolom *job_id* pada tabel *applications*.

Ketiga, tabel *users* berelasi *one-to-many* dengan tabel *applications*, di mana satu pengguna dengan peran *Job Seeker* dapat mengajukan banyak lamaran pekerjaan melalui kolom *job_seeker_id* pada tabel *applications*.

Keempat, tabel *applications* berelasi *one-to-one* dengan tabel *payments*, di mana satu lamaran yang telah berstatus *accepted* dapat memiliki satu data transaksi

pembayaran melalui kolom *application_id* pada tabel *payments*. Relasi ini bersifat *nullable* karena tidak semua lamaran yang diterima langsung diproses pembayarannya.

Kelima, tabel *users* berelasi *one-to-many* dengan tabel *payments* melalui dua *foreign key*, yaitu kolom *employer_id* yang merujuk kepada pengguna dengan peran *Employer* sebagai pihak pembayar, dan kolom *job_seeker_id* yang merujuk kepada pengguna dengan peran *Job Seeker* sebagai pihak penerima pembayaran. Satu pengguna dengan peran *Employer* dapat melakukan banyak transaksi pembayaran, dan satu pengguna dengan peran *Job Seeker* dapat menerima banyak pembayaran dari berbagai *Employer*.

Keenam, tabel *users* berelasi *one-to-many* dengan tabel *messages* melalui dua *foreign key*, yaitu *sender_id* dan *receiver_id*, di mana satu pengguna dapat mengirim dan menerima banyak pesan komunikasi dengan pengguna lainnya.

Ketujuh, tabel *users* berelasi *one-to-many* dengan tabel *personal_access_tokens* melalui kolom *tokenable_id*, di mana satu pengguna dapat memiliki beberapa token JWT yang aktif secara bersamaan.

Kedelapan, tabel *sessions* berelasi *nullable* dengan tabel *users* melalui kolom *user_id*, di mana kolom tersebut bernilai *null* apabila *session* dimiliki oleh *Guest* yang belum melakukan *login*. Apabila pengguna berhasil melakukan *login*, kolom *user_id* akan diperbarui dengan *id* pengguna yang bersangkutan.

Berikut adalah rancangan *Entity-Relationship Diagram* (ERD) yang menggambarkan seluruh relasi antar tabel tersebut secara visual.

E. Perancangan Antarmuka

Perancangan antarmuka (*User Interface/UI*) merupakan tahapan dalam proses pengembangan sistem yang bertujuan untuk merancang tampilan visual dan alur interaksi yang akan digunakan oleh pengguna dalam mengoperasikan sistem. Perancangan antarmuka dalam penelitian ini mengacu pada prinsip desain yang mengutamakan kemudahan penggunaan (*usability*), konsistensi tampilan, dan kenyamanan pengguna. Perancangan antarmuka mencakup dua platform, yaitu aplikasi Android berbasis Flutter dan *website* admin berbasis Laravel.

1. Perancangan Antarmuka Aplikasi Android

Antarmuka aplikasi Android dirancang menggunakan Flutter dengan mengikuti panduan *Material Design* dari Google. Aplikasi dirancang untuk mendukung empat jenis pengguna, yaitu *Guest*, *Job Seeker*, *Employer*, dan *Administrator* melalui alur navigasi yang berbeda sesuai dengan status dan peran masing-masing pengguna. Berikut adalah rancangan halaman-halaman utama pada aplikasi Android.

a. Halaman Splash Screen

Halaman *splash screen* merupakan halaman pertama yang ditampilkan saat pengguna membuka aplikasi. Halaman ini menampilkan logo dan nama aplikasi di bagian tengah layar dengan latar belakang berwarna sesuai tema aplikasi. *Splash screen* ditampilkan selama kurang lebih dua detik sebelum sistem secara otomatis memeriksa status *login* pengguna. Apabila pengguna belum pernah *login* atau tidak memiliki token yang valid, sistem akan mengarahkan pengguna ke halaman daftar pekerjaan dalam mode *Guest*. Apabila pengguna sebelumnya telah *login* dan token masih valid, sistem akan mengarahkan pengguna langsung ke halaman pemilihan peran.

b. Halaman Daftar Pekerjaan (Guest Mode)

Halaman daftar pekerjaan merupakan halaman utama yang ditampilkan saat pengguna pertama kali membuka aplikasi tanpa melakukan *login* terlebih dahulu. Halaman ini menampilkan daftar lowongan pekerjaan dalam format kartu (*card*) yang tersusun secara vertikal. Setiap kartu pekerjaan menampilkan informasi ringkas yang terdiri dari judul pekerjaan, nama perusahaan, kategori, jenis pekerjaan (*freelance* atau *job sharing*), lokasi, dan nominal gaji. Di bagian atas halaman terdapat *search bar* untuk melakukan pencarian pekerjaan berdasarkan kata kunci, serta tombol filter untuk menyaring pekerjaan berdasarkan kategori, jenis pekerjaan, lokasi, dan jenis kerja (*remote*, *onsite*, atau *hybrid*). Di bagian pojok kanan atas terdapat tombol *login* yang dapat diakses apabila pengguna ingin masuk ke dalam sistem. Apabila pengguna menekan salah satu kartu pekerjaan untuk melihat detail, sistem akan menampilkan halaman peringatan yang mengarahkan pengguna untuk melakukan *login* atau registrasi terlebih dahulu.

c. Halaman Peringatan Login

Halaman peringatan *login* ditampilkan ketika *Guest* mencoba mengakses fitur yang memerlukan autentikasi, yaitu saat menekan kartu pekerjaan untuk melihat pratinjau detail. Halaman ini menampilkan pesan peringatan yang menginformasikan bahwa fitur tersebut hanya dapat diakses oleh pengguna yang telah *login*, disertai dengan dua tombol pilihan yaitu tombol *Login* yang mengarahkan pengguna ke halaman *login* dan tombol *Daftar* yang mengarahkan pengguna ke halaman registrasi.

Terdapat pula tautan untuk kembali ke halaman daftar pekerjaan apabila pengguna memilih untuk tetap menjelajahi aplikasi tanpa *login* sehingga membuat aplikasi ini akan terasa lebih fleksibel.

d. Halaman Registrasi

Halaman registrasi menampilkan formulir pendaftaran akun baru yang terdiri dari kolom nama lengkap, alamat *email*, *password*, dan konfirmasi *password*. Di bagian bawah formulir terdapat tombol *Daftar* untuk memproses registrasi dan tautan menuju halaman *login* bagi pengguna yang sudah memiliki akun. Sistem akan menampilkan pesan validasi apabila terdapat kolom yang tidak diisi atau data yang dimasukkan tidak sesuai format yang ditentukan.

e. Halaman Login

Halaman *login* menampilkan formulir masuk yang terdiri dari kolom *email* dan *password*, tombol *Login*, tautan lupa *password*, serta tautan menuju halaman registrasi bagi pengguna baru. Setelah pengguna berhasil *login*, sistem akan menyimpan JWT di penyimpanan lokal aplikasi dan mengarahkan pengguna ke halaman pemilihan peran.

f. Halaman Pemilihan Peran (Role Selection)

Halaman pemilihan peran merupakan halaman yang ditampilkan setelah pengguna berhasil melakukan *login*. Halaman ini menampilkan dua pilihan peran dalam format kartu besar yang tersusun secara vertikal, yaitu kartu *Job Seeker* (Pencari Kerja) dan kartu *Employer* (Pemberi Kerja). Setiap kartu menampilkan ikon, nama peran, dan deskripsi singkat mengenai fitur yang tersedia pada peran tersebut. Pengguna memilih salah satu peran dengan menekan kartu yang sesuai, kemudian sistem akan menyimpan peran aktif (*active_role*) dan mengarahkan pengguna ke halaman beranda yang sesuai dengan peran yang dipilih.

g. Halaman Beranda Job Seeker

Halaman beranda *Job Seeker* menampilkan antarmuka yang dirancang khusus untuk pencari kerja. Di bagian atas halaman terdapat sapaan kepada pengguna beserta

foto profil. Di bawahnya terdapat *search bar* dan tombol filter untuk pencarian pekerjaan. Bagian utama halaman menampilkan daftar lowongan pekerjaan terbaru dalam format kartu yang dapat di-*scroll* secara vertikal. Setiap kartu menampilkan informasi ringkas pekerjaan beserta tombol *Lihat Detail* untuk mengakses pratinjau lengkap pekerjaan. Di bagian bawah layar terdapat navigasi bawah (*bottom navigation bar*) yang terdiri dari menu Beranda, Cari Pekerjaan, Lamaran Saya, Pesan, dan Profil.

h. Halaman Detail Pekerjaan (Job Seeker)

Halaman detail pekerjaan menampilkan informasi lengkap mengenai lowongan pekerjaan yang dipilih oleh *Job Seeker*. Informasi yang ditampilkan meliputi judul pekerjaan, nama dan deskripsi perusahaan, jenis pekerjaan (*freelance* atau *job sharing*), kategori, lokasi, jenis kerja, nominal gaji, batas waktu pendaftaran, dan deskripsi lengkap pekerjaan beserta kualifikasi yang dibutuhkan. Di bagian bawah halaman terdapat tombol *Lamar Sekarang* untuk mengajukan lamaran pekerjaan.

i. Halaman Pengajuan Lamaran

Halaman pengajuan lamaran menampilkan formulir yang terdiri dari kolom surat lamaran (*cover letter*) dan kolom tautan portofolio (*portfolio URL*). Di bagian bawah formulir terdapat tombol *Kirim Lamaran* untuk memproses pengajuan lamaran. Setelah lamaran berhasil dikirim, sistem menampilkan notifikasi konfirmasi dan mengarahkan pengguna kembali ke halaman daftar pekerjaan.

j. Halaman Lamaran Saya

Halaman lamaran saya menampilkan riwayat seluruh lamaran yang telah diajukan oleh *Job Seeker* dalam format daftar. Setiap item daftar menampilkan judul pekerjaan, nama perusahaan, tanggal pengajuan lamaran, dan status lamaran (*pending*, *accepted*, atau

rejected) yang ditampilkan dalam bentuk label berwarna untuk memudahkan pembacaan status.

k. Halaman Beranda Employer

Halaman beranda *Employer* menampilkan antarmuka yang dirancang khusus untuk pemberi kerja. Di bagian atas halaman terdapat sapaan kepada pengguna beserta foto profil perusahaan. Bagian utama halaman menampilkan ringkasan statistik yang terdiri dari jumlah lowongan aktif, jumlah total lamaran yang masuk, dan jumlah lamaran yang menunggu tindakan. Di bawah statistik terdapat daftar lowongan pekerjaan yang telah diposting oleh *Employer* beserta status masing-masing lowongan. Di bagian bawah layar terdapat *bottom navigation bar* yang terdiri dari menu Beranda, Kelola Lowongan, Lamaran Masuk, Pesan, dan Profil. Terdapat pula tombol *tambah (floating action button)* untuk memposting lowongan pekerjaan baru.

l. Halaman Posting Lowongan

Halaman posting lowongan menampilkan formulir pembuatan lowongan pekerjaan baru yang terdiri dari kolom judul pekerjaan, deskripsi pekerjaan, jenis pekerjaan (*freelance* atau *job sharing*), kategori, nominal gaji, jenis bayaran, lokasi, jenis kerja (*remote*, *onsite*, atau *hybrid*), batas waktu pendaftaran, serta kualifikasi yang dibutuhkan. Di bagian bawah formulir terdapat tombol *Posting Lowongan* untuk menyimpan dan menerbitkan lowongan ke dalam sistem.

m. Halaman Kelola Lamaran (Employer)

Halaman kelola lamaran menampilkan daftar seluruh lamaran yang masuk untuk setiap lowongan yang dimiliki oleh *Employer*. Setiap item daftar menampilkan nama pelamar, foto profil, tanggal pengajuan lamaran, dan status lamaran. *Employer* dapat

menekan setiap item untuk melihat detail lamaran, membaca surat lamaran dan portofolio pelamar, serta memperbarui status lamaran menjadi *accepted* atau *rejected*.

n. Halaman Pesan

Halaman pesan menampilkan daftar percakapan antara pengguna dengan mitra kerjanya, baik antara *Job Seeker* dengan *Employer* maupun sebaliknya. Setiap item percakapan menampilkan foto profil, nama pengguna, pratinjau pesan terakhir, dan waktu pengiriman. Pengguna dapat menekan salah satu item percakapan untuk membuka halaman *chat* yang menampilkan riwayat pesan secara lengkap beserta kolom *input* pesan di bagian bawah layar.

o. Halaman Profil

Halaman profil menampilkan informasi lengkap akun pengguna yang terdiri dari foto profil, nama lengkap, *email*, nomor telepon, dan alamat. Untuk *Job Seeker*, halaman profil juga menampilkan daftar keahlian (*skills*) dan tautan portofolio. Untuk *Employer*, halaman profil menampilkan nama perusahaan dan deskripsi perusahaan. Di halaman ini terdapat tombol *Edit Profil* untuk memperbarui data diri, tombol *Ganti Peran* untuk kembali ke halaman pemilihan peran, serta tombol *Keluar (logout)* untuk mengakhiri sesi pengguna.

2. Perancangan Antarmuka Website Admin

Antarmuka *website* admin dirancang menggunakan Laravel dengan memanfaatkan *framework* CSS Bootstrap untuk menghasilkan tampilan yang responsif dan konsisten. *Website* admin hanya dapat diakses oleh *Administrator* melalui halaman *login* khusus yang terpisah dari aplikasi Android. Berikut adalah rancangan halaman-halaman utama pada *website* admin.

a. Halaman Login Admin

Halaman *login* admin menampilkan formulir masuk yang terdiri dari kolom *email* dan *password* serta tombol *Login*. Halaman ini dirancang sederhana dan hanya dapat diakses oleh *Administrator* yang telah terdaftar dalam sistem. Apabila kredensial tidak valid, sistem akan menampilkan pesan kesalahan.

b. Halaman Dashboard

Halaman *dashboard* merupakan halaman utama yang ditampilkan setelah *Administrator* berhasil *login*. Halaman ini menampilkan ringkasan statistik sistem dalam bentuk kartu statistik yang terdiri dari total pengguna terdaftar, total lowongan pekerjaan aktif, total lamaran yang masuk, dan total pesan yang terkirim. Di bawah kartu statistik terdapat grafik pertumbuhan pengguna dan lowongan pekerjaan berdasarkan periode waktu, serta tabel aktivitas terbaru sistem yang menampilkan pendaftaran pengguna baru dan posting lowongan terbaru.

c. Halaman Manajemen Pengguna

Halaman manajemen pengguna menampilkan daftar seluruh pengguna yang terdaftar dalam sistem dalam format tabel. Kolom yang ditampilkan meliputi nomor urut, nama pengguna, *email*, peran (*role*), status akun, dan tanggal registrasi. *Administrator* dapat melakukan pencarian pengguna berdasarkan nama atau *email*, melihat detail profil pengguna, mengaktifkan atau menonaktifkan akun pengguna, serta menghapus akun pengguna yang melanggar ketentuan sistem.

d. Halaman Manajemen Pekerjaan

Halaman manajemen pekerjaan menampilkan daftar seluruh lowongan pekerjaan yang tersedia dalam sistem dalam format tabel. Kolom yang ditampilkan meliputi nomor urut, judul pekerjaan, nama perusahaan, jenis pekerjaan, kategori, status lowongan, dan

tanggal posting. *Administrator* dapat melakukan pencarian dan filter data pekerjaan, melihat detail lowongan, mengubah status lowongan, serta menghapus lowongan yang tidak sesuai dengan ketentuan sistem.

e. Halaman Manajemen Lamaran

Halaman manajemen lamaran menampilkan daftar seluruh lamaran pekerjaan yang ada dalam sistem dalam format tabel. Kolom yang ditampilkan meliputi nomor urut, nama pelamar, judul pekerjaan yang dilamar, nama perusahaan, status lamaran, dan tanggal pengajuan. *Administrator* dapat melihat detail lamaran dan memantau seluruh aktivitas pelamaran yang terjadi di dalam sistem.

f. Halaman Manajemen Pesan

Halaman manajemen pesan menampilkan daftar seluruh percakapan yang terjadi antar pengguna dalam sistem. *Administrator* dapat memantau aktivitas komunikasi antar pengguna untuk memastikan tidak terjadi penyalahgunaan fitur pesan di dalam platform.

g. Halaman Pengaturan Sistem

Halaman pengaturan sistem menampilkan konfigurasi umum platform, seperti nama aplikasi, deskripsi platform, pengaturan *email* notifikasi, dan pengaturan keamanan sistem. *Administrator* dapat memperbarui konfigurasi sistem sesuai kebutuhan melalui formulir yang tersedia pada halaman ini.

F. Perancangan API (Application Programming Interface)

Perancangan REST API dalam penelitian ini menggunakan *framework* Laravel sebagai *backend* dengan mengikuti prinsip *RESTful API* yang menggunakan metode HTTP (*GET*, *POST*, *PUT*, *DELETE*) secara konsisten. Seluruh *endpoint* API menggunakan format pertukaran data JSON dan dibagi menjadi dua kelompok, yaitu

public endpoint yang dapat diakses tanpa autentikasi untuk mendukung fitur *guest mode*, dan *protected endpoint* yang wajib menyertakan JWT sebagai *header* autentikasi. Berikut adalah rancangan *endpoint* API yang tersedia dalam sistem.

1. Autentikasi

Kelompok *endpoint* autentikasi bersifat publik dan tidak memerlukan JWT. *Endpoint* ini menangani proses registrasi, *login*, pemilihan peran, dan *logout* pengguna.

Tabel 8.tabe Autentikasi Api

No	Method	Endpoint	Fungsi	Akses
1	POST	/api/register	Registrasi akun pengguna baru	Publik
2	POST	/api/login	Proses <i>login</i> dan mendapatkan JWT	Publik
3	POST	/api/select-role	Memilih peran aktif (<i>job_seeker</i> / <i>employer</i>)	<i>Protected</i>
4	POST	/api/logout	Proses <i>logout</i> dan invalidasi token	<i>Protected</i>
5	GET	/api/me	Mendapatkan data pengguna yang sedang <i>login</i>	<i>Protected</i>

Contoh Request POST /api/login:

```
json
{
  "email": "user@example.com",
  "password": "password123"
}
```

Contoh Response Berhasil (200):

```
json
{
  "status": "success",
  "message": "Login berhasil",
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 1,
    "name": "Fajri Nur Akbar",
    "email": "user@example.com",
    "role": "job_seeker",
    "active_role": null
  }
}
```

Contoh Response Gagal (401):

```

json
{
  "status": "error",
  "message": "Email atau password tidak valid"
}

```

2. Pekerjaan (Jobs)

Kelompok *endpoint* pekerjaan terdiri dari *public endpoint* untuk mendukung fitur *guest mode* dan *protected endpoint* untuk pengelolaan lowongan oleh *Employer*.

Tabel 9. Tabel pekerjaan Api

No	Method	Endpoint	Fungsi	Akses
1	GET	/api/jobs	Mendapatkan daftar pekerjaan	Publik
2	GET	/api/jobs/{id}	Mendapatkan detail pekerjaan	<i>Protected</i>
3	GET	/api/jobs/search	Mencari pekerjaan berdasarkan filter	Publik
4	POST	/api/jobs	Memposting lowongan baru	<i>Protected (Employer)</i>
5	PUT	/api/jobs/{id}	Memperbarui data lowongan	<i>Protected (Employer)</i>
6	DELETE	/api/jobs/{id}	Menghapus lowongan	<i>Protected (Employer)</i>
7	GET	/api/jobs/employer	Mendapatkan daftar lowongan milik <i>Employer</i>	<i>Protected (Employer)</i>

Contoh Request GET /api/jobs (dengan parameter filter):

GET /api/jobs?type=freelance&category=design&location=Jakarta&page=1

Contoh Response Berhasil (200):

```

json
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "title": "UI/UX Designer Freelance",
      "employer": {
        "id": 2,
        "company_name": "PT Digital Kreatif"
      }
    }
  ],
}

```



```

    "type": "freelance",
    "category": "Design",
    "salary": 5000000,
    "salary_type": "per_project",
    "location": "Jakarta",
    "work_type": "remote",
    "deadline": "2025-08-01",
    "status": "open",
    "created_at": "2025-06-01"
  }
],
"meta": {
  "total": 50,
  "per_page": 10,
  "current_page": 1,
  "last_page": 5
}
}

```

3. Lamaran (Applications)

Kelompok *endpoint* lamaran seluruhnya bersifat *protected* dan hanya dapat diakses oleh pengguna yang telah *login*. *Job Seeker* dapat mengajukan dan memantau lamaran, sedangkan *Employer* dapat mengelola lamaran yang masuk.

Tabel 10.tabel lamaran Api

No	Method	Endpoint	Fungsi	Akses
1	POST	/api/applications	Mengajukan lamaran pekerjaan	<i>Protected (Job Seeker)</i>
2	GET	/api/applications/my	Mendapatkan daftar lamaran milik <i>Job Seeker</i>	<i>Protected (Job Seeker)</i>
3	GET	/api/applications/{id}	Mendapatkan detail lamaran	<i>Protected</i>
4	GET	/api/applications/job/{job_id}	Mendapatkan daftar lamaran per lowongan	<i>Protected (Employer)</i>
5	PUT	/api/applications/{id}/status	Memperbarui status lamaran	<i>Protected (Employer)</i>
6	DELETE	/api/applications/{id}	Membatalkan lamaran	<i>Protected (Job Seeker)</i>

Contoh Request POST /api/applications:

```

json
{
  "job_id": 1,
  "cover_letter": "Saya tertarik untuk melamar posisi ini...",

```

```
"portfolio_url": "https://portofolio.example.com"
}
```

Contoh Request PUT /api/applications/{id}/status:

```
json
{
  "status": "accepted",
  "notes": "Kandidat memenuhi kualifikasi yang dibutuhkan"
}
```

4. Pesan (Messages)

Kelompok *endpoint* pesan seluruhnya bersifat *protected* dan hanya dapat diakses oleh *Job Seeker* dan *Employer* yang telah *login* untuk melakukan komunikasi satu sama lain.

Tabel 11.tabel pesan Api

No	Method	Endpoint	Fungsi	Akses
1	POST	/api/applications	Mengajukan lamaran pekerjaan	<i>Protected (Job Seeker)</i>
2	GET	/api/applications/my	Mendapatkan daftar lamaran milik <i>Job Seeker</i>	<i>Protected (Job Seeker)</i>
3	GET	/api/applications/{id}	Mendapatkan detail lamaran	<i>Protected</i>
4	GET	/api/applications/job/{job_id}	Mendapatkan daftar lamaran per lowongan	<i>Protected (Employer)</i>
5	PUT	/api/applications/{id}/status	Memperbarui status lamaran	<i>Protected (Employer)</i>

Contoh Request POST /api/messages:

```
json
{
  "receiver_id": 3,
  "content": "Selamat siang, apakah posisi ini masih tersedia?"
}
```

5. Profil (Profile)

Kelompok *endpoint* profil seluruhnya bersifat *protected* dan digunakan untuk mengelola data profil pengguna yang sedang *login*, baik sebagai *Job Seeker* maupun *Employer*.

Tabel 12.tabel Profil api

No	Method	Endpoint	Fungsi	Akses
1	GET	/api/profile	Mendapatkan data profil pengguna	<i>Protected</i>
2	PUT	/api/profile	Memperbarui data profil pengguna	<i>Protected</i>
3	POST	/api/profile/photo	Mengunggah foto profil	<i>Protected</i>
4	PUT	/api/profile/password	Memperbarui password pengguna	<i>Protected</i>

Contoh Request PUT /api/profile:

```

json
{
  "name": "Fajri Nur Akbar",
  "phone": "081234567890",
  "address": "Padang, Sumatera Barat",
  "bio": "Flutter Developer dengan pengalaman 2 tahun",
  "skills": "Flutter, Laravel, MySQL",
  "company_name": null,
  "company_desc": null
}

```

6. Admin (Panel Web Admin)

Kelompok *endpoint* admin hanya dapat diakses oleh pengguna dengan peran *Administrator* melalui panel *web admin* berbasis Laravel. Seluruh *endpoint* dalam kelompok ini bersifat *protected* dan memerlukan JWT dengan peran *admin*.

Tabel 13.tabel Admin Api

No	Method	Endpoint	Fungsi	Akses
1	GET	/api/admin/users	Mendapatkan daftar seluruh pengguna	<i>Admin</i>
2	GET	/api/admin/users/{id}	Mendapatkan detail pengguna	<i>Admin</i>
3	PUT	/api/admin/users/{id}/status	Mengaktifkan atau menonaktifkan akun	<i>Admin</i>
4	DELETE	/api/admin/users/{id}	Menghapus akun pengguna	<i>Admin</i>
5	GET	/api/admin/jobs	Mendapatkan daftar seluruh pekerjaan	<i>Admin</i>
6	DELETE	/api/admin/jobs/{id}	Menghapus lowongan pekerjaan	<i>Admin</i>
7	GET	/api/admin/applications	Mendapatkan daftar seluruh lamaran	<i>Admin</i>
8	GET	/api/admin/dashboard	Mendapatkan data statistik sistem	<i>Admin</i>

7. Enpoint pembayaran

Tabel 14. tabel pembayaran Api

No	Method	Endpoint	Fungsi
1	POST	/api/payments/create	Membuat transaksi pembayaran baru ke Midtrans
2	GET	/api/payments/{id}	Mengecek detail dan status pembayaran
3	GET	/api/payments/history	Mendapatkan riwayat transaksi pengguna
4	POST	/api/payments/notification	Menerima <i>webhook</i> notifikasi dari Midtrans
5	POST	/api/payments/{id}/cancel	Membatalkan transaksi yang masih <i>pending</i>
6	GET	/api/admin/payments	Mendapatkan seluruh data transaksi
7	GET	/api/admin/payments/revenue	Mendapatkan laporan pendapatan platform

Contoh Request POST /api/payments/create:

```
json
{
  "application_id": 5,
  "project_value": 1000000,
  "payment_type": "bank_transfer"
}
```

Contoh Response Berhasil (201):

```
json
{
  "status": "success",
  "message": "Transaksi berhasil dibuat",
  "data": {
    "order_id": "ORDER-20250601-0001",
    "project_value": 1000000,
    "service_fee_employer": 25000,
    "total_paid": 1025000,
    "job_seeker_received": 980000,
    "platform_revenue": 45000,
    "midtrans_token": "66e4fa55-addf-4b5e-8891...",
    "midtrans_redirect_url": "https://app.sandbox.midtrans.com/snap/...",
    "expired_at": "2025-06-01 14:00:00"
  }
}
```

8. Penanganan Respons API

Seluruh respons API dalam sistem ini menggunakan format JSON yang konsisten dengan struktur sebagai berikut.

Respons Berhasil (2xx):

```
json
{
  "status": "success",
  "message": "Keterangan pesan",
  "data": {}
}
```

Respons Gagal (4xx / 5xx):

```
json
{
  "status": "error",
  "message": "Keterangan pesan kesalahan",
  "errors": {}
}
```

Kode status HTTP yang digunakan dalam sistem ini adalah sebagai berikut. Kode **200** (OK) digunakan untuk permintaan yang berhasil diproses. Kode **201** (*Created*) digunakan untuk data yang berhasil dibuat. Kode **400** (*Bad Request*) digunakan apabila terdapat kesalahan pada data yang dikirimkan. Kode **401** (*Unauthorized*) digunakan apabila pengguna belum melakukan autentikasi atau token tidak valid. Kode **403** (*Forbidden*) digunakan apabila pengguna tidak memiliki hak akses terhadap *endpoint* yang dituju. Kode **404** (*Not Found*) digunakan apabila data yang diminta tidak ditemukan. Kode **500** (*Internal Server Error*) digunakan apabila terjadi kesalahan pada sisi *server*.

G. Teknologi yang Digunakan

Pengembangan platform layanan *freelance* dan *job sharing* dalam penelitian ini memanfaatkan beberapa teknologi yang saling terintegrasi, mulai dari sisi *frontend*

aplikasi Android, *backend* dan *web admin*, basis data, hingga alat pendukung pengembangan sistem. Pemilihan setiap teknologi didasarkan pada kesesuaian dengan kebutuhan sistem, kemudahan pengembangan, serta keandalan teknologi tersebut dalam ekosistem pengembangan perangkat lunak modern.

1. Flutter

Flutter merupakan *framework* pengembangan aplikasi *mobile* lintas platform (*cross-platform*) yang dikembangkan oleh Google menggunakan bahasa pemrograman Dart. Dalam penelitian ini, Flutter digunakan sebagai teknologi utama dalam pembangunan aplikasi Android yang berperan sebagai *client* sistem. Flutter dipilih karena mampu menghasilkan antarmuka aplikasi yang responsif dan berkinerja tinggi dengan memanfaatkan *widget-based UI* yang disediakan oleh *framework Material Design*. Selain itu, Flutter mendukung pengelolaan *state* aplikasi secara efisien serta menyediakan berbagai paket pendukung yang memudahkan integrasi dengan REST API, pengelolaan JWT, penyimpanan *session*, dan fitur-fitur lainnya yang dibutuhkan oleh sistem. Beberapa paket Flutter utama yang digunakan dalam penelitian ini adalah sebagai berikut. Paket *http* atau *dio* digunakan untuk melakukan *HTTP request* ke *endpoint* REST API *backend*. Paket *shared_preferences* atau *flutter_secure_storage* digunakan untuk menyimpan JWT dan data *session* di penyimpanan lokal perangkat. Paket *provider* atau *riverpod* digunakan untuk manajemen *state* aplikasi secara menyeluruh. Paket *flutter_local_notifications* digunakan untuk menampilkan notifikasi lokal kepada pengguna.

2. Laravel

Laravel merupakan *framework* pengembangan aplikasi berbasis PHP yang menggunakan pola arsitektur *Model-View-Controller* (MVC). Dalam penelitian ini,

Laravel digunakan sebagai teknologi utama dalam pembangunan *backend* sistem sekaligus panel *web admin* bagi *Administrator*. Laravel dipilih karena menyediakan ekosistem pengembangan yang lengkap, termasuk sistem *routing* yang terstruktur, *Eloquent ORM* untuk pengelolaan basis data, sistem autentikasi bawaan, serta kemudahan dalam pembuatan REST API melalui fitur *API Resource* dan *middleware*. Beberapa paket Laravel utama yang digunakan dalam penelitian ini adalah sebagai berikut.

Paket *tymon/jwt-auth* digunakan untuk mengimplementasikan mekanisme autentikasi berbasis *JSON Web Token (JWT)* pada seluruh *protected endpoint* API. Paket *laravel/sanctum* digunakan sebagai alternatif pengelolaan token autentikasi pada panel *web admin*. Paket *intervention/image* digunakan untuk memproses dan mengoptimalkan unggahan foto profil pengguna. Paket *spatie/laravel-permission* digunakan untuk mengelola peran dan hak akses pengguna (*role-based access control*) antara *Job Seeker*, *Employer*, dan *Administrator*.

3. MySQL

MySQL merupakan sistem manajemen basis data relasional (*Relational Database Management System/RDBMS*) yang bersifat *open source* dan banyak digunakan dalam pengembangan aplikasi berbasis web maupun *mobile*. Dalam penelitian ini, MySQL digunakan sebagai sistem penyimpanan data terpusat yang mengelola seluruh tabel basis data sistem, meliputi tabel *users*, *jobs*, *applications*, *messages*, *personal_access_tokens*, dan *sessions*. MySQL dipilih karena memiliki performa yang andal dalam menangani transaksi data dalam jumlah besar, terintegrasi dengan baik bersama *framework* Laravel melalui *Eloquent ORM*, serta didukung oleh dokumentasi dan komunitas pengembang yang sangat luas.

4. JSON Web Token (JWT)

JSON Web Token (JWT) merupakan standar terbuka (*open standard*) berbasis RFC 7519 yang digunakan untuk mentransmisikan informasi secara aman antara dua pihak dalam format JSON yang terenkripsi. Dalam penelitian ini, JWT digunakan sebagai mekanisme autentikasi *stateless* yang melindungi seluruh *protected endpoint* API dari akses tidak sah. JWT diimplementasikan menggunakan paket *tymon/jwt-auth* pada Laravel, di mana token dihasilkan oleh *backend* setelah pengguna berhasil *login* dan selanjutnya disimpan di penyimpanan lokal aplikasi Android untuk digunakan pada setiap permintaan berikutnya. JWT dipilih karena bersifat *stateless* sehingga tidak memerlukan penyimpanan sesi di sisi *server*, mendukung komunikasi lintas platform antara aplikasi Android dan *backend* Laravel, serta memiliki mekanisme kedaluwarsa (*expiry*) yang dapat dikonfigurasi untuk meningkatkan keamanan sistem.

5. Visual Studio Code

Visual Studio Code (VS Code) merupakan *code editor* ringan yang dikembangkan oleh Microsoft dan banyak digunakan dalam pengembangan perangkat lunak berbagai platform. Dalam penelitian ini, VS Code digunakan sebagai *code editor* utama dalam penulisan kode program, baik untuk pengembangan aplikasi Android menggunakan Flutter maupun *backend* menggunakan Laravel. VS Code dipilih karena menyediakan ekstensi yang lengkap untuk Flutter, Dart, PHP, dan Laravel, serta dilengkapi dengan fitur *IntelliSense*, *debugging*, dan integrasi dengan sistem kontrol versi Git.

6. Postman

Postman merupakan platform pengembangan API yang menyediakan antarmuka grafis untuk melakukan pengujian dan dokumentasi *endpoint* API secara terstruktur. Dalam penelitian ini, Postman digunakan sebagai alat utama untuk menguji seluruh

endpoint REST API yang telah dibangun menggunakan Laravel sebelum diintegrasikan dengan aplikasi Android. Postman dipilih karena memudahkan proses pengujian *request* dan *response* API secara menyeluruh, mendukung pengujian autentikasi berbasis JWT, serta dapat digunakan untuk mendokumentasikan seluruh *endpoint* API sistem secara otomatis.

7. XAMPP

XAMPP merupakan paket perangkat lunak *open source* yang menyediakan lingkungan pengembangan lokal (*local development environment*) yang terdiri dari Apache *web server*, MySQL, dan PHP dalam satu instalasi. Dalam penelitian ini, XAMPP digunakan sebagai server lokal untuk menjalankan *backend* Laravel dan basis data MySQL selama proses pengembangan dan pengujian sistem berlangsung sebelum sistem di-*deploy* ke server produksi.

8. Android Studio

Android Studio merupakan *Integrated Development Environment* (IDE) resmi yang dikembangkan oleh Google khusus untuk pengembangan aplikasi Android. Android Studio dibangun di atas platform IntelliJ IDEA dan menyediakan berbagai fitur pengembangan yang lengkap dan terintegrasi, meliputi editor kode, emulator Android, *debugger*, dan alat pengujian aplikasi. Dalam penelitian ini, Android Studio digunakan sebagai lingkungan pengembangan utama dalam proses pembangunan aplikasi Android berbasis Flutter, karena Android Studio menyediakan dukungan penuh terhadap *framework* Flutter melalui plugin resmi yang disediakan oleh tim Flutter dari Google.

Pemilihan Android Studio didasarkan pada beberapa pertimbangan. Pertama, Android Studio menyediakan emulator Android bawaan (*Android Virtual Device*/AVD) yang memungkinkan pengembang menjalankan dan menguji aplikasi secara langsung tanpa

memerlukan perangkat fisik. Kedua, Android Studio terintegrasi penuh dengan Flutter SDK dan Dart SDK sehingga proses pengembangan, *debugging*, dan pengujian aplikasi dapat dilakukan dalam satu lingkungan pengembangan yang terpadu. Ketiga, Android Studio menyediakan fitur *Hot Reload* dan *Hot Restart* pada Flutter yang memungkinkan perubahan kode dapat langsung terlihat hasilnya secara real-time tanpa harus menjalankan ulang aplikasi dari awal, sehingga mempercepat proses pengembangan secara signifikan.

9. Git dan GitHub

Git merupakan sistem kontrol versi (*version control system*) terdistribusi yang digunakan untuk melacak perubahan kode program selama proses pengembangan. GitHub merupakan platform *hosting* berbasis *cloud* yang menyediakan layanan penyimpanan *repository* Git secara *online*. Dalam penelitian ini, Git dan GitHub digunakan untuk mengelola versi kode program, memudahkan pemulihan kode apabila terjadi kesalahan, serta menjadi media penyimpanan cadangan (*backup*) seluruh kode program yang telah dikembangkan.

10. Midtrans Payment Gateway

Midtrans merupakan *payment gateway* terkemuka di Indonesia yang menyediakan layanan pemrosesan pembayaran digital secara aman dan terintegrasi. Midtrans dikembangkan oleh PT Midtrans dan telah diakuisisi oleh GoTo Financial, serta telah mendapatkan lisensi resmi dari Bank Indonesia sebagai penyedia layanan sistem pembayaran. Dalam penelitian ini, Midtrans digunakan sebagai *payment gateway* utama yang menghubungkan sistem dengan berbagai metode pembayaran yang tersedia di Indonesia.

Integrasi Midtrans dalam sistem ini menggunakan pendekatan Snap API, yaitu metode integrasi yang memungkinkan sistem menampilkan halaman pembayaran Midtrans secara langsung di dalam aplikasi tanpa harus membangun antarmuka pembayaran sendiri. Alur kerja integrasi Midtrans adalah sebagai berikut. Pertama, *backend* Laravel membuat transaksi baru dengan mengirimkan data pembayaran ke Midtrans API menggunakan *Server Key*. Kedua, Midtrans mengembalikan *Snap Token* dan *Redirect URL* yang digunakan oleh aplikasi Flutter untuk menampilkan halaman pembayaran. Ketiga, pengguna menyelesaikan pembayaran melalui metode yang dipilih. Keempat, Midtrans mengirimkan notifikasi *webhook* ke *endpoint* `/api/payments/notification` pada *backend* Laravel untuk memperbarui status transaksi secara otomatis.

Metode pembayaran yang didukung dalam sistem ini melalui Midtrans meliputi transfer bank melalui *virtual account* (BCA, BNI, BRI, Mandiri, Permata), dompet digital (*e-wallet*) seperti GoPay, OVO, Dana, dan ShopeePay, QRIS yang dapat digunakan di seluruh aplikasi pembayaran yang mendukung standar QRIS, serta kartu kredit Visa dan Mastercard.

Paket Laravel yang digunakan untuk integrasi Midtrans adalah `midtrans/midtrans-php` yang disediakan secara resmi oleh Midtrans, serta paket `midtrans-flutter` untuk integrasi pada sisi aplikasi Android berbasis Flutter.

H. Rancangan Pengujian Sistem

Pengujian sistem merupakan tahapan penting dalam proses pengembangan perangkat lunak yang bertujuan untuk memastikan bahwa seluruh fitur dan fungsi sistem berjalan sesuai dengan kebutuhan yang telah ditetapkan. Menurut Wijaya (2021), pengujian

perangkat lunak adalah proses yang dirancang untuk memastikan bahwa sistem berfungsi sesuai dengan tujuan yang telah ditentukan dan tidak melakukan hal-hal yang tidak diinginkan. Dalam penelitian ini, pengujian sistem dilakukan menggunakan dua metode, yaitu *Black Box Testing* dan *User Acceptance Testing (UAT)*.

1. Black Box Testing

Black Box Testing merupakan metode pengujian yang berfokus pada fungsionalitas sistem tanpa memperhatikan struktur kode program di dalamnya. Menurut (Wijaya & Astuti, 2021), *Black Box Testing* bertujuan untuk menemukan fungsi yang tidak benar, kesalahan antarmuka, kesalahan pada struktur data, kesalahan performansi, serta kesalahan inisialisasi dan terminasi sistem. Pengujian ini dilakukan dengan cara memberikan masukan (*input*) tertentu kepada sistem dan membandingkan hasil keluaran (*output*) yang diperoleh dengan hasil yang diharapkan (*expected result*).

Berikut adalah rancangan skenario *Black Box Testing* yang akan dilakukan pada sistem:

a. Pengujian Fitur Guest Mode

Tabel 15. tabel Pengujian Fitur Guest Mode

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Membuka aplikasi tanpa <i>login</i>	Membuka aplikasi	Halaman daftar pekerjaan ditampilkan	
2	Mencari pekerjaan sebagai <i>Guest</i>	Kata kunci pencarian	Daftar pekerjaan sesuai kata kunci ditampilkan	
3	Filter pekerjaan berdasarkan kategori	Memilih kategori filter	Daftar pekerjaan sesuai filter ditampilkan	
4	Menekan kartu pekerjaan sebagai <i>Guest</i>	Menekan kartu pekerjaan	Halaman peringatan <i>login</i> ditampilkan	

5	Menekan tombol <i>Login</i> pada halaman peringatan	Menekan tombol <i>Login</i>	Diarahkan ke halaman <i>login</i>	
6	Menekan tombol <i>Daftar</i> pada halaman peringatan	Menekan tombol <i>Daftar</i>	Diarahkan ke halaman registrasi	

b. Pengujian Fitur Registrasi dan Login

Tabel 16.tabel Pengujian Fitur Registrasi dan Login

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Registrasi dengan data valid	Nama, email, password lengkap	Akun berhasil dibuat, diarahkan ke halaman <i>login</i>	
2	Registrasi dengan email yang sudah terdaftar	Email yang sudah digunakan	Pesan kesalahan "Email sudah terdaftar"	
3	Registrasi dengan kolom kosong	Kolom tidak diisi	Pesan validasi ditampilkan	
4	Registrasi dengan <i>password</i> tidak sesuai	<i>Password</i> dan konfirmasi berbeda	Pesan validasi ditampilkan	
5	<i>Login</i> dengan data valid	Email dan <i>password</i> benar	JWT diterima, diarahkan ke halaman pemilihan peran	
6	<i>Login</i> dengan <i>password</i> salah	<i>Password</i> tidak sesuai	Pesan kesalahan "Email atau <i>password</i> tidak valid"	
7	<i>Login</i> dengan email tidak terdaftar	Email tidak ada di sistem	Pesan kesalahan ditampilkan	
8	<i>Login</i> dengan kolom kosong	Kolom tidak diisi	Pesan validasi ditampilkan	

c. Pengujian Fitur Pemilihan Peran (Role Selection)

Tabel 17.tabel fitur pemilihan peran

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Memilih peran <i>Job Seeker</i> setelah <i>login</i>	Menekan kartu <i>Job Seeker</i>	Diarahkan ke halaman beranda <i>Job Seeker</i>	
2	Memilih peran <i>Employer</i> setelah <i>login</i>	Menekan kartu <i>Employer</i>	Diarahkan ke halaman beranda <i>Employer</i>	

3	Beralih peran dari <i>Job Seeker</i> ke <i>Employer</i>	Menekan tombol <i>Ganti Peran</i>	Diarahkan kembali ke halaman pemilihan peran	
4	Beralih peran dari <i>Employer</i> ke <i>Job Seeker</i>	Menekan tombol <i>Ganti Peran</i>	Diarahkan kembali ke halaman pemilihan peran	

d. Pengujian Fitur Job Seeker

Tabel 18.tabel Pengujian Fitur Job Seeker

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Melihat daftar pekerjaan sebagai <i>Job Seeker</i>	Membuka halaman beranda	Daftar pekerjaan terbaru ditampilkan	
2	Melihat detail pekerjaan	Menekan kartu pekerjaan	Halaman detail pekerjaan ditampilkan	
3	Mengajukan lamaran dengan data valid	Mengisi <i>cover letter</i> dan <i>portfolio URL</i>	Lamaran berhasil dikirim, notifikasi konfirmasi ditampilkan	
4	Mengajukan lamaran dengan kolom kosong	Kolom tidak diisi	Pesan validasi ditampilkan	
5	Mengajukan lamaran yang sudah pernah dilamar	Melamar pekerjaan yang sama	Pesan "Kamu sudah melamar pekerjaan ini" ditampilkan	
6	Melihat riwayat lamaran	Membuka halaman <i>Lamaran Saya</i>	Daftar riwayat lamaran beserta status ditampilkan	
7	Membatalkan lamaran berstatus <i>pending</i>	Menekan tombol batalkan lamaran	Lamaran berhasil dibatalkan	
8	Memperbarui data profil <i>Job Seeker</i>	Mengisi formulir profil	Data profil berhasil diperbarui	

e. Pengujian Fitur Employer

Tabel 19.tabel fitur employer

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Memposting lowongan dengan data valid	Mengisi formulir lowongan lengkap	Lowongan berhasil diposting dan tampil di daftar pekerjaan	
2	Memposting lowongan dengan kolom kosong	Kolom wajib tidak diisi	Pesan validasi ditampilkan	
3	Memperbarui data lowongan	Mengubah data lowongan	Data lowongan berhasil diperbarui	

4	Menutup lowongan (<i>close job</i>)	Mengubah status lowongan menjadi <i>closed</i>	Lowongan tidak lagi tampil di daftar pencarian	
5	Menghapus lowongan	Menekan tombol hapus lowongan	Lowongan berhasil dihapus dari sistem	
6	Melihat daftar lamaran masuk	Membuka halaman <i>Kelola Lamaran</i>	Daftar lamaran yang masuk ditampilkan	
7	Menerima lamaran (<i>accept</i>)	Menekan tombol <i>Accept</i>	Status lamaran berubah menjadi <i>accepted</i>	
8	Menolak lamaran (<i>reject</i>)	Menekan tombol <i>Reject</i>	Status lamaran berubah menjadi <i>rejected</i>	
9	Memperbarui data profil <i>Employer</i>	Mengisi formulir profil perusahaan	Data profil perusahaan berhasil diperbarui	

f. Pengujian Fitur Pesan

Tabel 20.tabel Pengujian Fitur Pesan

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Mengirim pesan baru	Mengisi kolom pesan dan menekan kirim	Pesan berhasil terkirim dan tampil di percakapan	
2	Mengirim pesan kosong	Kolom pesan kosong	Pesan validasi ditampilkan	
3	Melihat riwayat percakapan	Membuka halaman Pesan	Daftar percakapan ditampilkan	
4	Menandai pesan sebagai sudah dibaca	Membuka percakapan	Status pesan berubah menjadi sudah dibaca	

g. Pengujian Fitur Web Admin

Tabel 21.tabel Pengujian Fitur Web Admin

No	Skenario Pengujian	Input	Expected Result	Hasil
1	<i>Login</i> admin dengan data valid	Email dan <i>password</i> admin benar	Diarahkan ke halaman <i>dashboard</i> admin	
2	<i>Login</i> admin dengan data tidak valid	Email atau <i>password</i> salah	Pesan kesalahan ditampilkan	
3	Melihat statistik sistem di <i>dashboard</i>	Membuka halaman <i>dashboard</i>	Data statistik sistem ditampilkan dengan benar	

4	Menonaktifkan akun pengguna	Menekan tombol nonaktifkan akun	Status akun pengguna berubah menjadi nonaktif	
5	Menghapus akun pengguna	Menekan tombol hapus akun	Akun pengguna berhasil dihapus dari sistem	
6	Menghapus lowongan pekerjaan	Menekan tombol hapus lowongan	Lowongan berhasil dihapus dari sistem	
7	Melihat daftar seluruh lamaran	Membuka halaman manajemen lamaran	Seluruh data lamaran ditampilkan	

h. Pengujian Autentikasi API (JWT)

Tabel 22.tabel aunthentikasi

No	Skenario Pengujian	Input	Expected Result	Hasil
1	Mengakses <i>public endpoint</i> tanpa JWT	<i>Request</i> tanpa JWT	Data berhasil dikembalikan (200 OK)	
2	Mengakses <i>protected endpoint</i> dengan JWT valid	<i>Request</i> dengan JWT valid	Data berhasil dikembalikan (200 OK)	
3	Mengakses <i>protected endpoint</i> tanpa JWT	<i>Request</i> tanpa JWT	Respons 401 <i>Unauthorized</i>	
4	Mengakses <i>protected endpoint</i> dengan JWT kadaluarsa	<i>Request</i> dengan JWT kadaluarsa	Respons 401 <i>Unauthorized</i>	
5	Mengakses <i>endpoint</i> admin dengan peran bukan admin	<i>Request</i> dengan JWT <i>non-admin</i>	Respons 403 <i>Forbidden</i>	

2. User Acceptance Testing (UAT)

User Acceptance Testing (UAT) merupakan metode pengujian yang melibatkan pengguna akhir secara langsung untuk mengevaluasi kesesuaian dan kemudahan penggunaan sistem. Menurut (Aliyah et al., 2025), UAT merupakan fase pengujian akhir yang melibatkan pengguna akhir untuk memvalidasi bahwa perangkat lunak memenuhi kebutuhan dan berfungsi sesuai harapan dalam skenario penggunaan nyata. Pengujian UAT dalam penelitian ini dilakukan dengan memberikan skenario penggunaan kepada

responden, kemudian hasilnya dianalisis menggunakan skala *Likert* dengan lima tingkatan penilaian sebagai berikut.

Tabel 23.skala likert

Skala	Keterangan
5	Sangat Setuju (SS)
4	Setuju (S)
3	Netral (N)
2	Tidak Setuju (TS)
1	Sangat Tidak Setuju (STS)

Responden UAT dalam penelitian ini terdiri dari dua kelompok, yaitu kelompok *Job Seeker* yang terdiri dari mahasiswa atau pencari kerja, dan kelompok *Employer* yang terdiri dari pelaku usaha atau pemilik bisnis. Berikut adalah kisi-kisi instrumen UAT yang akan digunakan.

a. Kisi-kisi UAT untuk Job Seeker

Tabel 24. tabel Kisi-kisi UAT untuk Job Seeker

No	Aspek	Pernyataan
1	Kemudahan penggunaan	Aplikasi mudah digunakan tanpa perlu petunjuk khusus
2	Kemudahan penggunaan	Tampilan aplikasi bersih, rapi, dan mudah dipahami
3	Fitur <i>Guest Mode</i>	Saya dapat melihat daftar pekerjaan tanpa harus <i>login</i> terlebih dahulu
4	Fitur <i>Guest Mode</i>	Peringatan untuk <i>login</i> saat mengakses detail pekerjaan sudah jelas
5	Fitur pemilihan peran	Halaman pemilihan peran mudah dipahami dan dioperasikan
6	Fitur pencarian	Fitur pencarian dan filter pekerjaan membantu saya menemukan pekerjaan yang sesuai
7	Fitur lamaran	Proses pengajuan lamaran mudah dan tidak membingungkan
8	Fitur lamaran	Status lamaran dapat dipantau dengan mudah
9	Fitur pesan	Fitur pesan memudahkan komunikasi dengan pemberi kerja
10	Kepuasan keseluruhan	Secara keseluruhan saya puas dengan fitur dan tampilan aplikasi ini

b. Kisi-kisi UAT untuk Employer

Tabel 25. tabel Kisi-kisi UAT untuk Employer

No	Aspek	Pernyataan
1	Kemudahan penggunaan	Aplikasi mudah digunakan tanpa perlu petunjuk khusus
2	Kemudahan penggunaan	Tampilan aplikasi bersih, rapi, dan mudah dipahami
3	Fitur pemilihan peran	Halaman pemilihan peran mudah dipahami dan dioperasikan
4	Fitur posting lowongan	Proses memposting lowongan pekerjaan mudah dilakukan
5	Fitur posting lowongan	Formulir posting lowongan sudah mencakup informasi yang diperlukan
6	Fitur kelola lamaran	Saya dapat dengan mudah melihat dan mengelola lamaran yang masuk
7	Fitur kelola lamaran	Proses menerima atau menolak lamaran mudah dilakukan
8	Fitur pesan	Fitur pesan memudahkan komunikasi dengan pencari kerja
9	Keamanan	Saya merasa data akun dan informasi perusahaan saya aman dalam sistem ini
10	Kepuasan keseluruhan	Secara keseluruhan saya puas dengan fitur dan tampilan aplikasi ini

Hasil pengujian UAT akan dihitung menggunakan rumus persentase sebagai berikut:

$$\text{Persentase (\%)} = (\text{Total Skor yang Diperoleh} / \text{Total Skor Maksimal}) \times 100\%$$

Hasil persentase kemudian diinterpretasikan menggunakan kategori kelayakan sebagai berikut:

Tabel 26. tabel patokan kelayakan hasil UAT

Persentase	Kategori
81% — 100%	Sangat Layak
61% — 80%	Layak
41% — 60%	Cukup Layak
21% — 40%	Tidak Layak
0% — 20%	Sangat Tidak Layak

Ringkasan Rancangan Pengujian

Tabel 27. tabl ringkasan hasil pengujian

No	Metode Pengujian	Objek Pengujian	Pelaksana
1	<i>Black Box Testing</i>	Seluruh fitur dan fungsi sistem	Peneliti
2	UAT — <i>Job Seeker</i>	Fitur pencarian, lamaran, pesan, dan profil	Responden <i>Job Seeker</i>
3	UAT — <i>Employer</i>	Fitur posting lowongan, kelola lamaran, pesan, dan profil	Responden <i>Employer</i>

DAFTAR PUSTAKA

- Abdulloh, R. (2016). *Easy & Simple-Web Programming*. Elex Media Komputindo.
- Alessandro, F., Steven, S., Sudianto, K. F., & Sudrajat, A. W. (2024). Analisis & Pengujian Black Box Pada Aplikasi Pencatatan Material Menggunakan Metode Boundary Value Analysis. *Device : Journal of Information System, Computer Science and Information Technology*, 5(1), 188–201. <https://doi.org/10.46576/device.v5i1.4542>
- Aliyah, A., Hartono, N., & Muin, A. A. (2025). Penggunaan User Acceptance Testing (UAT) pada pengujian sistem informasi pengelolaan keuangan dan inventaris barang. *Switch: Jurnal Sains Dan Teknologi Informasi*, 3(2), 42–58.
- Amrozi, Y., Wardani, S. D. K., & Ramadhan, R. (2020). Quo Vadis Pengembangan Rekayasa Perangkat Lunak. *Jurnal Teknologi Terapan: G-Tech*, 3(2), 237–244. <https://doi.org/10.33379/gtech.v3i2.427>
- Apriliando, A. (2021). Implementasi framework laravel pada rancang bangun website iakn palangka raya dengan metode prototype: implementation of the laravel framework in the website design of iakn palangka raya with the prototype method. *Jurnal Sains Komputer Dan Teknologi Informasi*, 3(2), 87–96.
- Arifiyanto, T. (2011). Membuat Interface aplikasi android lebih keren dengan LWUIT. *Yogyakarta: Andi Publisher*.
- As, R., & Shalahudin, M. (2021). *Rekayasa perangkat lunak terstruktur dan berorientasi objek*.
- Asosiasi Penyelenggara Jasa Internet Indonesia. (2025). *Laporan Survei Penetrasi dan Perilaku Internet Indonesia*. <https://survei.apjii.or.id/survei/group/11>
- Bakri, S. N., & Nasution, M. I. P. (2024). Penerapan Metodologi Rekayasa Perangkat Lunak untuk Efisiensi Pengembangan Sistem. *JSITIK: Jurnal Sistem Informasi Dan Teknologi Informasi Komputer*, 3(1), 53–66. <https://doi.org/10.53624/jsitik.v3i1.542>
- Buwono, R. C. (2019). *Web Services Menggunakan Format JSON*. XIV, 1–10.
- Cahyaningtyas, R., & Iriyani, S. (2014). Perancangan Sistem Informasi Perpustakaan Pada Smp Negeri 3 Tulakan, Kecamatan Tulakan Kabupaten Pacitan. *Indonesian Journal of Networking and Security (IJNS)*, 4(2).
- Connolly, T. M., & Begg, C. E. (2015). *Database systems: a practical approach to design, implementation, and management*. Pearson Education.
- Dessler, G. . C. N. & C. N. (2015). *Manajemen sumber daya manusia: Hal-hal penting. Management of Human Resources: The Essentials*. London: Pearson,.
- Elmasri, R., & Navathe, S. B. (2016). The Relational Data Model and Relational Database Constraints. In *Fundamentals of Database Systems* (Issue September 2019).
- Hendini, A. (2016). Pemodelan Uml Sistem Informasi Monitoring Penjualan Dan Stok Barang. *Jurnal Khatulistiwa Informatika*, 2(9), 107–116. <https://doi.org/10.1017/CBO9781107415324.004>
- Herdiansah, A. (2024). Pemanfaatan Framework Laravel dalam pengembangan Sistem Job Tiket & Monitoring Kinerja Teknisi Berbasis Android. *Prosiding Seminar Nasional Teknik Elektro, Sistem Informasi, Dan Teknik Informatika (SNESTIK)*, 1(1), 251–261.
- Hermawan, R., & Hidayat, A. (2016). Sistem Informasi Penjadwalan Kegiatan Belajar Mengajar

- Berbasis Web (Studi Kasus: Yayasan Ganesha Operation Semarang). *IJSE-Indonesian Journal on Software Engineering*, 2(1).
- Heryana, A. (2024). Ade Heryana | Sistem: Pengertian dan Karakteristik | September 2024. September, 1–7.
- Indriana, A. (2021). Kebijakan Pemerintah Terhadap Perlindungan Hak Pekerja Freelance (Harian Lepas) Di Indonesia. *Investama: Jurnal Ekonomi Dan Bisnis*, 5(2), 120–134.
- Kasman, A. D. (2013). Kolaborasi Dahsyat Android dengan PHP dan MySQL. *Yogyakarta: Lokomedia*.
- Manajemen, P. (2014). *Ketenagakerjaan disekolah*.
- Miftahuljannah, V., & Suharso, A. (2023). Pengimplementasian Berbagai Web Berdasarkan Kebutuhan Pengguna Dengan Menggunakan Metode Systematic Literature Review. *INFOTECH Journal*, 9(2), 401–405. <https://doi.org/10.31949/infotech.v9i2.6341>
- Najibulloh Muzaki, M., Karaman, J., & Zakur, Y. (n.d.). *Enhancing RESTful API Authentication with Cryptography in Student Information Systems*.
- Nazruddin Safaat, H. (2012). *Android: Pemrograman Aplikasi Mobile Smartphone dan Tablet PC Berbasis Android (Edisi Revisi)*. Android.
- Nurhayati, E., & Agussalim, A. (2023). Rancang Bangun Back-end API pada Aplikasi Mobile AyamHub Menggunakan Framework Node JS Express. *JUSTIN (Jurnal Sistem Dan Teknologi Informasi)*, 11(3), 524–531.
- Pressman, R. S. (2015a). *Rekayasa perangkat lunak: Pendekatan praktisi buku I*. Yogyakarta: Andi.
- Pressman, R. S. (2015b). *Software engineering: a practitioner's approach*. Palgrave macmillan.
- Renaldi, R., Cahyo Santoso, B., Natasya, Y., Willian, S., & Alfando, F. (2020). Tinjauan Pustaka Sistematis terhadap Basis Data MongoDB. *Jurnal Inovasi Informatika*, 5(2), 132–142. <https://doi.org/10.51170/jii.v5i2.79>
- Saifudin, I., Suharso, W., & Mubaroq, S. (2026). Website-Based STEM Learning Using Encyclopedias for Students with Special Needs. *JUSTINDO (Jurnal Sistem Dan Teknologi Informasi Indonesia)*, 11(1), 55–64.
- Setiawan, A., & Purnamasari, A. I. (2020). Implementasi JSON Web Token Berbasis Algoritma SHA-512 untuk Otentikasi Aplikasi BatikKita. *Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi)*, 4(6), 4–10. <https://doi.org/10.29207/resti.v4i6.2533>
- Shalahuddin, M., & Rosa, A. S. (2013). *Rekayasa perangkat lunak terstruktur dan berorientasi objek*. Bandung: Informatika.
- Silitonga, D. (2018). Pengaruh Gig Ekonomi, Freelance, Dan Kerja Kantoran di Masa Depan. *Diakses Dari Website Journal Morselo: <https://Journal.Moselo.Com/Pengaruh-Gig-Ekonomi-Freelance-Dan-Kerja-Kantor-Di-Masa-Depan-55ccdbd0ef40>*. Agustus, 7, 2020.
- Sis.binus.ac.id. (2022). *JSONObject and JSONArray*. School of Information Systems BINUS UNIVERSITY School of Information Systems. <https://sis.binus.ac.id/2022/02/22/jsonobject-and-jsonarray/>
- Solichin, A. (2016). *Pemrograman web dengan PHP dan MySQL*. Penerbit Budi Luhur.

- Sommerville, I. (2011). *YAZILIM MÜHENDİSLİĞİ-Software Engineering*. Nobel Akademik Yayıncılık.
- Soufitri, F. (2023). *Konsep sistem informasi*. PT Inovasi Pratama Internasional.
- Stallings, W. (2017). *Cryptography and Principles and Practice Seventh Edition*.
- Sulistyo, H. W., Oktavianto, H., Arifin, Z., Wardoyo, A. E., & Fitriyah, N. Q. (2024). PEMANFAATAN API PADA APLIKASI PORTAL BERITA SEDERHANA. *Jurnal Teknologi Dan Ilmu Komputer Prima (JUTIKOMP)*, 7(1), 88–97.
- Triawan, A., & Siboro, A. R. Y. (2021). Penerapan Application Programming Interface (API) Pada Push Notification Untuk Informasi Monitoring Stok Barang Minim. *TeknoIS: Jurnal Ilmiah Teknologi Informasi Dan Sains*, 11(2), 107–114.
- Wahyudi, T. (2022). Pengembangan Aplikasi Berbasis Web dan Android Sebagai Penunjang Kerja di Indonesia: Systematic Literature Review. *Indonesian Journal Computer Science*, 1(2), 96–102.
- Wijaya, Y. D., & Astuti, M. W. (2021). Pengujian Blackbox Sistem Informasi Penilaian Kinerja Karyawan Pt Inka (Persero) Berbasis Equivalence Partitions. *Jurnal Digital Teknologi Informasi*, 4(1), 22. <https://doi.org/10.32502/digital.v4i1.3163>
- Xie, X., Han, Y., Anderson, A., & Ribeiro-Navarrete, S. (2022). Digital platforms and SMEs' business model innovation: Exploring the mediating mechanisms of capability reconfiguration. *International Journal of Information Management*, 65, 102513.
- Yuhefizar, C. M., & Membangun, M. (2013). Mengelola Website. *Graha Ilmu, Yogyakarta*.
- Zuraidah, D. N., Apriyadi, M. F., & Fatoni, A. R. (2021). Menelisik Platform Digital Dalam Teknologi Bahasa Pemrograman. *Teknois: Jurnal Ilmiah Teknologi Informasi Dan Sains*, 11(2), 1–6.